Práctica de programación de aplicaciones

Sistemas Operativos 2026/2027

Jesús Torres <jmtorres@ull.es>

Universidad de La Laguna

Contenidos

Presentación **8**

1. Manipulación de archivos **9**
1.1. Objetivo . . . 9
1.2. Línea de comandos y funcionamiento básico . . . 9
1.3. Manejo de errores . . . 10
1.4. Implementación . . . 11
1.4.1. Comprobar los argumentos de la línea de comandos . . . 11
1.4.2. Comprobar si `DESTINO` es un directorio . . . 11
1.4.3. Construir la ruta de destino dentro de un directorio . . . 12
1.4.4. Comprobar si `ORIGEN` y `DESTINO` son el mismo archivo . . . 13
1.4.5. Copiar el archivo . . . 13
1.5. Comprobación . . . 15
1.5.1. Casos de prueba adicionales . . . 15

2. Procesos y redirecciones de entrada/salida **17**
2.1. Objetivo . . . 17
2.2. Línea de comandos y funcionamiento . . . 18
2.3. Arquitectura de compresión . . . 19
2.4. Implementación . . . 21
2.4.1. Procesamiento de argumentos de línea de comandos . . . 22
2.4.2. Obtener el comando de compresión . . . 23
2.4.3. Localizar el comando de compresión . . . 23
2.4.4. Extensiones de archivo . . . 25
2.4.5. Función copy_file_compressed() . . . 25
2.5. Manejo de errores . . . 28
2.6. Comprobación . . . 29
2.6.1. Casos de prueba adicionales . . . 30

3. Concurrencia y manejo síncrono de señales **31**
3.1. Objetivo . . . 31
3.2. Línea de comandos y funcionamiento . . . 32
3.3. Arquitectura concurrente . . . 33
3.4. Cancelación cooperativa . . . 35
3.4.1. Dónde se comprueba la cancelación . . . 35
3.4.2. Qué hay que limpiar . . . 36
3.4.3. Cancelar una copia con compresión . . . 36
3.5. Informe de progreso . . . 36

3.6. Manejo de errores . . . 37
3.7. Implementación . . . 38
3.7.1. Construir la ruta de destino de cada archivo . . . 39
3.7.2. Bloquear las señales . . . 40
3.7.3. Esperar y atender las señales . . . 40
3.7.4. Copiar el archivo . . . 41
3.7.5. Copiar el archivo con compresión . . . 41
3.7.6. El hilo de copia . . . 42
3.7.7. El orden de los hilos en `main()` . . . 43
3.8. Comprobación . . . 43
3.8.1. Casos de prueba adicionales . . . 44

4. Comunicación entre procesos con sockets **46**
4.1. Objetivo . . . 46
4.2. Línea de comandos y funcionamiento . . . 47
4.3. Arquitectura del sistema . . . 49
4.3.1. El directorio de ejecución . . . 49
4.3.2. El protocolo . . . 51
4.3.3. Esperar sin bloquearse . . . 51
4.4. Manejo de errores . . . 51
4.5. Implementación . . . 52
4.5.1. El servidor . . . 52
4.5.2. El cliente . . . 54
4.5.3. El socket del servidor . . . 55
4.5.4. La conexión del cliente . . . 56
4.5.5. Enviar y recibir mensajes . . . 56
4.5.6. Esperar con un plazo . . . 57
4.5.7. El archivo del PID . . . 57
4.5.8. El hilo de copia del servidor . . . 58
4.5.9. El hilo de envío del cliente . . . 60
4.5.10. Convertir la ruta en absoluta . . . 61
4.5.11. Informe de progreso . . . 61
4.6. Comprobación . . . 62
4.6.1. Casos de prueba adicionales . . . 63

5. Sockets TCP/IP y servidores concurrentes **66**
5.1. Objetivo . . . 66
5.2. Línea de comandos y funcionamiento . . . 67
5.3. Arquitectura del sistema . . . 68
5.3.1. Dos transportes a la vez . . . 68
5.3.2. El protocolo . . . 70
5.3.3. El servidor concurrente . . . 71
5.4. Manejo de errores . . . 71
5.5. Implementación . . . 72
5.5.1. La configuración . . . 72
5.5.2. El servidor . . . 74

5.5.3. El cliente . . . 75
5.5.4. Los sockets del servidor . . . 76
5.5.5. La conexión del cliente . . . 76
5.5.6. Esperar por varios descriptores . . . 77
5.5.7. Enviar y recibir archivos . . . 78
5.5.8. El hilo que acepta conexiones . . . 79
5.5.9. El hilo de cada conexión . . . 80
5.5.10. El hilo de envío del cliente . . . 81
5.5.11. Informe de progreso . . . 82
5.6. Comprobación . . . 82
5.6.1. Casos de prueba adicionales . . . 83

6. Gestión automática de los recursos del sistema 85
6.1. Objetivo . . . 85
6.2. Qué cambia y qué no . . . 86
6.3. Los recursos tienen dueño . . . 86
6.3.1. El descriptor es un *handle* . . . 86
6.3.2. Lo que `std::jthread` hace . . . 87
6.4. La clase `SafeFD` . . . 88
6.4.1. Cerrar antes de tiempo . . . 90
6.5. Manejo de errores . . . 91
6.6. Implementación . . . 92
6.6.1. Una función que crea un descriptor . . . 92
6.6.2. Una función que solo usa el descriptor . . . 93
6.6.3. Una función que se queda con el descriptor . . . 94
6.6.4. El resto del programa . . . 95
6.7. Comprobación . . . 96
6.7.1. Casos de prueba adicionales . . . 96

Apéndices 97

A. Operaciones con archivos con la API POSIX 97
A.1. Descriptores de archivo . . . 97
A.1.1. Identificación de recursos . . . 97
A.1.2. Descriptores de archivo en POSIX . . . 98
A.1.3. Descriptores estándar . . . 98
A.1.4. Descriptores y otros recursos del sistema . . . 99
A.2. Gestión de errores en la API POSIX . . . 99
A.2.1. Convención de valores de retorno . . . 99
A.2.2. Valores de `errno` . . . 100
A.2.3. Lectura inmediata de `errno` . . . 101
A.3. Abrir archivos . . . 102
A.3.1. Opciones de apertura . . . 103
A.3.2. Permisos de los nuevos archivos . . . 103
A.3.3. Permisos y máscara umask . . . 104

A.4. Cerrar descriptores de archivo . . . 104
A.5. Leer datos de archivos . . . 105
A.5.1. Consideraciones importantes . . . 105
A.6. Escribir datos en archivos . . . 106
A.7. Acceder a los atributos de un archivo . . . 107
A.7.1. Comprobar si dos archivos son el mismo . . . 108
A.7.2. Acceso a los permisos y al tipo de archivo . . . 108
A.7.3. Comprobar el acceso a un archivo . . . 109
A.8. Eliminar un archivo . . . 110

B. Gestión de errores en C++ 112
B.1. Valores que pueden faltar con `std::optional` . . . 112
B.2. Propagación de errores con `std::expected` . . . 114
B.2.1. Funciones con valor de retorno . . . 114
B.2.2. Funciones sin valor de retorno . . . 115
B.3. Información de error enriquecida . . . 116
B.3.1. Una función auxiliar para no repetirse . . . 118
B.4. Forzar la comprobación de errores con `[[nodiscard]]` . . . 119
B.5. Excepciones . . . 120
B.5.1. Qué ocurre cuando se lanza una excepción . . . 120
B.5.2. De dónde salen las excepciones si no las lanzamos . . . 121
B.5.3. Capturar excepciones . . . 121
B.5.4. El patrón `protected_main()` . . . 122

C. Tuberías anónimas 124
C.1. Creación de tuberías anónimas . . . 124
C.1.1. Uso básico de tuberías anónimas . . . 125
C.2. Comunicación entre procesos con tuberías . . . 126
C.3. Escribir en una tubería rota . . . 128
C.4. Redirección con tuberías . . . 129
C.4.1. Redirigir la entrada y la salida estándar . . . 129
C.5. Patrón completo: Pipe + Fork + Exec + Redirección . . . 130
C.6. Redirección bidireccional . . . 132

D. Gestión de procesos 133
D.1. Creación de procesos con `fork()` . . . 133
D.1.1. Cómo debe terminar el proceso hijo . . . 134
D.1.2. Descriptores de archivo compartidos . . . 135
D.2. Reemplazo de programa con `exec()` . . . 136
D.2.1. Diferencias entre las variantes de `exec()` . . . 137
D.2.2. Ejemplo con execvp() . . . 139
D.3. Espera de procesos hijos con wait() . . . 140
D.3.1. Interpretar el estado de salida . . . 140
D.3.2. Códigos de salida al ejecutar un comando . . . 141
D.3.3. Importancia de `wait()` . . . 142

D.4. Redirección de entrada/salida 142
D.4.1. Ejemplos de redirección a archivos 142
D.4.2. Redirección con `fork()` y `exec()` 143

E. Procesamiento de la línea de comandos **146**
E.1. Argumentos de la línea de comandos 146
E.2. Función `parse_arguments()` 148
E.3. Procesamiento con `getopt()` 151
E.3.1. Variables globales de `getopt()` 151
E.3.2. Cadena de opciones `optstring` 151
E.3.3. Valor de retorno 152
E.3.4. Uso básico de `getopt()` 152
E.3.5. Implementación completa de `parse_arguments()` con `getopt()` . . 153

F. Variables de entorno **155**
F.1. Consultar una variable 155
F.2. Definir y eliminar variables 155
F.3. Recorrer todo el entorno 156

G. Hilos **158**
G.1. Crear hilos y esperar a que terminen 158
G.2. Repartir el trabajo entre varios hilos 160
G.3. Compartir datos entre hilos 161
G.4. Cancelar hilos 162
G.5. Excepciones en los hilos 164
G.6. Hilos y procesos 165
G.7. Hilos y señales 165
G.7.1. Esperar las señales con un plazo 166
G.7.2. Separar el hilo 167

H. Señales POSIX **169**
H.1. Características de las señales 169
H.2. Señales comunes 169
H.3. Manejo de señales 170
H.3.1. Configurar manejadores con sigaction() 170
H.3.2. Precauciones con manejadores de señales 172
H.3.3. Patrón típico de terminación segura 172
H.3.4. Señales y llamadas al sistema 173
H.4. Bloquear señales 174
H.4.1. Conjuntos de señales 174
H.4.2. Bloquear y desbloquear con sigprocmask() 175
H.4.3. Consultar las señales pendientes 176
H.5. Esperar señales de forma síncrona 177
H.6. Enviar señales 179
H.7. Verificar si un proceso existe 180

I. Operaciones con sockets **181**
I.1. Crear un socket . . . 181
I.2. Direcciones de socket . . . 182
I.2.1. Direcciones `AF_INET` . . . 182
I.2.2. Direcciones `AF_UNIX` . . . 183
I.3. Asignar una dirección al socket . . . 183
I.4. Enviar y recibir datagramas . . . 184
I.5. Sockets orientados a conexión . . . 186
I.5.1. Escuchar conexiones . . . 186
I.5.2. Aceptar conexiones . . . 186
I.5.3. Conectar con un servidor . . . 187
I.5.4. Enviar y recibir datos . . . 187
I.5.5. Cerrar media conexión . . . 188
I.6. Esperar con un plazo . . . 188
I.7. Cerrar un socket . . . 189

Presentación

Este documento reúne el guión y los contenidos de la práctica de programación de aplicaciones de la asignatura. Cada capítulo es una unidad, y a lo largo del curso se irá indicando cuál toca trabajar en cada momento.

Las unidades no son ejercicios independientes. En la primera se escribe un programa que copia archivos, y cada una de las siguientes lo amplía con conceptos nuevos, hasta convertirlo en un sistema de copias de seguridad. Por eso conviene conservar el código de una unidad para la siguiente, y por eso hay que hacerlas en orden.

Los apéndices del final documentan las llamadas al sistema y las bibliotecas que hacen falta para resolver cada unidad. No están pensados para leerse de principio a fin, sino para consultarlos cuando el enunciado remite a ellos.

1. Manipulación de archivos

Vamos a aprender a manipular archivos utilizando directamente las llamadas al sistema del sistema operativo, mientras desarrollamos un programa que copia archivos entre diferentes rutas del sistema de archivos. Aprenderemos a usar `open()`, `read()`, `write()` y `close()` para leer y escribir archivos, a consultar sus metadatos con `stat()` y a gestionar los errores en C++ con `std::expected`.

1.1. Objetivo

Vamos a crear un programa llamado `copy` que copia el contenido de un archivo en otra ubicación. El programa debe cumplir con los siguientes requisitos:

- La ruta del archivo de origen y la ruta del archivo de destino se indicarán como argumentos en la línea de comandos al ejecutar el programa.
- Podrá copiar archivos de cualquier tamaño, incluyendo archivos muy grandes –aunque no quepan completamente en memoria– o muy pequeños.
- Para manipular los archivos usará, exclusivamente, las llamadas al sistema `open()`, `read()`, `write()`, `stat()`, `close()` y similares, tal y como se explica en el Apéndice A.
- Antes de finalizar, el programa cerrará todos los descriptores de archivo abiertos, liberará correctamente cualquier otro recurso reservado y finalizará con el código `EXIT_SUCCESS` (`0`).

> **!** Importante
>
> Para manipular los archivos no se pueden usar funciones de más alto nivel como `std::ifstream`, `std::ofstream`, `std::fstream`, `std::FILE`, `fopen()`, `fread()`, `fwrite()`, `fclose()` o similares. Hacerlo es motivo de suspenso en la práctica, porque el objetivo es aprender a usar las llamadas al sistema sobre las que estas funciones están construidas.

1.2. Línea de comandos y funcionamiento básico

El programa `copy` acepta los siguientes argumentos de línea de comandos:

```
copy ORIGEN DESTINO
```

Cuando se ejecuta con estos dos argumentos:

- El programa debe copiar el contenido del archivo en la ruta `ORIGEN` a la ruta del archivo `DESTINO`.
- Si `DESTINO` ya existe y es un archivo, debe sobrescribirse.
- Si `DESTINO` ya existe y es un directorio, el programa debe copiar `ORIGEN` dentro de ese directorio.
- Si `DESTINO` no existe, debe crearse el archivo para hacer la copia.

Si no se indican exactamente estos dos argumentos, el programa debe mostrar un mensaje de error indicando cómo debe usarse el programa y terminar con un código de salida de error:

```
$ copy /tmp/testfile.txt
copy: se deben indicar los archivos ORIGEN y DESTINO
```

1.3. Manejo de errores

Las funciones que dan acceso a los servicios y recursos del sistema pueden fallar por diversos motivos. Por eso debemos comprobar y tratar cualquier condición de error posible.

En todas las situaciones de error:

- El programa debe mostrar un mensaje de error por la **salida de error**.
- El programa debe terminar con un código de salida distinto de `EXIT_SUCCESS` (`0`).
- Las funciones que implementemos deben retornar la condición de error a quien las llamó, en lugar de mostrar ellas el mensaje, pues de esta forma son más reutilizables.
- Como regla general, los mensajes de error deben imprimirse en `main()` o lo más cerca posible de `main()`, antes de terminar el programa. Estos mensajes deben incluir el nombre del programa, el contexto del error y el mensaje de error que proporciona el sistema operativo, tal y como se muestra en los ejemplos de la Sección 1.5.
- La terminación debe ocurrir retornando desde `main()` con `return`, nunca usando `std::exit()`, `exit()` ni funciones similares, para seguir las buenas prácticas recomendadas en C++.

Todos los errores de esta práctica se retornan usando `std::expected`, nunca lanzando excepciones, según se explica en la Sección B.2. Aun así, conviene saber reconocer las excepciones, porque algunas funciones de la librería estándar informan de sus errores lanzándolas, y conviene añadir a `main()` una red de seguridad para capturar las que puedan escaparse, como se describe en la Sección B.5.

1.4. Implementación

El flujo de ejecución del programa es muy sencillo, como ilustra el siguiente pseudocódigo:

```
if not check_args(argc, argv):
    error("argumentos inválidos")

if is_directory(destino):
    destino = destino / filename(origen)

if is_same_file(origen, destino):
    error("ORIGEN y DESTINO son el mismo archivo")

result = copy_file(origen, destino)
if has_error(result):
    error("error al copiar archivo: {result.error}")
```

En este pseudocódigo se indican las funciones que se deben implementar y cuándo se deben llamar. Cuando se indica `error("...")`, significa que se aplicará el manejo de errores descrito en la Sección 1.3.

Vamos a ver cada uno de estos pasos y las funciones indicadas con más detalle.

1.4.1. Comprobar los argumentos de la línea de comandos

Para comprobar las condiciones sobre el número de argumentos indicados en la línea de comandos, implementa la siguiente función:

```
bool check_args(int argc, char* argv[]);
```

A la que se le pasan los parámetros `argc` y `argv` recibidos en `main()`. Esta función debe devolver `true` si los argumentos son correctos y `false` en caso contrario.

Para que los argumentos sean correctos, deben indicarse exactamente dos argumentos, además del nombre del programa en `argv[0]`.

Como esta función es muy específica de este programa, puede mostrar directamente el mensaje de error adecuado en caso de que los argumentos no sean correctos.

1.4.2. Comprobar si `DESTINO` es un directorio

Para comprobar si el archivo indicado en `DESTINO` es un directorio, implementa la siguiente función:

```
bool is_directory(const std::string& path);
```

Esta función debe devolver `true` si la ruta `path` corresponde a un directorio y `false` en caso contrario. Para comprobarlo se puede usar la función `stat()` descrita en la Sección A.7.

1.4.3. Construir la ruta de destino dentro de un directorio

Cuando `DESTINO` es un directorio, necesitamos obtener el nombre del archivo de `ORIGEN` para construir la ruta del archivo que se creará dentro de dicho directorio. Es decir, si `ORIGEN` es `/home/user/document.txt` y `DESTINO` es `/tmp/`, debemos construir la ruta `/tmp/document.txt` para copiar el archivo dentro del directorio de destino.

Para trabajar con rutas, la librería estándar de C++ proporciona la clase `std::filesystem::path`, declarada en la cabecera `<filesystem>`:

- El método `filename()` devuelve el último componente de la ruta, es decir, el nombre del archivo.
- El operador `/` –y su versión de asignación `/=`– concatena dos rutas, añadiendo entre ellas el separador `/` solo si es necesario.
- El método `string()` devuelve la ruta como un `std::string`, que es el tipo que esperan las funciones que vamos a implementar.

Por ejemplo:

```
#include <filesystem>

std::filesystem::path origen{"/home/user/document.txt"};
std::filesystem::path destino{"/tmp"};

destino /= origen.filename();                                    ①
std::string ruta = destino.string();
```

① `destino` pasa a ser `/tmp/document.txt`. El resultado es el mismo si `DESTINO` se indicó como `/tmp/`, porque el separador solo se añade si es necesario.

> **!** Importante
>
> De la cabecera `<filesystem>` solo se pueden usar en esta práctica dos cosas: la clase `std::filesystem::path`, para **manipular rutas** –una operación sobre cadenas de caracteres–, y la función `std::filesystem::canonical()`, que se utilizará más adelante para convertir rutas relativas en absolutas. El resto de funciones de `std::filesystem` –como `std::filesystem::exists()`, `std::filesystem::is_directory()` o `std::filesystem::copy_file()`– no se pueden utilizar, porque consultan o modifican el sistema de archivos, haciendo precisamente lo que estamos aprendiendo a hacer con las

llamadas al sistema.

El estándar POSIX define las funciones `basename()` y `dirname()` para obtener el nombre del archivo y el directorio de una ruta, respectivamente, pero presentan algunos problemas de diseño, por lo que es preferible usar `std::filesystem::path` o crear nuestras propias funciones.

1.4.4. Comprobar si `ORIGEN` y `DESTINO` son el mismo archivo

Para comprobar si las rutas de origen y destino conducen al mismo archivo, implementa la siguiente función:

```
bool is_same_file(
    const std::string& src_path,
    const std::string& dest_path);
```

Esta función debe devolver `true` si ambas rutas corresponden al mismo archivo y `false` en caso contrario, usando la técnica descrita en la Sección A.7.1.

> **!** Importante
>
> Esta comprobación debe hacerse **después** de construir la ruta de destino definitiva. Si se hiciera solo con los argumentos de la línea de comandos, una orden como `copy testfile.dat .` la superaría –porque `testfile.dat` y `.` son rutas distintas– y el programa acabaría abriendo el archivo de origen como archivo de destino, truncándolo antes de leerlo.

1.4.5. Copiar el archivo

Para copiar el archivo de `ORIGEN` a `DESTINO`, implementa la siguiente función:

```
std::expected<void, std::system_error> copy_file(
  const std::string& src_path, const std::string& dest_path,
  mode_t dst_perms = 0644);
```

Esta función debe abrir el archivo en `src_path` para lectura y el archivo en `dest_path` para escritura –creándolo con los permisos `dst_perms` si no existe, o sobrescribiéndolo si ya existe– y debe copiar todo el contenido del archivo de origen al archivo de destino. Para ello, debe usar un búfer de tamaño fijo para leer y escribir los datos en bloques de 128 KiB (131 072 bytes) hasta que todo el archivo haya sido copiado, tal y como se ilustra en la Figura 1.1.

El valor por defecto `0644` (`rw-r--r--`) es el habitual para los archivos que crea un programa. Ten en cuenta que los permisos reales del archivo dependen además de la máscara `umask` del proceso, tal y como se explica en la Sección A.3.3.

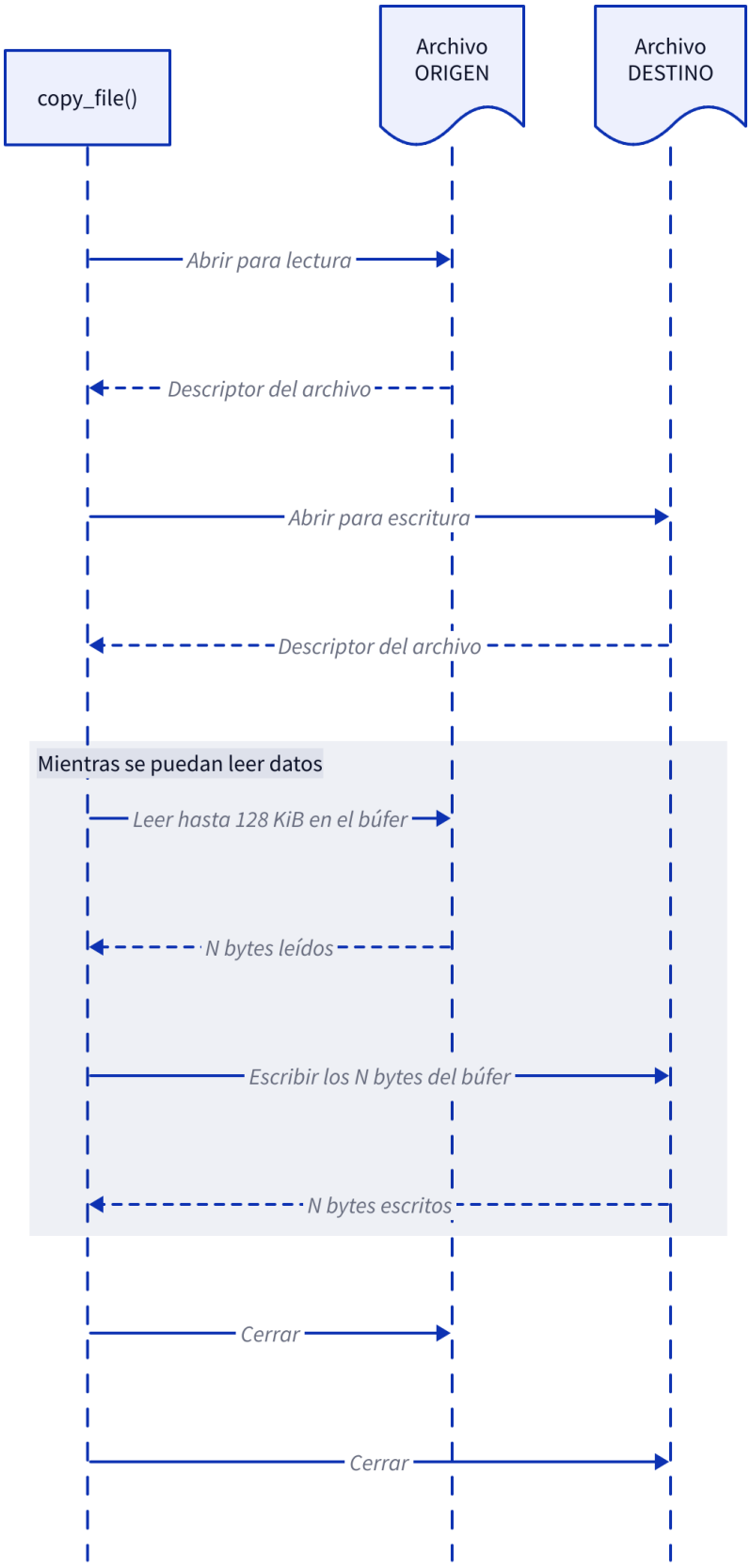


Figura 1.1.: Diagrama de secuencia del funcionamiento de la función `copy_file()`.

El uso de las funciones `open()`, `read()`, `write()` y `close()` está explicado en el Apéndice A.

La función debe retornar `std::expected<void, std::system_error>` para indicar si la operación de copia fue exitosa o si ocurrió algún error. En caso de error, debe devolver un objeto `std::system_error` con el código de error de `errno`. El uso de `std::expected` está descrito en la Sección B.2.

1.5. Comprobación

Para comprobar que `copy` funciona correctamente, podemos crear archivos con datos aleatorios generados desde el dispositivo `/dev/urandom`. Por ejemplo, usando el comando `dd` se puede crear un archivo de nombre `testfile.dat` con 16 MiB de datos aleatorios:

```
$ dd if=/dev/urandom of=testfile.dat bs=1M count=16 iflag=fullblock
```

Para comprobar que la copia funciona, primero usamos nuestro programa `copy` para copiar `testfile.dat` a `testfile2.dat`:

```
$ copy testfile.dat testfile2.dat
```

Y luego el comando `cmp` nos dirá si ambos archivos son iguales o diferentes:

```
$ cmp testfile.dat testfile2.dat
```

Si los archivos son iguales, `cmp` no mostrará nada y terminará con código de salida 0.

> **!** Importante
>
> Si los archivos tienen diferente tamaño, es porque seguramente el programa `copy` no ha copiado todo el contenido del archivo origen porque no ha manejado correctamente la situación en la que `read()` lee menos bytes de los solicitados (ver la Sección A.5.1).

1.5.1. Casos de prueba adicionales

Adicionalmente, es importante probar los siguientes escenarios:

- **Intentar copiar un archivo sin indicar el destino**:

```
$ copy testfile.dat
copy: se deben indicar los archivos ORIGEN y DESTINO
```

- **Intentar copiar un archivo a sí mismo**:

```
$ copy testfile.dat testfile.dat
copy: ORIGEN y DESTINO son el mismo archivo
```

- **Intentar copiar un archivo al directorio en el que ya está**:

```
$ copy testfile.dat .
copy: ORIGEN y DESTINO son el mismo archivo
```

- **Intentar copiar un archivo que no existe**:

```
$ copy noexiste.dat destino.dat
copy: error al abrir el archivo de origen: No such file or directory
```

- **Intentar copiar un archivo a un directorio sin permisos de escritura**:

```
$ copy testfile.dat /
copy: error al abrir el archivo de destino: Permission denied
```

- **Intentar copiar un archivo sin permisos de lectura**:

```
$ chmod 000 testfile.dat
$ copy testfile.dat destino.dat
copy: error al leer el archivo de origen: Permission denied
```

2. Procesos y redirecciones de entrada/salida

Vamos a extender el programa `copy` de la Unidad 1 para que pueda guardar la copia comprimida, aplicando conocimientos de creación de procesos, tuberías anónimas y redirecciones de entrada/salida. Aprenderemos a usar `fork()`, `exec()`, `pipe()` y `dup2()` para ejecutar comandos externos y comunicarnos con ellos.

2.1. Objetivo

Los sistemas operativos tipo UNIX traen instalados comandos que ya resuelven muchos problemas, así que es habitual que un programa, en lugar de reimplementarlos, los ejecute y aproveche su resultado. Vamos a hacerlo con `copy`, extendiéndolo para que pueda guardar la copia comprimida sin implementar nosotros ningún algoritmo de compresión.

El programa debe:

- Aceptar opciones en la línea de comandos para indicar el formato de compresión deseado.
- Crear un proceso hijo que ejecute el comando de compresión correspondiente (`gzip`, `bzip2` o `xz`), en lugar de copiar el archivo directamente como hasta ahora.
- Usar una tubería anónima para enviarle al proceso hijo el contenido del archivo de origen.
- Redirigir la entrada y la salida estándar del proceso hijo para conectarlo con la tubería y con el archivo de destino.
- Añadir al archivo de destino la extensión que corresponda al formato de compresión.
- Comprobar, antes de empezar a copiar, que el comando de compresión solicitado está instalado en el sistema.
- Seguir comportándose exactamente como en la Unidad 1 cuando no se indica ninguna opción de compresión.
- Terminar sin dejar ningún descriptor de archivo abierto ni ningún proceso hijo sin esperar.

2.2. Línea de comandos y funcionamiento

El programa `copy` acepta ahora las siguientes opciones:

```
copy [-z | -j | -x] ORIGEN DESTINO
```

`-z` (opcional) Comprime la copia usando **gzip** (extensión `.gz`).
`-j` (opcional) Comprime la copia usando **bzip2** (extensión `.bz2`).
`-x` (opcional) Comprime la copia usando **xz** (extensión `.xz`).
`ORIGEN` y `DESTINO` Las rutas de los archivos de origen y destino, con el mismo significado que en la Unidad 1. Es decir, si `DESTINO` es un directorio, la copia se hace dentro de él con el nombre del archivo de origen.

Las opciones de compresión son **mutuamente excluyentes**. Es decir, solo se puede indicar una a la vez; indicar más de una debe producir un error.

Cuando se indica una opción de compresión, la extensión correspondiente se añade **al final** de la ruta de destino que resulte:

```
$ copy -z testfile.dat /tmp/copia.dat
$ ls /tmp/copia.dat.gz
/tmp/copia.dat.gz
```

Estos son algunos ejemplos de uso correcto del programa:

```
# Sin compresión (comportamiento de la Unidad 1)
$ copy testfile.dat /tmp/copia.dat

# Con compresión gzip
$ copy -z testfile.dat /tmp/copia.dat

# Con compresión xz, copiando dentro de un directorio
$ copy -x testfile.dat /tmp/
```

Mientras que estos son algunos ejemplos de uso incorrecto, que deben producir mensajes de error:

```
# Múltiples opciones de compresión (ERROR)
$ copy -z -j testfile.dat /tmp/copia.dat
copy: error: solo se admite una opción de compresión
uso: copy [-z | -j | -x] ORIGEN DESTINO

# Opción inválida
$ copy -w testfile.dat /tmp/copia.dat
copy: error: opción desconocida
uso: copy [-z | -j | -x] ORIGEN DESTINO
```

2.3. Arquitectura de compresión

Cuando el programa se ejecuta con una opción de compresión, en lugar de copiar el archivo directamente al destino usando `copy_file()`, empleará una nueva función llamada `copy_file_compressed()` que debe crear un proceso hijo que ejecute el comando de compresión adecuado (`gzip`, `bzip2` o `xz`).

La comunicación entre el proceso padre –nuestro programa– y el proceso hijo –el comando de compresión– para enviarle los datos que debe comprimir se realiza mediante una **tubería anónima**.

Como se ilustra en la Figura 2.1, el sistema funciona de la siguiente manera:

1. El programa crea una tubería anónima con `pipe()` (ver la Sección C.1).
2. El programa crea un proceso hijo con `fork()` (ver la Sección D.1).
3. **Proceso hijo:**
   - Redirige `stdin` (descriptor `0`) al extremo de lectura de la tubería (`pipe[0]`) usando `dup2()` (ver la Sección D.4).
   - Abre el archivo de destino para escritura con `open()` como se indicó en la Sección 1.4.5. Deben usarse las opciones adecuadas de `open()` para que el archivo sea creado si no existe y truncado si existe.
   - Redirige `stdout` (descriptor `1`) al archivo de destino usando `dup2()`.
   - Cierra los descriptores originales que ya no son necesarios.
   - Ejecuta el comando de compresión con `execv()` (ver la Sección D.2).
4. **Proceso padre:**
   - Cierra los descriptores de archivo de la tubería que no son necesarios.
   - Lee del archivo origen en bloques de 128 KiB.
   - Escribe cada bloque en el extremo de escritura de la tubería (`pipe[1]`).
   - Cuando termina, cierra `pipe[1]` para señalar EOF al hijo.
   - Espera a que el hijo termine con `wait()` o `waitpid()` (ver la Sección D.3).
   - Verifica el código de salida del hijo para detectar errores.

> **!** Importante
>
> **El programa tiene que tener bloqueada la señal `SIGPIPE` mientras escribe en la tubería** para evitar que termine inesperadamente si el compresor termina antes de tiempo (ver la Sección C.3).

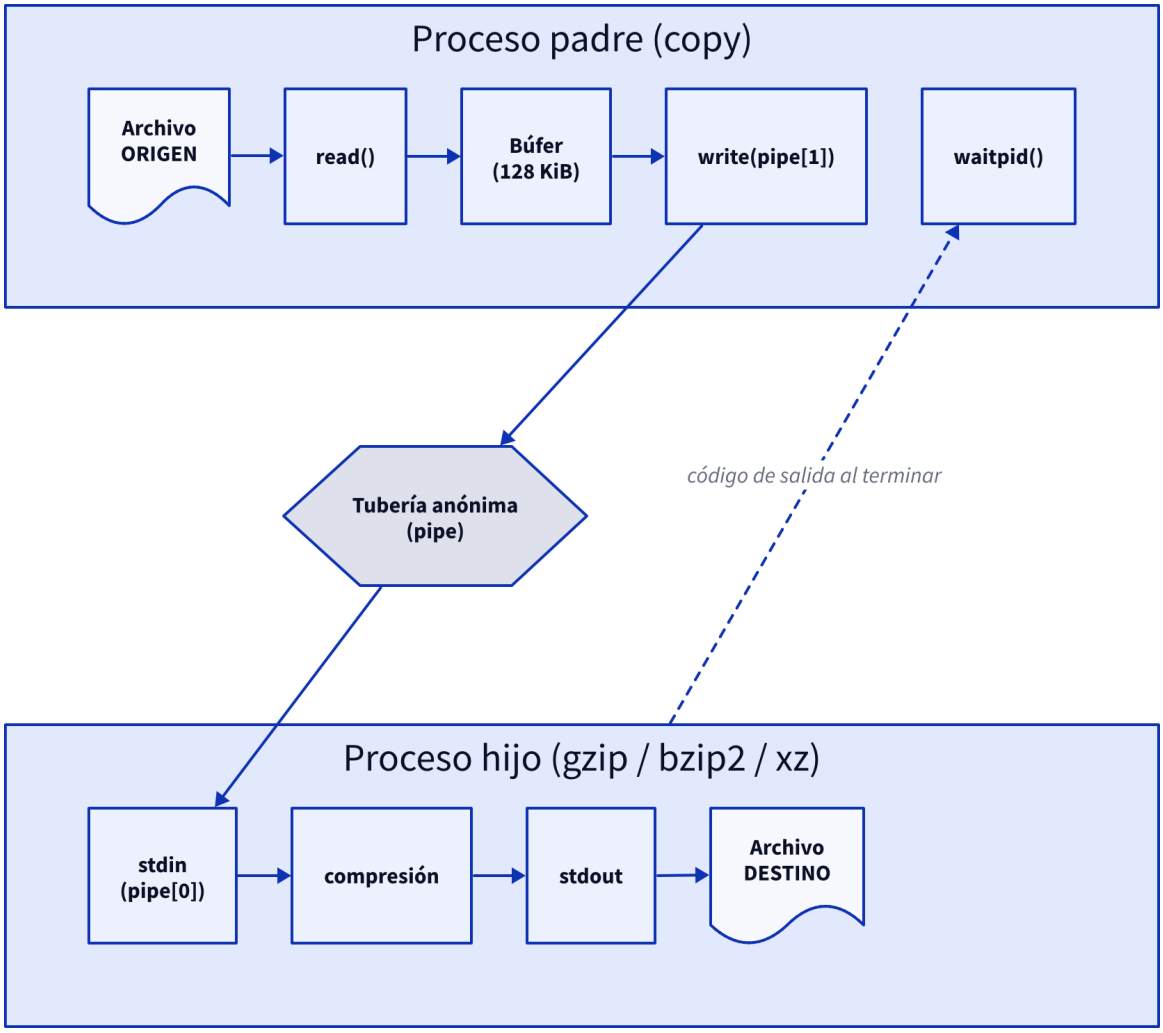


Figura 2.1.: Arquitectura de compresión usando un proceso hijo y una tubería anónima

2.4. Implementación

El flujo del programa es el mismo de la Unidad 1, con la decisión de qué función de copia usar al final:

```
options = parse_arguments(argc, argv)
if has_error(options):
    error("argumentos inválidos")

if compression_requested(options):
    comando = get_compression_command(options)
    compression_path = find_command(comando)                    ①
    if not has_value(compression_path):
        error("{comando} no está instalado")

if is_directory(destino):                          # Unidad 1
    destino = destino / filename(origen)

destino += get_compression_extension(options)                   ②

if is_same_file(origen, destino):                  # Unidad 1
    error("ORIGEN y DESTINO son el mismo archivo")

if compression_requested(options):
    result = copy_file_compressed(origen, destino, compression_path)
else:
    result = copy_file(origen, destino)

if has_error(result):
    error("error al copiar el archivo: {result.error}")
```

① El comando de compresión se busca antes de empezar, para que el programa falle enseguida si no está instalado en lugar de hacerlo a mitad de la copia.

② La extensión se añade después de resolver `DESTINO`, para que la reciba la ruta final calculada cuando `DESTINO` es un directorio. Sin compresión la extensión es la cadena vacía, así que la misma línea sirve en los dos casos.

Como en la Unidad 1, el pseudocódigo indica las funciones que hay que implementar y cuándo hay que llamarlas, y `error("...")` significa aplicar el manejo de errores descrito en la Sección 2.5. La comprobación `compression_requested()` no es una de esas funciones que hay que implementar. Simplemente la usamos en el pseudocódigo para indicar la acción de comparar la opción de compresión elegida con `CompressionType::NONE`.

Vamos a ver con más detalle las funciones nuevas.

2.4.1. Procesamiento de argumentos de línea de comandos

Ahora que el programa acepta opciones, la función `check_args()` de la Unidad 1 se queda corta y se sustituye por `parse_arguments()`:

```
std::expected<Options, ParseArgsError>
parse_arguments(int argc, char* argv[]);
```

Esta función debe:

- Analizar los argumentos de línea de comandos usando las técnicas descritas en el Apéndice E.
- Verificar que solo se especifica una opción de compresión.
- Verificar que se indican exactamente los dos argumentos `ORIGEN` y `DESTINO`.
- Retornar un objeto `Options` con la configuración del programa o un código de error `ParseArgsError`.

```
struct Options
{
    enum class CompressionType
    {
        NONE,
        GZIP,
        BZIP2,
        XZ
    };

    CompressionType compression = CompressionType::NONE;
    std::string src_path;
    std::string dest_path;
};
```

```
enum class ParseArgsError
{
    unknown_option,
    multiple_compression_options,
    missing_arguments,
    too_many_arguments,
    //
    // Aquí otros errores posibles de parse_arguments()...
    //
};
```

La función se usará al inicio del programa para procesar los argumentos. Por ejemplo:

```
int main(int argc, char* argv[])
{
    auto arguments_result = parse_arguments(argc, argv);

    if (!arguments_result.has_value())
    {
        switch (arguments_result.error())
        {
            case ParseArgsError::unknown_option:
                std::println(stderr,
                    "copy: error: opción desconocida");
                break;

            // Manejo del resto de posibles condiciones de error...
        }

        print_usage();
        return EXIT_FAILURE;
    }

    Options options = arguments_result.value();

    // Continuar con la copia...
}
```

2.4.2. Obtener el comando de compresión

Para facilitar la obtención del comando de compresión adecuado según la opción seleccionada, se implementará la siguiente función:

```
std::string get_compression_command(
    Options::CompressionType compression);
```

Simplemente debe devolver el nombre del comando correspondiente a cada posible valor de `Options::CompressionType`.

2.4.3. Localizar el comando de compresión

Antes de empezar la copia, el programa debe comprobar que el comando de compresión solicitado está instalado en el sistema y mostrar un error en caso contrario. Para ello se implementará la siguiente función:

```
std::optional<std::string> find_command(const std::string& command);
```

También se puede definir aprovechando las ventajas de `std::string_view`, porque su argumento nunca se pasa a una función de C.

```
std::optional<std::string> find_command(std::string_view command);
```

Esta función debe:

- Leer la variable de entorno `PATH`, tal y como se explica en el Apéndice F, usando `get_environment_variable()`, el envoltorio de `getenv()` que se presenta en la Sección B.1, y extraer las rutas que contiene, que es una lista de directorios separados por '`:`'. Si la variable no está definida no hay dónde buscar, así que la función retorna `std::nullopt`.
- Para cada ruta, construir la posible ruta absoluta del comando concatenándola con `/` + `command`. Por ejemplo, si en `PATH` está el directorio `/usr/bin` y el comando es `gzip` la posible ruta será `/usr/bin/gzip`.
- Comprobar si el comando existe en la ruta calculada usando `access()` con el flag `X_OK`, como se explica en la Sección A.7.3.

Si el comando existe en alguna de las rutas de `PATH` y es ejecutable, la función devuelve esa ruta absoluta. En caso contrario, devuelve `std::nullopt`.

Quedarnos con la ruta, en lugar de limitarnos a comprobar que el comando existe, permite ejecutarlo después con `execv()` en vez de con `execvp()`, sin tener que recorrer `PATH` una segunda vez.

El programa solo necesita llamarla una vez, con el nombre que devuelve `get_compression_command()`:

```
if (options.compression != Options::CompressionType::NONE)
{
    std::string comando =
        get_compression_command(options.compression);

    compression_path = find_command(comando);                    ①
    if (!compression_path)
    {
        std::println(stderr,
            "copy: error: {} no está instalado", comando);
        return EXIT_FAILURE;
    }
}
```

① La ruta encontrada se guarda para pasársela después a `copy_file_compressed()`.

2.4.4. Extensiones de archivo

El programa debe añadir la extensión apropiada al nombre del archivo de destino según el formato de compresión. Para eso, se propone crear una función que devuelva una cadena con la extensión adecuada para cada formato de compresión posible:

```
std::string get_compression_extension(
    Options::CompressionType compression);
```

Esta función retorna:

- `".gz"` para `Options::CompressionType::GZIP`
- `".bz2"` para `Options::CompressionType::BZIP2`
- `".xz"` para `Options::CompressionType::XZ`
- `""` (cadena vacía) para `Options::CompressionType::NONE`

> **!** Importante
>
> El valor devuelto por esta función se debe concatenar al final de la ruta del archivo de destino, antes de llamar a `copy_file_compressed()` para hacer la copia.

2.4.5. Función copy_file_compressed()

Esta es la función principal que realiza la copia con compresión:

```
std::expected<void, std::system_error> copy_file_compressed(
    const std::string& src_path,
    const std::string& dest_path,
    const std::string& compression_path);
```

`src_path` Ruta del archivo de origen.
`dest_path` Ruta del archivo de destino (ya con la extensión apropiada).
`compression_path` Ruta absoluta del comando de compresión, la que encontró `find_command()`. Esa misma ruta se pasa como `argv[0]` al ejecutar el comando con `execv()`.

La función debe implementar la arquitectura descrita en la Sección 2.3 usando `pipe()`, `fork()`, `dup2()`, `execv()`, `wait()` y las funciones de lectura/escritura de archivos vistas en la Unidad 1. El uso detallado de `fork()`, `exec()`, `wait()` y `dup2()` se explica en el Apéndice D. El uso de `pipe()` y la comunicación entre procesos mediante tuberías se explica en el Apéndice C.

2.4.5.1. Aspectos importantes

Respecto a la implementación de `copy_file_compressed()`, es importante no olvidar los siguientes **aspectos críticos**:

1. **Cierre de descriptores:** Cada proceso debe cerrar los extremos de la tubería que no usa.
   - Hijo cierra `pipe[1]` (escritura).
   - Padre cierra `pipe[0]` (lectura).
2. **Fin de la escritura:** El padre debe cerrar `pipe[1]` en cuanto termina de escribir el contenido del archivo de origen. Mientras siga abierto, el compresor se queda esperando más datos de entrada y nunca termina. Por tanto, si no se cierra `pipe[1]`, el padre se bloquearía para siempre en `wait()` (ver la Sección C.2).
3. **Manejo de errores en el hijo:** Si el proceso hijo encuentra errores, debe terminar con `_exit()`, siguiendo la estructura descrita en la Sección D.1.1. Respecto al código de salida al terminar el hijo en este caso:
   - Para cualquier error del hijo antes de llegar a ejecutar el comando –por ejemplo, que `open()` no pueda abrir el archivo de destino– usar el código `125`.
   - Si `exec()` falla con `EACCES` (permiso denegado) o `ENOEXEC` (formato ejecutable incorrecto), usar el código `126`.
   - Si `exec()` falla con `ENOENT` (archivo no encontrado), usar el código `127`.

   Son los códigos que la *shell* reserva para distinguir los fallos al lanzar un comando de los códigos de salida del propio comando (ver la Sección D.3).

   Además, ninguna excepción debe escapar del cuerpo del hijo. Aquí el desenrollado de la pila la llevaría fuera de `copy_file_compressed()` hasta el `try/catch` de `main()` –el patrón `protected_main()` de la Sección B.5–, que terminaría el hijo con `EXIT_FAILURE`. El padre recibiría entonces el código de salida `1` y no podría distinguir qué había fallado, perdiendo la información que codifican los códigos `125`, `126` y `127`.
4. **Espera del hijo:** El padre debe esperar con `wait()` o `waitpid()` para:
   - Evitar procesos zombies.
   - Verificar el código de salida del proceso hijo, detectar errores y comunicarlo al llamador de `copy_file_compressed()`.
5. **Redirecciones:** Usar `dup2()` para redirigir descriptores y luego cerrar los descriptores originales.
6. **Bloqueo de señales:** Lo más sencillo es bloquear `SIGPIPE` una sola vez, al principio del programa, y dejarla bloqueada mientras este se ejecuta, por el motivo que se explicó en la Sección 2.3.

7. **Máscara de señales del hijo:** El hijo hereda la máscara de señales del padre y la conserva a través de `exec()`, así que el comando de compresión se ejecutaría con `SIGPIPE` bloqueada, que no es lo que espera un programa lanzado desde la *shell*. Para evitarlo, entre el `fork()` y el `execv()` el hijo debe vaciar la máscara llamando a `sigprocmask(SIG_SETMASK, ...)` con un conjunto vacío, que es lo que también hace la *shell* con los programas que ejecuta.

2.4.5.2. Construcción del error devuelto

Observa que `copy_file_compressed()` devuelve `std::expected<void, std::system_error>`, el mismo tipo que `copy_file()` en la Unidad 1, de manera que el programa puede tratar igual el resultado de ambas. La diferencia está en que aquí no todos los errores proceden de una llamada al sistema que haya dejado un valor en `errno`:

- Los fallos de las llamadas que hace el padre –`pipe()`, `fork()`, `write()` o `waitpid()`– ocurren en nuestro propio proceso, así que se tratan exactamente igual que en `copy_file()`, construyendo el error a partir de `errno`.
- Los fallos del proceso hijo, en cambio, llegan a nuestro proceso convertidos en el código de salida que recoge `waitpid()`, así que hay que traducirlos de vuelta.

Para los primeros sirve `system_error_from_errno()`, la función auxiliar de la Sección B.3.1, que captura `errno` en el momento en que se detecta el error:

```
int pipe_fds[2];
if (pipe(pipe_fds) == -1)
{
    return system_error_from_errno("No se pudo crear la tubería");
}
```

Traducir los segundos consiste en convertir el código de salida del hijo –los `125`, `126` y `127` acordados en la Sección 2.4.5.1, o cualquier otro que devuelva el compresor– en un valor de `errno` con el que construir el `std::system_error` que vamos a devolver en `copy_file_compressed()`.

Cuando el hijo termina con el código `126` o `127`, el valor de `errno` es el que provocó el fallo de `exec()` en el hijo: `ENOENT` para el `127` y `EACCES` para el `126` –que el hijo usa también para `ENOEXEC`, pero desde el padre ya no podemos distinguir un caso del otro–:

```
return std::unexpected(std::system_error(ENOENT,
    std::system_category(),
    std::format("No se encontró el comando '{}'",
        compression_path)));
```

Esto nos deja con tres situaciones que no se corresponden con ningún valor de `errno` directamente: que el código de salida del hijo sea `125` –un error en el código que prepara al hijo, antes de llamar a `exec()`–, que el comando de compresión termine con un código de salida distinto de `0` o que sea terminado por una señal. En estos casos no conocemos la causa exacta del error, así que no podemos devolver un valor de `errno` que la describa, pero se puede tomar prestado un valor de `errno` genérico, como `EIO`, y dejar que sea el mensaje el que explique lo que ha ocurrido:

```
return std::unexpected(std::system_error(EIO,                                    ①
    std::system_category(),
    std::format("El comando '{}' terminó con el código {}",
        compression_path, exit_code)));                                          ②
```

① `EIO` no describe lo que ha pasado realmente, pero mantiene todos los errores del programa en la misma categoría, `std::system_category()`, como los que sí vienen de `errno`.
② El mensaje es la parte que describe de verdad el error, ya que el código es solo aproximado.

> **Nota**
>
> En este último caso, el texto devuelto por `what()` termina con la descripción que el sistema da a `EIO`:
>
> `El comando '/usr/bin/gzip' terminó con el código 2: Input/output error`
>
> Es la primera parte del mensaje la que informa realmente al usuario. C++ permite definir códigos y categorías de error propios para personalizar esa segunda parte del mensaje, pero eso se sale del alcance de esta práctica.

2.5. Manejo de errores

Se mantienen las pautas de manejo de errores de la Unidad 1: las funciones devuelven el error a quien las llamó, solo `main()` muestra mensajes, estos van a la salida de error y el programa termina con un código de salida distinto de `EXIT_SUCCESS`.

A las situaciones de error de la Unidad 1 se añaden ahora:

- **Al procesar los argumentos:**
  - Opciones de línea de comandos inválidas.
  - Múltiples opciones de compresión indicadas.
- **Antes de empezar la copia:**
  - El comando de compresión no está instalado.
- **Durante la compresión:**

- Error al crear la tubería (`pipe()` falla).
- Error al crear el proceso hijo (`fork()` falla).
- Error al ejecutar el comando de compresión (`exec()` falla en el hijo).
- El comando de compresión termina con un código de error.
- El comando de compresión es terminado por una señal.
- Error al escribir en la tubería, porque el hijo ha terminado antes de tiempo y está rota (`EPIPE`).

Recuerda que **antes de salir es necesario liberar todos los recursos**: cerrar los descriptores de archivo, incluidos los de la tubería, y esperar por el proceso hijo si ya se había creado.

2.6. Comprobación

Para comprobar el programa nos vienen bien un archivo de texto, que se comprime mucho, y uno de datos aleatorios, que no se comprime casi nada:

```
$ yes "Archivo de texto para comprimir" | head -c 1M > texto.txt
$ dd if=/dev/urandom of=testfile.dat bs=1M count=16 iflag=fullblock
```

Lo primero es comprobar que el programa sigue copiando como antes cuando no se indica ninguna opción, para asegurarnos de que los cambios no han roto lo que ya funcionaba en la Unidad 1:

```
$ copy testfile.dat /tmp/copia.dat
$ cmp testfile.dat /tmp/copia.dat
```

Y a continuación, que la compresión funciona:

```
$ copy -z texto.txt /tmp/copia.txt
$ ls -lh /tmp/copia.txt.gz
-rw-r--r-- 1 user user 5,1K sep 12 10:05 /tmp/copia.txt.gz

$ file /tmp/copia.txt.gz
/tmp/copia.txt.gz: gzip compressed data
```

La prueba definitiva es descomprimir la copia y comprobar que el contenido es idéntico al del archivo original:

```
$ gunzip -c /tmp/copia.txt.gz | cmp - texto.txt
```

El mismo procedimiento sirve para `bzip2` (`bunzip2 -c`) y para `xz` (`unxz -c`), y conviene repetirlo con `testfile.dat` para comprobar que los archivos grandes y los datos binarios también se copian correctamente.

2.6.1. Casos de prueba adicionales

Adicionalmente, es importante probar los siguientes escenarios:

- **Comprimir dentro de un directorio de destino**, comprobando que la extensión se añade al nombre del archivo copiado y no al del directorio:

```
$ copy -j texto.txt /tmp/
$ ls /tmp/texto.txt.bz2
/tmp/texto.txt.bz2
```

- **El comando de compresión no está instalado**, que se puede provocar ejecutando el programa con un `PATH` en el que no esté, ya que es donde lo busca (ver la Sección 2.4.3):

```
$ PATH=/nonexistent copy -j texto.txt /tmp/copia.txt
copy: error: bzip2 no está instalado
```

- **Indicar varias opciones de compresión**:

```
$ copy -z -j texto.txt /tmp/copia.txt
copy: error: solo se admite una opción de compresión
uso: copy [-z | -j | -x] ORIGEN DESTINO
```

- **No dejar procesos hijos sin recoger**, comprobando después de varias copias que no queda ningún proceso del compresor, ni en ejecución ni convertido en un *zombi*:

```
$ ps -e | grep -E 'gzip|bzip2|xz'
```

- **Comprimir un archivo vacío**, que debe producir un archivo comprimido válido, aunque ocupe más que el original:

```
$ touch vacio.dat
$ copy -z vacio.dat /tmp/vacio.dat
$ gunzip -c /tmp/vacio.dat.gz | cmp - vacio.dat
```

3. Concurrencia y manejo síncrono de señales

Vamos a extender el programa `copy` de la Unidad 2 para que copie varios archivos y atienda señales mientras lo hace, repartiendo el trabajo entre varios hilos de ejecución. Aprenderemos a crear hilos con `std::jthread`, a manejar las señales de forma síncrona en un hilo dedicado y a cancelar de forma cooperativa un trabajo en curso sin dejar archivos a medias.

3.1. Objetivo

Hasta ahora `copy` ha sido un programa de una sola secuencia de ejecución: mientras copia no puede hacer nada más, ni siquiera responder a quien lo está ejecutando. Vamos a cambiarlo para que copie varios archivos y atienda señales mientras trabaja, usando para ello hilos de ejecución.

El programa debe cumplir con los siguientes requisitos:

- El programa aceptará varios archivos de origen en la línea de comandos y los copiará uno tras otro.
- Las señales `SIGINT`, `SIGTERM` y `SIGUSR1` se atenderán de forma **síncrona**, en un hilo dedicado creado con `std::jthread` que las espera con `sigtimedwait()`.
- La copia se realizará en otro hilo, de manera que el hilo de las señales pueda atenderlas mientras tanto.
- Las señales `SIGINT` –que es la que envía la terminal al pulsar `Ctrl-C`– y `SIGTERM` cancelarán la copia en curso. El archivo que se estuviera escribiendo **no debe quedarse** incompleto en el sistema de archivos, pero los que ya estuvieran copiados por completo se conservan.
- La señal `SIGUSR1` mostrará por la salida estándar, para cada archivo, cuántos bytes se han copiado ya y cuál es su tamaño total, sin interrumpir la copia.
- Que falle la copia de uno de los archivos no impedirá copiar los demás, pero el programa terminará con un código de salida distinto de `EXIT_SUCCESS`.
- Las funciones `copy_file()` y `copy_file_compressed()` recibirán un `std::stop_token` y comprobarán si se ha solicitado la cancelación **entre** operaciones de entrada/salida, nunca en medio de una.

- Cuando el programa termine –tanto si la copia se completó como si fue cancelada– todos los hilos deben haber terminado, todos los descriptores de archivo deben estar cerrados y no debe quedar ningún proceso hijo sin terminar ni sin haber esperado por él.
- Para todo ello se usarán exclusivamente `std::jthread`, `std::stop_source`, `std::stop_token`, `std::atomic` y las llamadas al sistema `pthread_sigmask()` y `sigtimedwait()`, tal y como se explica en el Apéndice G y el Apéndice H.

> **!** Importante
>
> La cancelación de esta práctica es **cooperativa**: un hilo solo termina cuando él mismo decide hacerlo, porque ha comprobado que se lo han pedido. No se pueden usar `pthread_cancel()` ni ningún otro mecanismo para terminar un hilo desde fuera, por los motivos que se explican en la Sección G.4.
> Tampoco se puede usar `std::thread` en lugar de `std::jthread`, ni crear hilos directamente con `pthread_create()`, ni instalar manejadores de señales con `sigaction()`. Y no se pueden separar los hilos con `detach()` –la alternativa que se describe en la Sección G.7.2–, porque todos los que creemos deben haber terminado antes que el programa.

3.2. Línea de comandos y funcionamiento

La línea de comandos de la Unidad 2 admite ahora más de un archivo de origen:

```
copy [-z | -j | -x] ORIGEN... DESTINO
```

Cuando se indica más de un `ORIGEN`, `DESTINO` tiene que ser un directorio que ya exista, lo que se comprueba con `is_directory()` (Sección 1.4.2). Con un único `ORIGEN`, el programa acepta lo mismo que aceptaba hasta ahora, tanto una ruta de destino que sea un archivo como una que sea un directorio.

El otro aspecto que cambia es lo que el programa puede hacer **mientras** copia.

Al pulsar `Ctrl-C` durante una copia, el programa no muere a medias dejando un archivo incompleto, sino que la cancela ordenadamente, borra el archivo de destino que estaba escribiendo y termina con un código de salida de error:

```
$ copy bigfile1.dat bigfile2.dat /tmp/copias
^C
copy: copia cancelada, se ha eliminado /tmp/copias/bigfile2.dat
$ echo $?
1
$ ls /tmp/copias
bigfile1.dat
```

Las copias que ya hayan terminado se mantienen, porque son archivos completos y copiados correctamente. Sin embargo, la que estaba en curso desaparece y las que quedaban por empezar no se hacen.

Enviando la señal `SIGUSR1` se puede consultar cómo va la copia, tantas veces como se quiera, sin que esta se detenga:

```
$ copy -z bigfile1.dat bigfile2.dat /tmp/copias &
[1] 3721
$ kill -USR1 $!
copy: bigfile1.dat: 268435456 de 1073741824 bytes (25%)
copy: bigfile2.dat: 0 de 1073741824 bytes (0%)
$ kill -USR1 $!
copy: bigfile1.dat: 1073741824 de 1073741824 bytes (100%)
copy: bigfile2.dat: 713031680 de 1073741824 bytes (66%)
```

Y `SIGTERM` cancela la copia igual que `Ctrl-C`, que es lo que hace el comando `kill` cuando se le envía a un programa que se ejecuta en segundo plano:

```
$ kill $!
copy: copia cancelada, se ha eliminado /tmp/copias/bigfile2.dat.gz
```

Observa que el archivo eliminado es el de destino **con la extensión** que le corresponda a la compresión elegida, que es el que el programa estaba escribiendo realmente.

3.3. Arquitectura concurrente

El programa pasa a tener tres secuencias de ejecución avanzando a la vez, como muestra la Figura 3.1: la de `main()`, que solo crea los hilos y espera por ellos; la del hilo que atiende las señales; y la del hilo que copia los archivos, uno tras otro.

La pregunta que responde esta arquitectura es por qué un hilo para manejar las señales y no un manejador de señales. Un manejador solo puede llamar a funciones seguras en señales, así que no puede mostrar mensajes, ni reservar memoria, ni tomar un *mutex*, como se explica en la Sección H.3.2. Lo único que puede hacer con seguridad es anotar en una variable que la señal ha llegado y confiar en que el flujo principal se dé cuenta más tarde.

Un hilo que espera las señales, en cambio, **es** flujo normal del programa: cuando `sigtimedwait()` le devuelve una señal, ese hilo puede mostrar el progreso o pedir la cancelación de la copia como cualquier otra parte del código. El problema de la seguridad en señales simplemente desaparece. Por eso es la forma recomendada de atender señales en programas con varios hilos, como se explica en la Sección G.7.

Para que funcione, las señales tienen que estar bloqueadas en **todos** los hilos, y por eso se bloquean en `main()` antes de crear ninguno, como se explica en la Sección G.7. Recuerda

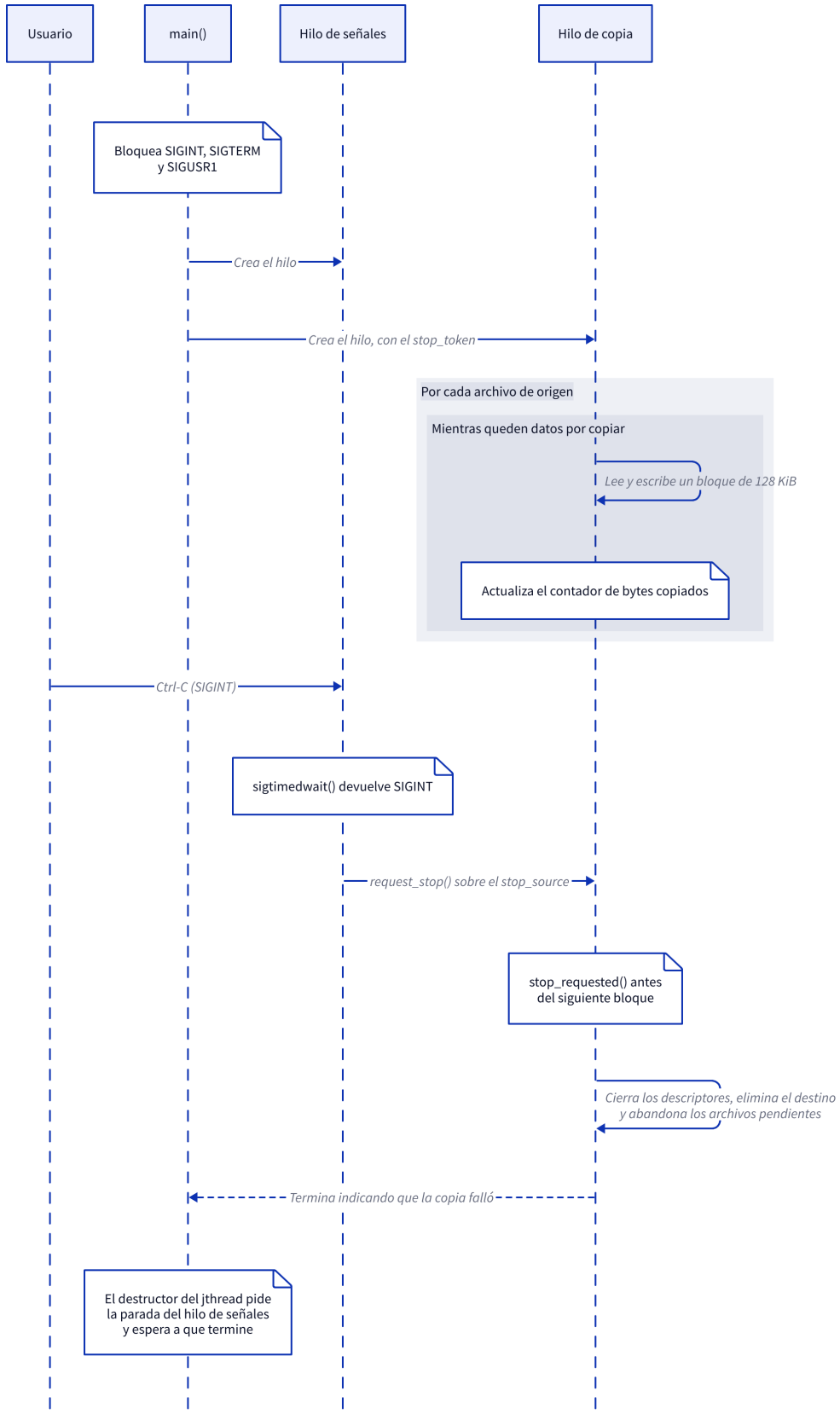


Figura 3.1.: Arquitectura del programa con la copia y el manejo de señales en hilos separados

además que `SIGPIPE` –que la Unidad 2 ya dejaba bloqueada durante toda la ejecución del programa– debe ser bloqueada pero **no** debe añadirse al conjunto que espera el hilo de señales, porque una tubería rota no es una solicitud del usuario que tengamos que atender, sino un error que `write()` ya informa devolviendo `EPIPE`. Ese bloqueo puede seguir haciéndose con `sigprocmask()`, como hasta ahora, porque ocurre cuando el programa todavía tiene un solo hilo. A partir del primer `std::jthread`, cualquier cambio de la máscara tiene que hacerse ya con `pthread_sigmask()`.

3.4. Cancelación cooperativa

Cancelar la copia consiste en que el hilo de señales pida la parada y el hilo de copia se dé cuenta y termine por su cuenta, dejándolo todo recogido. El mecanismo es el `std::stop_source` descrito en la Sección G.4, creado en `main()` antes que los hilos:

- El hilo de señales recibe una copia del `std::stop_source` y llama a `request_stop()` cuando llega `SIGINT` o `SIGTERM`.
- El hilo de copia recibe el `std::stop_token` correspondiente y lo comprueba con `stop_requested()` mientras copia.

> **!** Importante
>
> En este programa conviven **dos** mecanismos de parada independientes y confundirlos es el error más fácil de cometer:
>
> - El `std::stop_source` que creamos nosotros, que sirve para cancelar **la copia** y que activa el hilo de señales.
> - El que cada `std::jthread` lleva dentro, que sirve para pedirle **al propio hilo** que termine y que activa su destructor. Es el que usa el hilo de señales para salir de su bucle, porque su función principal recibe un `std::stop_token` como primer parámetro.

3.4.1. Dónde se comprueba la cancelación

La comprobación va al principio del bucle exterior de `copy_file()`, justo antes de leer el siguiente bloque que se va a copiar. Eso significa que entre que se pide la cancelación y el hilo de copia se entera puede pasar, como mucho, lo que tarde en leer y escribir un bloque de 128 KiB, que es un tiempo imperceptible. La cancelación no es inmediata, es **cooperativa**, y a cambio ocurre siempre en un punto en el que el programa sabe exactamente qué recursos tiene abiertos.

3.4.2. Qué hay que limpiar

Al detectar la cancelación, la función debe cerrar los dos descriptores de archivo, eliminar el archivo de destino que se esté escribiendo actualmente con `unlink()` y devolver el error correspondiente.

> **Nota**
>
> Si `DESTINO` ya existía antes de empezar, cancelar la copia lo destruye, porque se truncó en el momento de abrirlo para escritura. Es el comportamiento que se pide en esta actividad, aunque no sea el que tienen las herramientas reales, que en muchas implementaciones realmente escriben en un archivo temporal y lo renombran al terminar.

3.4.3. Cancelar una copia con compresión

Cancelar `copy_file_compressed()` no necesita ningún mecanismo nuevo, ni hay que enviarle ninguna señal al proceso del compresor. Basta con aprovechar el funcionamiento de la tubería que ya se implementó en la Unidad 2:

1. La función deja de escribir en la tubería y cierra su extremo de escritura.
2. El comando de compresión recibe un EOF al leer de su entrada estándar, así que termina normalmente por su cuenta.
3. La función lo recoge con `waitpid()`, exactamente igual que cuando la copia termina bien.
4. Y solo entonces elimina con `unlink()` el archivo de destino, que ya nadie está escribiendo.

Como la copia se ha cancelado, el resultado retornado por `copy_file_compressed()` es una cancelación, aunque `waitpid()` devuelva un código de salida `0` indicando que el compresor ha terminado con éxito.

3.5. Informe de progreso

Para poder informar del progreso, el hilo de copia tiene que contar lo que lleva copiado y el hilo de señales tiene que poder leer esa cuenta en cualquier momento. Es el único dato que ambos hilos comparten, y para ello se sugiere la siguiente estructura, de la que habrá una por cada archivo:

```
struct CopyProgress
{
    std::string src_path;                                    ①
    std::string dest_path;
```

```
    std::uintmax_t total_bytes;
    std::atomic<std::uintmax_t> copied_bytes = 0;                    ②
};
```

① Las rutas y el tamaño total no cambian durante la copia, así que no necesitan ninguna protección. Se rellenan antes de crear los hilos y después estos solo los leen.
② El contador, en cambio, lo escribe un hilo mientras otro lo lee, así que debe ser atómico para que el hilo de señales nunca observe un valor a medio escribir, como se explica en la Sección G.3.

El tamaño total se obtiene con `stat()`, como ya se hacía en la Unidad 1, y el hilo de copia suma al contador los bytes de cada bloque cuando ha terminado de escribirlo entero. Si `stat()` falla para alguno de los archivos –porque no existe, por ejemplo– basta con dejar su tamaño a cero, ya que del error informará `copy_file()` o `copy_file_compressed()` al intentar abrirlo.

Una estructura que contiene un `std::atomic` no se puede copiar ni mover, lo que tiene dos consecuencias. La primera es que solo se puede pasar a los hilos por referencia, con `std::ref()` o `std::cref()`. La segunda es que el vector que las contiene no puede crecer con `push_back()`, así que hay que crearlo desde el principio con el tamaño necesario –`std::vector<CopyProgress> progress(options.src_paths.size())`– y rellenar después los campos de cada elemento.

Ten en cuenta además que ese vector debe seguir existiendo mientras los hilos lo usen, lo que en este programa se cumple sin esfuerzo declarándolo en `main()`, porque los hilos se crean y se esperan dentro de esa misma función. Es decir, no hace falta que sea global.

3.6. Manejo de errores

Se mantienen las pautas de manejo de errores de la Unidad 1: las funciones devuelven el error a quien las llamó usando `std::expected` y los mensajes van a la salida de error. A ellas se añaden las situaciones propias de esta actividad:

- **La copia cancelada es un error más.** Se informa de ella con `std::unexpected(std::system_error(ECANCELED, ...))`, de forma que `copy_file()` y `copy_file_compressed()` conservan el tipo de retorno que ya tenían. Así quien las llama puede distinguirla del resto comparando el código con `std::errc::operation_canceled` y mostrar el mensaje que corresponde.
- **Los errores de la copia los muestra la función del hilo de copia `copy_worker()`.** La regla de la Unidad 1 dice que los mensajes se muestren en `main()` o lo más cerca posible de ella, y la función principal de un hilo es precisamente el `main()` de ese hilo. Eso permite que el error en la copia de un archivo se vea en cuanto ocurre.

  `copy_file()` y `copy_file_compressed()` siguen sin mostrar nada. Como todas las funciones de la práctica, le devuelven el error a `copy_worker()`, que es quien lo muestra

y quien decide seguir con el siguiente archivo, porque que un origen no exista o no se pueda leer no impide que se puedan copiar los demás.

- **`main()` solo necesita saber si alguna copia falló**, para terminar con un código de salida distinto de `EXIT_SUCCESS`. Por tanto, le basta con una variable `bool` que el hilo de copia ponga a `true`, sin necesidad de recoger un resultado por archivo copiado.
- **Ninguna excepción debe escapar del cuerpo de un hilo.** Si lo hace, el programa termina de inmediato con `std::terminate()` (Sección G.5), sin llegar a `main()`. El patrón `protected_main()` de la Unidad 1 **no** protege a los hilos que creamos, solo al hilo principal, así que la función principal de cada hilo necesita su propio `try`/`catch`.
- **Los errores de `pthread_sigmask()` no están en `errno`**, sino en el valor que devuelve, así que no se le puede aplicar el patrón habitual de la práctica (Sección G.7).
- **Que expire el plazo de espera de `sigtimedwait()` no es un error**, sino lo que ocurre la mayoría de las veces (Sección G.7.1). Cualquier valor de `errno` distinto de `EAGAIN` y `EINTR` sí lo es.

Los errores que impiden que el programa arranque –que no se puedan bloquear las señales, por ejemplo– deben terminar la ejecución antes de crear ningún hilo.

3.7. Implementación

El flujo del programa es el siguiente:

```
options = parse_arguments(argc, argv)                # Unidad 2
if has_error(options):
    error("argumentos inválidos")

if len(options.src_paths) > 1 and not is_directory(options.dest_path):
    error("DESTINO debe ser un directorio")          # Unidad 1

progress = []                                                          ①
for src_path in options.src_paths:
    dest_path = make_dest_path(src_path, options)
    if is_same_file(src_path, dest_path):            # Unidad 1
        error("{src_path} y {dest_path} son el mismo archivo")
    progress.append(CopyProgress(src_path, dest_path, size_of(src_path)))

signal_set = block_signals()                                           ②
if has_error(signal_set):
    error("no se pudieron bloquear las señales")

copy_stop = stop_source()

signal_thread = start_thread(wait_for_signals,                         ③
```

```
                              signal_set, copy_stop, progress)

copy_thread = start_thread(copy_worker, options,
                           progress, copy_stop.get_token())

copy_failed = copy_thread.wait()                                    ④

if copy_failed:                                                     ⑤
    return EXIT_FAILURE
```

① Las comprobaciones que la Unidad 2 hacía una sola vez se repiten ahora para cada archivo, y su resultado se guarda en el vector que los dos hilos van a compartir. En C++ ese vector no se va llenando, sino que se crea desde el principio con su tamaño definitivo, por el motivo que explica la Sección 3.5. Cualquier error aquí termina el programa antes de crear ninguno de ellos.
② Las señales se bloquean antes de crear ningún hilo, porque cada hilo nace con una copia de la máscara del que lo crea, tal y como se explica en la Sección G.7.
③ El hilo de señales se crea una sola vez y sigue vivo hasta el final del programa, atendiendo las señales mientras la copia avanza.
④ En C++ esta espera no se programa: ocurre al destruirse el `std::jthread` del hilo de copia, y el valor llega en la variable `bool` que se le pasó por referencia al crearlo. La Sección 3.7.7 muestra cómo queda.
⑤ `main()` no muestra aquí ningún mensaje. De los errores de cada archivo ya informó `copy_worker()` en cuanto ocurrieron, así que lo único que queda es traducir el fallo en el código de salida.

Como en las unidades anteriores, el pseudocódigo indica qué funciones hay que implementar y cuándo hay que llamarlas, y `error("...")` significa aplicar el manejo de errores descrito en la Sección 3.6.

Vamos a ver cada una de esas funciones con más detalle.

3.7.1. Construir la ruta de destino de cada archivo

Los pasos con los que la Unidad 2 calculaba la ruta del archivo de destino hay que darlos ahora una vez por cada origen, así que conviene reunirlos en una función:

```
std::string make_dest_path(const std::string& src_path,
    const Options& options);
```

Aquí también se puede declarar la ruta de origen como `std::string_view`:

```
std::string make_dest_path(std::string_view src_path,
    const Options& options);
```

Debe añadir a `DESTINO` el nombre del archivo de origen cuando aquel es un directorio, tal y como se explica en la Sección 1.4.3, y después la extensión que corresponda a la compresión elegida.

La comprobación de que origen y destino no son el mismo archivo se queda fuera, en `main()`, porque no forma parte de construir la ruta y porque es un error que debe terminar el programa antes de crear ningún hilo.

3.7.2. Bloquear las señales

Para preparar el conjunto de señales y bloquearlas, implementa la siguiente función:

```
std::expected<sigset_t, std::system_error> block_signals();
```

Debe construir un conjunto con `SIGINT`, `SIGTERM` y `SIGUSR1`, bloquearlo con `pthread_sigmask()` y devolver el conjunto, porque es el que necesitará después el hilo de señales para esperarlas.

3.7.3. Esperar y atender las señales

La función principal del hilo de señales es:

```
void wait_for_signals(std::stop_token stop_token, sigset_t signal_set,
    std::stop_source copy_stop,
    const std::vector<CopyProgress>& progress);
```

`stop_token` El *token* que `std::jthread` entrega a las funciones principales cuyo primer parámetro es un `std::stop_token`. Es el que permite salir del bucle cuando el destructor del hilo pide su parada.

`signal_set` El conjunto de señales que devolvió `block_signals()`, que es por el que hay que esperar.

`copy_stop` La fuente de parada con la que se cancela la copia. Se pasa por valor porque todas las copias de un `std::stop_source` comparten el mismo estado, así que pedir la parada en cualquiera de ellas la pide en todas.

`progress` El progreso de la copia de cada archivo, que este hilo solo lee, por lo que se le pasa como referencia constante con `std::cref()`.

La función debe repetir la espera con `sigtimedwait()` hasta que se solicite su parada, tal y como se describe en la Sección G.7.1, y actuar según la señal recibida:

- Con `SIGINT` o `SIGTERM`, llamar a `request_stop()` sobre `copy_stop` para cancelar la copia. El hilo puede seguir esperando señales después, porque terminará cuando se destruya su `std::jthread`.

- Con `SIGUSR1`, mostrar por la salida estándar una línea por cada archivo, con su ruta de origen, los bytes copiados, el tamaño total y el porcentaje que representan.

> **Nota**
>
> Cuidado al calcular el porcentaje. Si el archivo de origen está vacío, `total_bytes` es `0` y la división provoca un comportamiento indefinido.

3.7.4. Copiar el archivo

La función `copy_file()` de la Unidad 1 ahora necesita dos parámetros más:

```
std::expected<void, std::system_error> copy_file(
  const std::string& src_path, const std::string& dest_path,
  CopyProgress& progress, std::stop_token stop_token,
  mode_t dst_perms = 0644);
```

Donde `progress` es la estructura del archivo que se está copiando, que la función va actualizando con los bytes que lleva copiados, y `stop_token` es el *token* que debe comprobar para saber si se ha cancelado la copia.

El resto del funcionamiento es el de la Unidad 1, con la comprobación de la cancelación al principio de cada vuelta del bucle exterior, según se explicó en la Sección 3.4.1, y la limpieza descrita en la Sección 3.4.2 cuando se detecta la cancelación.

> **Importante**
>
> El *token* se pasa en último lugar –antes del parámetro con valor por defecto– para dejar claro que es un `std::stop_token` que la función recibe como cualquier otro argumento, y no el que `std::jthread` entrega a las funciones principales de los hilos.

3.7.5. Copiar el archivo con compresión

La función `copy_file_compressed()` de la Unidad 2 recibe los mismos dos parámetros nuevos:

```
std::expected<void, std::system_error> copy_file_compressed(
    const std::string& src_path,
    const std::string& dest_path,
    const std::string& compression_path,
    CopyProgress& progress, std::stop_token stop_token);
```

Y comprueba la cancelación en el mismo sitio: antes de leer cada bloque del archivo de origen, que aquí es lo mismo que decir antes de escribir cada bloque en la tubería. Al detectarla, debe terminar la copia en el orden descrito en la Sección 3.4.3.

> **!** Importante
>
> Ahora el `fork()` de esta función lo hace un programa con varios hilos, y eso convierte en obligatorias las precauciones de la Sección G.6: entre el `fork()` y el `exec()`, el proceso hijo solo debe preparar sus descriptores, restablecer su máscara de señales y ejecutar el comando.
> La máscara, además, ahora incluye más señales que en la Unidad 2, así que restablecerla es más importante todavía. Sin ello el compresor heredaría bloqueadas `SIGINT` y `SIGTERM`, y no respondería a un `Ctrl-C`.

3.7.6. El hilo de copia

El hilo que hace la copia ejecuta una función que recorre los archivos uno tras otro y decide para cada uno si hay que llamar a `copy_file()` o a `copy_file_compressed()`, en función de las opciones de la línea de comandos, tal y como se explicó en la Unidad 2:

```
void copy_worker(const Options& options,
    std::vector<CopyProgress>& progress, bool& failed,
    std::stop_token stop_token);
```

Las rutas de origen y destino de cada archivo las toma de su `CopyProgress`, que es donde `main()` las dejó preparadas.

Es esta función la que muestra los mensajes de la copia, tal y como se explicó en la Sección 3.6. Si un archivo falla, muestra su error, pone `failed` a `true` y sigue con el siguiente. Si detecta la cancelación, muestra el mensaje correspondiente, marca el fallo y termina sin llegar a copiar los que quedaban.

La función principal de un hilo no puede devolver un valor a quien lo creó, así que `failed` es una variable que `main()` declara y le pasa con `std::ref()`. No necesita ser atómica, a diferencia del contador del progreso, porque `main()` no la lee hasta que el hilo haya terminado y la espera del destructor del `std::jthread` garantiza que para entonces ve su valor definitivo, como se explica en la Sección G.1. Recuerda además que `copy_worker()` es el límite del que no puede escapar ninguna excepción, según se explicó también en la Sección 3.6, así que todo su cuerpo debe estar dentro de un `try/catch` que muestre el mensaje de cualquier excepción y marque el fallo.

> **i** Nota
>
> Con este cambio hay dos hilos escribiendo por su cuenta en la E/S estándar: el de las señales muestra el progreso por la salida estándar y el de la copia los errores por la

> salida de error. Escribir a la vez en los flujos estándar no provoca una condición de carrera, pero los caracteres de una línea pueden acabar entremezclados con los de la otra. Para evitarlo, compón cada mensaje entero con `std::format` y escríbelo en una sola operación.

3.7.7. El orden de los hilos en `main()`

En `main()`, el hilo de señales se declara **antes** del ámbito en el que se crea el hilo de copia:

```
bool copy_failed = false;

std::jthread signal_thread(wait_for_signals, signal_set,
    copy_stop, std::cref(progress));

{
    std::jthread copy_thread(copy_worker, std::cref(options),
        std::ref(progress), std::ref(copy_failed), copy_stop.get_token());
}                                                                          (1)

// Aquí ya se puede consultar copy_failed...
```

(1) Al cerrar el ámbito se espera a que la copia termine, pero el hilo de señales sigue vivo, atendiendo las señales que lleguen mientras tanto.

Así los hilos se destruyen en el orden correcto: primero el de copia, que es el que hace el trabajo, y solo al terminar `main()` el de señales, que ya no tiene nada a lo que atender. Recuerda que no hay que llamar a `join()` en ningún momento, porque el destructor de `std::jthread` ya pide la parada del hilo y lo espera, como se explica en la Sección G.1.

3.8. Comprobación

Para poder cancelar una copia hace falta que dure lo suficiente como para que dé tiempo a hacerlo, así que conviene crear archivos de origen grandes:

```
$ dd if=/dev/urandom of=bigfile1.dat bs=1M count=1024 iflag=fullblock
$ cp bigfile1.dat bigfile2.dat
$ mkdir /tmp/copias
```

Lo primero es comprobar que el programa sigue copiando correctamente cuando nadie lo interrumpe, para asegurarnos de que los cambios no han roto lo que ya funcionaba:

```
$ copy bigfile1.dat bigfile2.dat /tmp/copias
$ cmp bigfile1.dat /tmp/copias/bigfile1.dat
$ cmp bigfile2.dat /tmp/copias/bigfile2.dat
```

Y a continuación, que la copia se cancela sin dejar detrás el archivo que estaba escribiendo:

```
$ rm /tmp/copias/*
$ copy bigfile1.dat bigfile2.dat /tmp/copias
^C
copy: copia cancelada, se ha eliminado /tmp/copias/bigfile1.dat
$ ls /tmp/copias
```

3.8.1. Casos de prueba adicionales

Adicionalmente, es importante probar los siguientes escenarios:

- **Consultar el progreso varias veces durante la misma copia**, comprobando que el contador avanza, que aparece una línea por cada archivo y que la copia no se detiene mientras tanto:

  ```
  $ copy bigfile1.dat bigfile2.dat /tmp/copias &
  [1] 3721
  $ kill -USR1 %1
  copy: bigfile1.dat: 268435456 de 1073741824 bytes (25%)
  copy: bigfile2.dat: 0 de 1073741824 bytes (0%)
  ```

- **Cancelar con `SIGTERM` desde otra terminal**, que debe comportarse igual que usando `Ctrl-C`:

  ```
  $ kill $(pidof copy)
  ```

- **Cancelar una copia con compresión**, comprobando que se elimina el archivo con su extensión y que no queda ningún proceso del compresor, ni en ejecución ni convertido en un *zombi*:

  ```
  $ copy -z bigfile1.dat /tmp/copias
  ^C
  copy: copia cancelada, se ha eliminado /tmp/copias/bigfile1.dat.gz
  $ ps -e | grep gzip
  ```

- **Consultar el progreso de una copia con compresión**, teniendo en cuenta que el contador refleja los bytes leídos del origen, no los que el compresor escribe en el destino.

- **Copiar varios archivos cuando uno de ellos no existe**, comprobando que los demás sí se copian y que el programa informa del error y termina indicándolo:

```
$ copy no-existe.dat bigfile1.dat /tmp/copias
copy: error al copiar no-existe.dat: No such file or directory
$ echo $?
1
```

- **Indicar varios orígenes con un `DESTINO` que no es un directorio**, comprobando que el programa avisa y no copia nada.
- **Copiar un archivo vacío**, que debe funcionar igual que en la Unidad 1 y no provocar ningún error al mostrar el progreso:

```
$ touch vacio.dat
$ copy vacio.dat /tmp/vacio.dat
```

- **Cancelar la copia de un archivo de destino que ya existía**, comprobando que desaparece, tal y como se advertía en la Sección 3.4.2.

4. Comunicación entre procesos con sockets

Vamos a separar el programa `copy` de la Unidad 3 en dos programas independientes: uno que pide copias de seguridad y otro que las hace. Aprenderemos a comunicar procesos que no tienen ningún parentesco entre sí mediante *sockets* de dominio UNIX orientados a conexión, y a esperar por ellos sin renunciar a poder terminar de forma ordenada.

4.1. Objetivo

Hasta ahora `copy` ha hecho la copia en el mismo proceso que la pedía. Vamos a repartir ese trabajo entre dos programas que se ejecutan por separado, iniciados en momentos distintos y sin ninguna relación de parentesco entre ellos:

- **`backup`**: el cliente, que pide que se haga una copia de seguridad de los archivos indicados en su línea de comandos.
- **`backup-server`**: el servidor, que espera conexiones y copia al directorio de copias cada archivo que le piden.

Los dos salen del mismo sitio: son el programa `copy` de la Unidad 3 duplicado, con su misma estructura de hilos, su mismo manejo síncrono de señales y su misma cancelación cooperativa. Lo único que cambia es por dónde entra y por dónde sale el trabajo.

Ambos programas deben cumplir con los siguientes requisitos:

- Se comunicarán mediante *sockets* del dominio `AF_UNIX` y del tipo `SOCK_STREAM`, es decir, orientados a conexión, tal y como se explica en el Apéndice I. El servidor crea el *socket* que escucha y el cliente se conecta a él.
- El cliente no copia nada. Por cada archivo de su línea de comandos envía por la conexión su ruta absoluta con una función nueva, `send_line()`, y no usa `copy_file()` ni `copy_file_compressed()`.
- El servidor recibe cada ruta con otra función nueva, `recv_line()`, y copia el archivo al directorio de copias llamando a `copy_file()` o a `copy_file_compressed()`, que no cambian respecto a la Unidad 3.
- Las opciones de compresión de la Unidad 2 las tiene ahora el servidor, que es quien copia. El cliente no las necesita.

- El servidor no puede quedarse bloqueado en `accept()` ni en `recv()` sin poder atender una señal, así que esas esperas se harán con `poll()` y un plazo, como se explica en la Sección I.6.
- El servidor atenderá conexiones indefinidamente, hasta que reciba `SIGINT` o `SIGTERM`. Entonces cancelará la copia en curso, igual que hacía `copy`, y terminará.
- Solo puede haber un servidor en marcha sobre el mismo directorio de ejecución, lo que se comprueba con el archivo en el que el servidor deja su PID.
- Los dos programas conservan la estructura de la Unidad 3: un hilo que espera las señales, otro que hace el trabajo y la cancelación cooperativa con `std::stop_source`.
- Cuando terminen, todos los hilos deben haber terminado y todos los descriptores de archivo deben estar cerrados, incluidos los de los *sockets*. El servidor, además, debe dejar el directorio de ejecución como estaba, sin el archivo del *socket* ni el del PID.
- Para todo ello se usarán exclusivamente las llamadas al sistema `socket()`, `bind()`, `listen()`, `accept()`, `connect()`, `send()`, `recv()`, `poll()`, `unlink()` y las que ya se venían usando, tal y como se explica en el Apéndice I.

> **!** Importante
>
> Siguen vigentes todas las restricciones de las unidades anteriores: los archivos se manipulan con las llamadas al sistema, nunca con `std::ifstream` ni `std::FILE`; la cancelación es **cooperativa**; los hilos se crean con `std::jthread` y no se separan con `detach()`; y las señales no se manejan con `sigaction()`.

4.2. Línea de comandos y funcionamiento

El servidor acepta las mismas opciones de compresión que `copy` y un único argumento:

```
backup-server [-z | -j | -x] [DIRECTORIO]
```

`-z`, `-j` y `-x` (opcionales) Guardan las copias comprimidas, con el mismo significado y las mismas extensiones que en la Unidad 2. Siguen siendo mutuamente excluyentes.

`DIRECTORIO` (opcional) El directorio en el que se guardan las copias, que debe existir de antemano. Si no se indica, se usa el directorio de trabajo actual del proceso.

El cliente no tiene opciones y acepta tantos archivos como se le quieran indicar:

```
backup ARCHIVO...
```

Los dos leen además del entorno la variable `BACKUP_RUN_DIR`, que completa la línea de comandos:

`BACKUP_RUN_DIR` (obligatoria) La ruta de un directorio que ya exista, en el que el servidor crea el *socket* por el que atiende y el cliente lo busca para conectarse. Es la forma que tienen los dos programas de ponerse de acuerdo en el directorio de ejecución, como se detalla en la Sección 4.3.1.

Una sesión completa empieza, por tanto, definiendo la variable y arrancando el servidor:

```
$ export BACKUP_RUN_DIR=/tmp/backup-run
$ backup-server /tmp/copias
backup-server: esperando conexiones en /tmp/backup-run/backup.sock
```

Y continúa pidiendo copias desde otra terminal, tantas veces como se quiera:

```
$ export BACKUP_RUN_DIR=/tmp/backup-run
$ backup documento.pdf informe.odt
backup: solicitud enviada para /home/user/documento.pdf
backup: solicitud enviada para /home/user/informe.odt
```

Mientras, en la terminal del servidor van apareciendo los resultados:

```
backup-server: copia completada: /home/user/documento.pdf ->
↪ /tmp/copias/documento.pdf
backup-server: copia completada: /home/user/informe.odt ->
↪ /tmp/copias/informe.odt
```

Que un archivo no se pueda copiar no interrumpe a los demás, ni termina el servidor. Simplemente se informa del error donde se detecta: en el cliente si la ruta ni siquiera se puede enviar, y en el servidor si falla la copia:

```
$ backup /tmp/no-existe.txt informe.odt
backup: error: /tmp/no-existe.txt no es un archivo regular
backup: solicitud enviada para /home/user/informe.odt
$ echo $?
1
```

`SIGUSR1` sigue sirviendo para preguntarle al servidor cómo va la copia que esté haciendo, igual que en la Unidad 3:

```
$ kill -USR1 $(cat /tmp/backup-run/backup-server.pid)
backup-server: copiando 268435456 de 1073741824 bytes (25%)
```

Y `SIGINT` –`Ctrl-C`– o `SIGTERM` terminan el servidor cancelando la copia en curso, que desaparece sin dejar un archivo a medias, exactamente igual que en la Unidad 3:

```
^C
backup-server: copia cancelada, se ha eliminado /tmp/copias/bigfile.dat
backup-server: terminando
```

4.3. Arquitectura del sistema

El servidor pone a escuchar un *socket* con una dirección conocida y se queda esperando. Cada ejecución del cliente abre una conexión con él, envía por ella las rutas de sus archivos y la cierra, como muestra la Figura 4.1.

Dentro de cada uno de los dos procesos no cambia nada. Son los tres flujos de ejecución de la Figura 3.1 –`main()`, el hilo de señales y el hilo que hace el trabajo– con el mismo reparto de responsabilidades. El hilo de trabajo del cliente, en lugar de copiar archivos, envía rutas; el del servidor, en lugar de tomar las rutas de `argv`, las recibe por la conexión.

> **Nota**
>
> El servidor atiende **una conexión cada vez**: hasta que no termina con un cliente no acepta al siguiente. Eso no deja fuera a nadie, porque las conexiones que llegan mientras tanto esperan en la cola de `listen()` (Sección I.5.1) y los clientes, que solo envían rutas, terminan sin tener que esperar a que sus copias se hagan. Atender a varios clientes a la vez, cada uno en su hilo, es el paso natural siguiente, pero no forma parte de esta actividad.

4.3.1. El directorio de ejecución

Cliente y servidor son programas distintos que no tienen relación padre-hijo, así que necesitan ponerse de acuerdo en la dirección del *socket*, que en el dominio UNIX es la ruta de un archivo (Sección I.2.2). Ese acuerdo se establece con la variable de entorno `BACKUP_RUN_DIR`, que debe estar definida y apuntar a un directorio existente en los dos. En él viven los dos archivos que el servidor crea al arrancar y borra al terminar:

`backup.sock` El *socket* por el que el cliente se conecta y envía las rutas.

`backup-server.pid` El archivo en el que el servidor deja su PID, que es lo que evita que se arranquen dos servidores sobre el mismo directorio y lo que permite enviarle señales desde la *shell*. El cliente no necesita acceder a este archivo. Que el servidor esté o no en marcha lo averigua al conectarse.

Los dos son **datos de ejecución**: solo tienen sentido mientras el servidor está en marcha y no deben sobrevivir a su parada. Por eso van juntos y por eso la variable se llama `BACKUP_RUN_DIR`. Es la misma idea con la que los servicios del sistema dejan sus *sockets* y sus archivos de PID en `/run`.

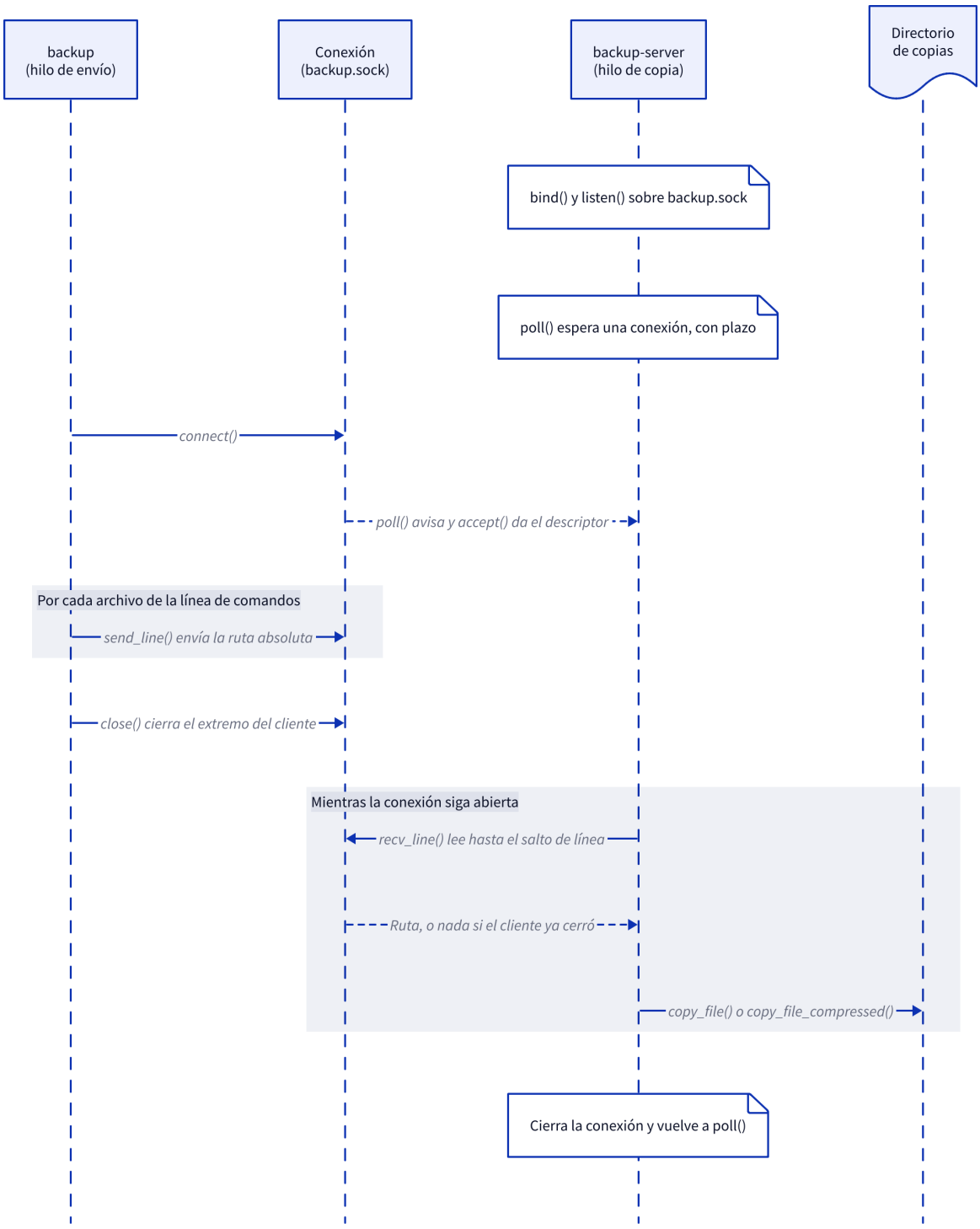


Figura 4.1.: Flujo de una solicitud de copia entre el cliente y el servidor

Para leer la variable se usa `get_environment_variable()`, el envoltorio de `getenv()` de la Sección B.1 que ya se utilizó en la Unidad 2, y cuyo funcionamiento se describe en el Apéndice F.

4.3.2. El protocolo

Por la conexión viajan **mensajes de texto terminados en un salto de línea** (`\n`). Por ahora el único mensaje que existe es la ruta absoluta de un archivo, pero `send_line()` y `recv_line()` no tienen por qué saber qué dice el mensaje que llevan, así que sirven para cualquier otro mensaje que quepa en una línea.

Ese salto de línea no es un adorno. Un *socket* `SOCK_STREAM` entrega un flujo continuo de bytes sin fronteras entre mensajes: lo que se envió en una llamada puede llegar partido en varias entregas o pegado a la siguiente (Sección I.1). El delimitador es lo único que le permite al receptor saber dónde acaba una ruta y empieza la siguiente.

Las rutas enviadas son absolutas porque el cliente y el servidor pueden tener directorios de trabajo distintos. Una ruta relativa que el cliente resuelve en su directorio de trabajo significaría otro archivo –o ninguno– en el del servidor, así que es el cliente quien la convierte antes de enviarla.

El final de la lista de rutas no lo marca ningún mensaje especial. El cliente cierra su extremo de la conexión y el servidor lo ve como un `recv()` que retorna `0` (Sección I.5.4).

4.3.3. Esperar sin bloquearse

`accept()` y `recv()` se bloquean indefinidamente mientras no llega nada, y eso es justo lo que un programa cancelable no se puede permitir: un hilo bloqueado en esas operaciones no comprueba su *token* de parada, así que el servidor no se enteraría de que le han pedido terminar hasta que apareciese un cliente.

La solución es la de la Sección I.6: esperar con `poll()` y un plazo, comprobar el *token* cada vez que el plazo expira y llamar a `accept()` o a `recv()` solo cuando ya se sabe que no se van a bloquear. Es el mismo patrón que el hilo de señales lleva usando desde la Unidad 3 con `sigtimedwait()`, aplicado ahora a un descriptor en lugar de a un conjunto de señales.

Como los dos sitios en los que el servidor espera –la llegada de una conexión y la de un mensaje– necesitan exactamente lo mismo, conviene reunirlo en una única función, `wait_for_input()`, que se describe en la Sección 4.5.6.

4.4. Manejo de errores

Se mantienen las pautas de las unidades anteriores: las funciones devuelven el error a quien las llamó con `std::expected`, los mensajes van a la salida de error y los muestra la función

principal del hilo que detecta el problema, tal y como se explicó en la Sección 3.6. A ellas se añaden las situaciones propias de esta actividad.

Impiden arrancar y terminan el programa antes de crear ningún hilo:

- Que `BACKUP_RUN_DIR` no esté definida, no apunte a un directorio existente o dé una ruta de *socket* que no quepa en `sun_path` (Sección I.2.2).
- En el servidor, que ya haya otro servidor en marcha, que fallen `socket()`, `bind()` o `listen()`, o que no se pueda escribir el archivo del PID.
- En el cliente, que falle `connect()`. `ECONNREFUSED` y `ENOENT` significan que el servidor no está en marcha, que es el caso más frecuente y merece un mensaje propio (Sección I.5.3).

Y no terminan el programa, sino que se informa de ellos y se sigue trabajando:

- En el cliente, que un archivo no exista o no sea un archivo regular, y que su ruta no se pueda convertir en absoluta. El cliente pasa al siguiente archivo y termina con un código de salida distinto de `EXIT_SUCCESS`, como hacía `copy` en la Unidad 3.
- En el servidor, cualquier error al aceptar una conexión, al recibir una ruta o al copiar un archivo. Un servidor no se cae porque una solicitud venga mal: cierra esa conexión, informa y sigue esperando la siguiente.

> **!** Importante
>
> El servidor debe borrar el archivo del *socket* y el del PID **también cuando termina por un error**. Cerrar el *socket* no borra su archivo (Sección I.7), y si lo deja atrás, el siguiente servidor encontrará ocupada su dirección y un archivo de PID de un proceso que ya no existe.
> Por eso la limpieza se hace en `main()`, que es la única función por la que el programa pasa siempre, y no en el hilo de copia.

Que expire el plazo de `poll()` no es un error, sino lo que ocurre la mayoría de las veces, igual que ocurría con `sigtimedwait()` en la Sección 3.6. Tampoco lo es que `recv()` retorne `0`: significa que el cliente ha cerrado la conexión.

4.5. Implementación

4.5.1. El servidor

El flujo de `main()` en `backup-server` es el de la Unidad 3 con la preparación del *socket* por delante:

```
options = parse_arguments(argc, argv)                    # Unidad 2
if has_error(options):
    error("argumentos inválidos")

if not is_directory(options.dest_path):                  # Unidad 1
    error("{options.dest_path} no es un directorio")

if compression_requested(options):                       # Unidad 2
    comando = get_compression_command(options)
    compression_path = find_command(comando)
    if not has_value(compression_path):
        error("{comando} no está instalado")

run_dir = get_environment_variable("BACKUP_RUN_DIR")
if not has_value(run_dir):
    error("BACKUP_RUN_DIR no está definida")

if not is_directory(run_dir):                            # Unidad 1
    error("{run_dir} no es un directorio")

server_pid = read_pid_file(run_dir)                                    ①
if has_value(server_pid) and is_process_alive(server_pid):
    error("ya hay un servidor en marcha con el PID {server_pid}")

listen_fd = create_server_socket(run_dir)                              ②
if has_error(listen_fd):
    error("no se pudo crear el socket")

if has_error(write_pid_file(run_dir)):
    error("no se pudo escribir el archivo del PID")

signal_set = block_signals()                             # Unidad 3
if has_error(signal_set):
    error("no se pudieron bloquear las señales")

backup_stop = stop_source()
progress = CopyProgress()

signal_thread = start_thread(wait_for_signals,                         ③
                             signal_set, backup_stop, progress)

backup_thread = start_thread(backup_worker, options,
                             compression_path, listen_fd, progress,
                             backup_stop.get_token())

backup_failed = backup_thread.wait()
```

```
close(listen_fd)                                                    ④
remove_socket_file(run_dir)
remove_pid_file(run_dir)

if backup_failed:
    return EXIT_FAILURE
```

① Que el archivo exista no basta para decidir que hay otro servidor: puede ser el que dejó atrás uno que terminó de forma anormal. Lo que decide es si el proceso con ese PID sigue vivo.
② A partir de aquí el programa deja cosas en el sistema de archivos, así que cualquier error posterior tiene que pasar por la limpieza del final antes de retornar.
③ Igual que en la Unidad 3, el hilo de señales se declara antes del ámbito del hilo de trabajo, para que siga atendiendo señales mientras el otro copia y sea el último en terminar.
④ La limpieza va aquí, y no en el hilo de copia, por el motivo que explica la Sección 4.4.

4.5.2. El cliente

Y el de `main()` en `backup` es el mismo sin nada que crear, porque del *socket* y del archivo del PID se ocupa el servidor:

```
options = parse_arguments(argc, argv)
if has_error(options):
    error("argumentos inválidos")

run_dir = get_environment_variable("BACKUP_RUN_DIR")
if not has_value(run_dir):
    error("BACKUP_RUN_DIR no está definida")

conn_fd = connect_to_server(run_dir)                                ①
if has_error(conn_fd):
    error("no se pudo conectar con el servidor")

signal_set = block_signals()
if has_error(signal_set):
    error("no se pudieron bloquear las señales")

send_stop = stop_source()

signal_thread = start_thread(wait_for_signals,                      ②
                             signal_set, send_stop)

send_thread = start_thread(send_worker, options, conn_fd,
```

```
                                send_stop.get_token())

send_failed = send_thread.wait()                                          ③

close(conn_fd)                                                            ④

if send_failed:
    return EXIT_FAILURE
```

① Aquí se descubre si el servidor está en marcha, sin necesidad de consultar su archivo de PID: si no lo está, no hay nadie escuchando y `connect()` falla.
② El hilo de señales del cliente no recibe ninguna estructura de progreso, porque no tiene nada que informar, por el motivo que explica la Sección 4.5.11.
③ Como en la Sección 3.7.7, esta espera no se programa: ocurre al destruirse el `std::jthread` del hilo de envío, y el valor llega en la variable `bool` que se le pasó por referencia.
④ Cerrar la conexión es lo que le dice al servidor que ya no van a llegar más rutas, tal y como se explicó en la Sección 4.3.2.

Como en las unidades anteriores, el pseudocódigo indica qué funciones hay que implementar y cuándo llamarlas, y `error("...")` significa aplicar el manejo de errores de la Sección 4.4.

La `parse_arguments()` de la Sección 2.4.1 se retoca en cada programa: en el servidor `ORIGEN` desaparece y `DESTINO` pasa a ser un `DIRECTORIO` opcional. En el cliente desaparecen también las opciones de compresión y lo que queda es la lista de archivos.

Vamos a ver con más detalle lo que es nuevo.

4.5.3. El socket del servidor

```
std::expected<int, std::system_error> create_server_socket(
    const std::string& run_dir);
```

Deja listo el *socket* que escucha en `backup.sock`, dentro del directorio de ejecución, y retorna su descriptor. Para ello debe, en este orden:

1. Comprobar que la ruta del *socket* cabe en `sun_path` antes de copiarla, tal y como advierte la Sección I.2.2. Si no cabe, retorna un error con el código `ENAMETOOLONG`.
2. Crear el *socket* con `socket()`, en el dominio `AF_UNIX` y del tipo `SOCK_STREAM` (Sección I.1).
3. Eliminar con `unlink()` el archivo que pueda haber dejado un servidor anterior. Sin esto `bind()` fallaría con `EADDRINUSE`, porque en el dominio UNIX la dirección es un archivo que nadie borra al terminar el proceso (Sección I.2.2).

4. Asignarle la dirección con `bind()` (Sección I.3) y ponerlo a escuchar con `listen()` (Sección I.5.1).

Los permisos de ese archivo deciden quién puede conectarse, y la `umask` del proceso los recorta como a los de cualquier otro archivo (Sección I.2.2).

Al terminar hay que cerrar el descriptor con `close()` y borrar el archivo con `unlink()`, porque cerrar el *socket* no lo elimina (Sección I.7).

4.5.4. La conexión del cliente

```
std::expected<int, std::system_error> connect_to_server(
    const std::string& run_dir);
```

Crea un *socket* igual que el del servidor –mismo dominio y mismo tipo– y lo conecta a `backup.sock` con `connect()`, retornando el descriptor de la conexión (Sección I.5.3). El cliente no necesita `bind()`. Solo quien espera conexiones necesita una dirección conocida.

4.5.5. Enviar y recibir mensajes

Son las dos funciones que implementan el protocolo de la Sección 4.3.2, y las únicas que saben que por debajo hay un *socket*. Se llaman `send_line()` y `recv_line()` porque es exactamente lo que mueven –una línea, delimitada por el salto de línea– y no cualquier otro tipo de lista de bytes:

```
std::expected<void, std::system_error> send_line(int conn_fd,
    const std::string& message);
```

Envía `message` seguido de un salto de línea. Como `send()` puede colocar menos bytes de los que se le piden, hay que llamarla en un bucle hasta haber enviado el mensaje entero, con la forma que describe la Sección I.5.4. Y recuerda que `SIGPIPE` sigue bloqueada desde la Unidad 2, así que un extremo cerrado se manifiesta como un `EPIPE` que `send()` retorna, y no como la muerte del proceso.

```
std::expected<std::optional<std::string>, std::system_error> recv_line(
    int conn_fd, std::stop_token stop_token);
```

Recibe un mensaje y lo retorna sin el salto de línea final, o `std::nullopt` cuando ya no va a llegar ninguno más, que ocurre en dos casos: porque el otro extremo cerró la conexión –`recv()` retorna `0`– o porque se ha solicitado la cancelación mientras esperaba. Los dos significan lo mismo para quien la llama: dejar de pedir mensajes por esta conexión.

Antes de cada `recv()` espera con `wait_for_input()` (Sección 4.5.6), para no quedarse bloqueada. Y como el mensaje puede llegar partido, hay que leer hasta encontrar el `\n`, o hasta acumular `max_line_size` caracteres sin haberlo encontrado, en cuyo caso retorna un error con el código `EMSGSIZE`:

```
constexpr std::size_t max_line_size = 4096;
```

> **Nota**
>
> La forma más sencilla de encontrar el delimitador es recibir de uno en uno los caracteres. Es ineficiente, pero recibirlos a bloques obliga a guardar entre llamada y llamada los bytes que ya pertenecen al mensaje siguiente, porque el flujo no respeta las fronteras entre mensajes.

4.5.6. Esperar con un plazo

```
std::expected<bool, std::system_error> wait_for_input(int fd,
    std::stop_token stop_token);
```

Espera hasta que haya algo que recoger en `fd`, retornando `true` cuando lo hay y `false` si se solicitó la cancelación mientras esperaba.

Implementa la espera descrita en la Sección 4.3.3. El servidor la usa en sus dos esperas –antes de `accept()` y dentro de `recv_line()`– y por eso recibe el descriptor como argumento en lugar de quedarse con uno concreto.

4.5.7. El archivo del PID

```
std::expected<void, std::system_error> write_pid_file(
    const std::string& run_dir);
```

Escribe el PID que devuelve `getpid()` en `backup-server.pid`, como texto y seguido de un salto de línea, abriendo el archivo con `O_WRONLY | O_CREAT | O_TRUNC` y permisos `0644`. Escribirlo como texto es lo que permite leerlo desde la *shell* con `cat` para enviarle señales al servidor.

```
std::expected<std::optional<pid_t>, std::system_error> read_pid_file(
    const std::string& run_dir);
```

Lee el archivo y convierte su contenido en un número, retornando `std::nullopt` si el archivo no existe –que no es un error, sino la señal de que no hay ningún servidor– y un error si existe pero no se puede leer o no contiene un número. Para borrarlo al terminar, como con el archivo del *socket*, basta usar `unlink()`.

Para la conversión se recomienda `std::from_chars()`, de la cabecera `<charconv>`, que informa de los errores en el miembro `ec` del resultado:

```
pid_t pid = 0;
auto [ptr, ec] = std::from_chars(
    contenido.data(), contenido.data() + contenido.size(), pid);
if (ec != std::errc{})
{
    // El contenido del archivo no es un número válido...
}
```

```
bool is_process_alive(pid_t pid);
```

Comprueba si el proceso sigue existiendo con `kill(pid, 0)`, tal y como se explica en la Sección H.7.

4.5.8. El hilo de copia del servidor

Es el `copy_worker()` de la Sección 3.7.6, con la lista de archivos sustituida por una espera indefinida de conexiones:

```
void backup_worker(const Options& options,
    const std::string& compression_path, int listen_fd,
    CopyProgress& progress, bool& failed, std::stop_token stop_token);
```

Las conexiones se recogen con `accept()`, que retorna un descriptor nuevo para cada una, tal y como se explica en la Sección I.5.2. Su flujo son dos bucles, uno dentro del otro:

```
while True:
    ready = wait_for_input(listen_fd, stop_token)                    ①
    if has_error(ready):
        print(stderr, "error al esperar conexiones: {error}")
        failed = True
        break

    if not ready:                                                    ②
        break
```

```
    conn_fd = accept(listen_fd)
    if has_error(conn_fd):
        print(stderr, "error al aceptar la conexión: {error}")
        continue

    while True:
        src_path = recv_line(conn_fd, stop_token)
        if has_error(src_path):
            print(stderr, "error al recibir una solicitud: {error}")
            break

        if not has_value(src_path):                                    ③
            break

        dest_path = make_dest_path(src_path, options)   # Unidad 3
        if is_same_file(src_path, dest_path):           # Unidad 1
            print(stderr, "{src_path} ya es su propia copia")
            continue

        result = copy_request(src_path, dest_path,                     ④
                              options, compression_path,
                              progress, stop_token)

        if has_error(result):
            print(stderr, "error al copiar {src_path}: {error}")
        else:
            print("copia completada: {src_path} -> {dest_path}")

    close(conn_fd)
```

① Un fallo de `poll()` sí es un error fatal del servidor, porque sin él no hay manera de esperar conexiones. Como el hilo no puede retornar un valor, lo anota en `failed` para que `main()` termine con un código de salida de error, igual que hacía `copy_worker()` en la Sección 3.7.6.

② `wait_for_input()` retorna `false` cuando se ha pedido la parada, así que esta es la salida normal del bucle.

③ Se acabaron las rutas de este cliente, porque cerró la conexión o porque se canceló el servidor. En ambos casos toca cerrar el descriptor y volver al bucle exterior, que decidirá si sigue esperando.

④ `copy_request()` es simplemente el cuerpo del bucle de `copy_worker()`: prepara el `progress` del archivo y llama a `copy_file()` o a `copy_file_compressed()` según las opciones, sin cambiar nada de lo que se hacía en la Unidad 3.

Esa función, la única que hay que separar del `copy_worker()` original, tiene esta forma:

```
std::expected<void, std::system_error> copy_request(
    const std::string& src_path, const std::string& dest_path,
    const Options& options, const std::string& compression_path,
    CopyProgress& progress, std::stop_token stop_token);
```

El *token* de parada llega hasta los dos puntos en los que el hilo podría quedarse esperando, así que pedirle al servidor que termine no le obliga a acabar antes con el cliente que tenga delante. Y como `copy_file()` y `copy_file_compressed()` reciben el mismo *token*, la copia que estuviera en curso se cancela y su archivo de destino desaparece, igual que en la Sección 3.4.2.

Recuerda que este hilo es el límite del que no puede escapar ninguna excepción, como se explicó en la Sección 3.6. Aquí importa más que nunca. Un servidor que muere por una excepción en mitad de una copia deja atrás el archivo del *socket* y el del PID.

4.5.9. El hilo de envío del cliente

Es el mismo hilo, con `send_line()` donde `copy` llamaba a `copy_file()`:

```
void send_worker(const Options& options, int conn_fd, bool& failed,
    std::stop_token stop_token);
```

Recorre los archivos de la línea de comandos y, por cada uno:

1. Comprueba con `stat()` y la macro `S_ISREG()` que existe y es un archivo regular, tal y como se explica en la Sección A.7. Comprobarlo en el cliente permite dar el error donde el usuario escribió la ruta, en lugar de en la terminal del servidor.
2. Convierte la ruta en absoluta con `get_absolute_path()`, que envuelve a `std::filesystem::canonical()` como se explica en la Sección 4.5.10.
3. La envía con `send_line()` y muestra el mensaje de confirmación.

Los dos primeros pasos afectan solo a ese archivo. Si fallan, el hilo muestra el error, pone `failed` a `true` y pasa al siguiente. Un fallo de `send_line()`, en cambio, significa que la conexión ya no sirve –por ejemplo porque el servidor terminó y `send()` falló con `EPIPE` (Sección I.5.4)– así que no tiene sentido seguir intentándolo con los demás. Y como `copy_worker()`, comprueba el *token* de parada entre un archivo y el siguiente, para dejar de enviar en cuanto se cancele.

4.5.10. Convertir la ruta en absoluta

```
std::expected<std::string, std::system_error> get_absolute_path(
    const std::string& path);
```

`std::filesystem::canonical()` tiene dos versiones: una que informa de los errores lanzando una excepción y otra que deja el error en un `std::error_code` que recibe como último argumento. Usamos la segunda, porque en esta práctica los errores se retornan:

```
std::error_code ec;
auto absolute_path = std::filesystem::canonical(path, ec);
if (ec)
{
    return std::unexpected(std::system_error(ec,                          (1)
        std::format("Error al obtener la ruta absoluta de '{}'", path)));
}

return absolute_path.string();
```

(1) `std::system_error` tiene un constructor que acepta el `std::error_code` directamente, así que no hace falta extraer el número del error, que es además el mismo que la operación que falló habría dejado en `errno`.

Es la otra de las dos únicas cosas que esta práctica permite usar de `<filesystem>`, junto a `std::filesystem::path`, tal y como se advirtió en la Sección 1.4.3.

4.5.11. Informe de progreso

El `CopyProgress` de la Sección 3.5 se queda en el servidor, pero con una forma más sencilla, porque ahora solo hay una copia en curso cada vez:

```
struct CopyProgress
{
    std::atomic<bool> in_progress = false;                                (1)
    std::atomic<std::uintmax_t> total_bytes = 0;                          (2)
    std::atomic<std::uintmax_t> copied_bytes = 0;
};
```

(1) El servidor pasa la mayor parte del tiempo sin copiar nada, así que el hilo de señales necesita saber si hay algo de lo que informar.

(2) El tamaño total ya no se fija antes de crear los hilos, sino que cambia con cada archivo que llega, así que también tiene que ser atómico.

`copy_file()` y `copy_file_compressed()` no se enteran del cambio, porque lo único que hacían con la estructura era sumar al contador `copied_bytes`. Quien pone `in_progress` y `total_bytes` es `copy_request()`, antes y después de cada copia.

> **Nota**
>
> La línea de progreso del servidor no incluye el nombre del archivo. Las rutas son `std::string` que el hilo de copia sustituye en cada archivo, y compartirlas con el hilo de señales exigiría un mecanismo de sincronización de los que menciona la Sección G.3, en lugar de bastar con variables atómicas. De todas formas el nombre aparece en el mensaje que el servidor muestra al terminar cada copia.

El cliente, en cambio, no informa de ningún progreso, así que no tiene ninguna estructura como esta. Quien tarda en hacer su trabajo es el servidor, porque es quien copia: enviar una ruta por la conexión es instantáneo y el cliente ya muestra una línea por cada archivo conforme lo envía. A su hilo de señales le queda entonces una sola tarea, cancelar el envío cuando llega `SIGINT` o `SIGTERM`, que sigue haciendo falta porque `send()` sí puede bloquearse si el servidor está muy ocupado.

Por eso `wait_for_signals()`, la función de la Sección 3.7.3, se queda en el servidor con una referencia constante a una sola estructura `CopyProgress`, en lugar de a un vector de ellas, y en el cliente pierde ese parámetro y el caso de `SIGUSR1`, que tampoco tiene ya que entrar en el conjunto que bloquea su `block_signals()` (Sección 3.7.2). En el servidor sigue valiendo la advertencia de la Sección 3.7.3 sobre el porcentaje cuando `total_bytes` es `0`.

4.6. Comprobación

Para probar el sistema hacen falta dos terminales, el directorio de ejecución que las dos comparten y algún archivo lo bastante grande como para que dé tiempo a cancelar su copia:

```
$ export BACKUP_RUN_DIR=/tmp/backup-run
$ mkdir -p /tmp/backup-run /tmp/copias
$ echo "Contenido importante" > /tmp/test1.txt
$ dd if=/dev/urandom of=/tmp/bigfile.dat bs=1M count=1024 iflag=fullblock
```

En la primera terminal arrancamos el servidor y lo dejamos ahí, para ver sus mensajes:

```
$ backup-server /tmp/copias
backup-server: esperando conexiones en /tmp/backup-run/backup.sock
```

Antes de pedir nada conviene mirar el directorio de ejecución, donde deben haber aparecido el *socket* –que `ls -l` marca con una `s`– y el archivo del PID:

```
$ ls -l /tmp/backup-run
srwxr-xr-x 1 user user 0 sep 12 10:05 backup.sock
-rw-r--r-- 1 user user 5 sep 12 10:05 backup-server.pid
```

Y en la segunda terminal, con la misma variable de entorno definida, pedimos la copia de los dos archivos a la vez, para comprobar que una sola conexión sirve para varias rutas:

```
$ backup /tmp/test1.txt /tmp/bigfile.dat
backup: solicitud enviada para /tmp/test1.txt
backup: solicitud enviada para /tmp/bigfile.dat
```

La prueba definitiva es que las copias son idénticas a los originales:

```
$ cmp /tmp/test1.txt /tmp/copias/test1.txt
$ cmp /tmp/bigfile.dat /tmp/copias/bigfile.dat
```

Y que el servidor se puede parar sin dejar nada atrás:

```
$ kill $(cat /tmp/backup-run/backup-server.pid)
$ ls /tmp/backup-run
```

4.6.1. Casos de prueba adicionales

Adicionalmente, es importante probar los siguientes escenarios:

- **Terminar el servidor mientras copia un archivo grande**, pidiendo la copia desde el cliente y pulsando `Ctrl-C` en la terminal del servidor mientras está copiando. Debe responder de inmediato y no dejar la copia a medias en el directorio:

  ```
  backup-server: copia cancelada, se ha eliminado
  ↪ /tmp/copias/bigfile.dat
  backup-server: terminando
  ```

- **Terminar el servidor cuando lleva un rato sin hacer nada**, comprobando que responde igual de rápido que si estuviera ocupado, que es lo que consigue esperar con un plazo (Sección 4.3.3).

- **Consultar el progreso del servidor** varias veces durante la copia de un archivo grande, comprobando que el contador avanza y que la copia no se detiene:

  ```
  $ kill -USR1 $(cat /tmp/backup-run/backup-server.pid)
  backup-server: copiando 268435456 de 1073741824 bytes (25%)
  ```

- **Guardar las copias comprimidas**, comprobando que el archivo lleva la extensión que le corresponde y que se descomprime bien:

```
$ backup-server -z /tmp/copias
$ backup /tmp/test1.txt
$ gunzip -c /tmp/copias/test1.txt.gz | cmp - /tmp/test1.txt
```

- **Arrancar dos servidores sobre el mismo directorio de ejecución**:

```
$ backup-server /tmp/copias
backup-server: error: ya hay un servidor en marcha con el PID 3721
```

- **Pedir una copia sin que haya ningún servidor**:

```
$ backup /tmp/test1.txt
backup: error: el servidor no está en marcha
```

- **Arrancar el servidor cuando quedó el archivo del *socket* de una ejecución anterior**, que es lo que ocurre si termina de forma anormal. El servidor debe arrancar igualmente, porque borra ese archivo antes del `bind()`:

```
$ kill -9 $(cat /tmp/backup-run/backup-server.pid)
$ backup-server /tmp/copias
backup-server: esperando conexiones en /tmp/backup-run/backup.sock
```

- **Pedir la copia de varios archivos cuando alguno no es válido**, comprobando que los demás sí se copian y que el cliente termina indicando el fallo:

```
$ backup /tmp/no-existe.txt /tmp/test1.txt
backup: error: /tmp/no-existe.txt no es un archivo regular
backup: solicitud enviada para /tmp/test1.txt
$ echo $?
1
```

- **Pedir la copia de un directorio**, que el cliente debe rechazar sin llegar a enviar nada.
- **Pedir copias desde dos clientes a la vez**, comprobando que ninguna solicitud se pierde aunque el servidor las atienda una detrás de otra:

```
$ backup /tmp/bigfile.dat & backup /tmp/test1.txt
```

- **No definir la variable de entorno**:

```
$ unset BACKUP_RUN_DIR
$ backup /tmp/test1.txt
backup: error: BACKUP_RUN_DIR no está definida
```

- **Apuntar con la variable a un directorio que no existe**, que el servidor debe rechazar al arrancar, antes de intentar crear nada:

```
$ BACKUP_RUN_DIR=/tmp/no-existe backup-server /tmp/copias
backup-server: error: /tmp/no-existe no es un directorio
```

- **Pedir la copia de un archivo que ya está en el directorio de copias**, que el servidor debe rechazar sin destruirlo, tal y como se comprobaba en la Unidad 1.

5. Sockets TCP/IP y servidores concurrentes

Vamos a sacar de la máquina el sistema de copias de la Unidad 4, para que el servidor pueda guardar los archivos que le envían desde otro equipo. Aprenderemos a usar *sockets* TCP/IP, a atender por varios de ellos a la vez, a transferir el contenido de un archivo por la red y a servir a varios clientes al mismo tiempo.

5.1. Objetivo

El sistema de la Unidad 4 tiene un límite que no se ve a simple vista: cliente y servidor comparten el sistema de archivos. Por eso al servidor le basta con recibir una ruta, abrir él mismo ese archivo y copiarlo. En cuanto los dos programas se ejecutan en máquinas distintas eso deja de funcionar, porque la ruta que el cliente conoce no significa nada para el otro proceso.

Vamos a arreglarlo cambiando las dos cosas que lo impiden: como se comunican y qué comunican.

`backup-server` y `backup` deben cumplir con los siguientes requisitos:

- El servidor atenderá **a la vez** por dos *sockets* que escuchan: el de dominio UNIX de la Unidad 4 y otro nuevo `AF_INET` del tipo `SOCK_STREAM`, es decir, TCP (Sección I.1). La espera por los dos se hace con una sola llamada a `poll()`, como se explica en la Sección I.6.
- La dirección y el puerto en los que escucha el servidor, y los que usa el cliente para encontrarlo, serán configurables, tal y como se describe en la Sección 5.5.1.
- El cliente elige el transporte. Si se le indica una dirección de red usa TCP y, si no, el *socket* de dominio UNIX, exactamente como hasta ahora.
- El cliente enviará por la conexión **el contenido** de cada archivo, además de su ruta, con el protocolo que describe la Sección 5.3.2. Ya no abre el archivo el servidor, sino el cliente.
- El servidor recibirá ese contenido y lo guardará en el directorio de copias, comprimido si se le pidió al arrancarlo.
- El servidor atenderá a **varios clientes al mismo tiempo**, cada uno en su propio hilo, sin que una transferencia grande impida aceptar nuevas conexiones. El número de conexiones simultáneas estará limitado, por los motivos que explica la Sección 5.3.3.

- Las transferencias serán cancelables en los dos extremos, y una copia cancelada o incompleta **no debe quedarse** en el directorio de copias.
- Cuando el servidor termine, todos sus hilos deben haber terminado, todos los descriptores –incluidos los de los *sockets*– deben estar cerrados y el directorio de ejecución debe quedar como estaba.
- Para todo ello se usarán exclusivamente las llamadas al sistema del Apéndice I y las que ya se venían usando.

> **!** Importante
>
> Siguen vigentes todas las restricciones de las unidades anteriores: los archivos se manipulan con las llamadas al sistema, nunca con `std::ifstream` ni `std::FILE`; la cancelación es **cooperativa**; los hilos se crean con `std::jthread` y no se separan con `detach()`; y las señales no se manejan con `sigaction()`.

5.2. Línea de comandos y funcionamiento

El servidor gana dos opciones para decir dónde escucha:

```
backup-server [-z | -j | -x] [-l DIRECCIÓN] [-p PUERTO] [DIRECTORIO]
```

`-l DIRECCIÓN` **(opcional)** La dirección IP en la que aceptar conexiones. Por omisión `127.0.0.1`, que solo permite conectarse desde la propia máquina.

`-p PUERTO` **(opcional)** El puerto TCP en el que escuchar. Por omisión `9000`.

Y el cliente gana una para decir con quién habla:

```
backup [-s DIRECCIÓN[:PUERTO]] ARCHIVO...
```

`-s DIRECCIÓN[:PUERTO]` **(opcional)** La dirección IP del servidor. Si se indica, el cliente se conecta por TCP; si no, usa el *socket* de dominio UNIX del directorio de ejecución, como en la Unidad 4. Cuando no se indica el puerto se usa el mismo `9000` por omisión.

Las tres se pueden fijar también en el entorno, con la precedencia que explica la Sección 5.5.1. El resto de opciones, y `BACKUP_RUN_DIR`, siguen significando lo mismo que en la Unidad 4.

Así, un servidor que acepta copias desde cualquier equipo de la red se arranca indicándolo de forma explícita:

```
$ export BACKUP_RUN_DIR=/tmp/backup-run
$ backup-server -l 0.0.0.0 -p 9000 /tmp/copias
backup-server: esperando conexiones en /tmp/backup-run/backup.sock y en
↪ 0.0.0.0:9000
```

Y desde otra máquina se le pide una copia con `-s`:

```
$ backup -s 192.168.1.5:9000 informe.odt
backup: enviado /home/user/informe.odt (2411520 bytes)
```

Mientras que en la misma máquina se le sigue pidiendo igual que antes, sin `-s`, y la copia viaja por el *socket* de dominio UNIX:

```
$ backup informe.odt
backup: enviado /home/user/informe.odt (2411520 bytes)
```

En la terminal del servidor los dos casos se ven igual, salvo por quién los pidió:

```
backup-server: [2] recibido /home/user/informe.odt de 192.168.1.9 ->
↪ /tmp/copias/informe.odt
backup-server: [1] recibido /home/user/informe.odt de backup.sock ->
↪ /tmp/copias/informe.odt
```

El número entre corchetes es la conexión que atendió cada copia, y es lo que permite seguir el rastro cuando hay varias a la vez. Es también lo que muestra `SIGUSR1`, que ahora informa de todas las transferencias en curso:

```
$ kill -USR1 $(cat /tmp/backup-run/backup-server.pid)
backup-server: [1] 268435456 de 1073741824 bytes (25%)
backup-server: [2] 52428800 de 104857600 bytes (50%)
```

5.3. Arquitectura del sistema

El servidor deja de tener un solo punto de entrada. Escucha por dos *sockets* a la vez y, por cada conexión que acepta en cualquiera de ellos, pone a trabajar un hilo, como muestra la Figura 5.1.

Lo que no cambia es tan importante como lo que cambia. El protocolo, las funciones que lo implementan y el hilo que atiende una conexión son los mismos vengan los bytes de donde vengan: en cuanto `accept()` entrega un descriptor, nada del resto del programa necesita saber si detrás hay un archivo del sistema de archivos o una tarjeta de red. Es a esto a lo que se refiere la Apéndice I cuando dice que un *socket* abstrae la tecnología de comunicación.

5.3.1. Dos transportes a la vez

Los dos *sockets* que escuchan se preparan igual –`socket()`, `bind()` y `listen()`– y solo se diferencian en la dirección que se les asigna: una ruta del sistema de archivos en el dominio UNIX (Sección I.2.2) y una IP con un puerto en `AF_INET` (Sección I.2.1).

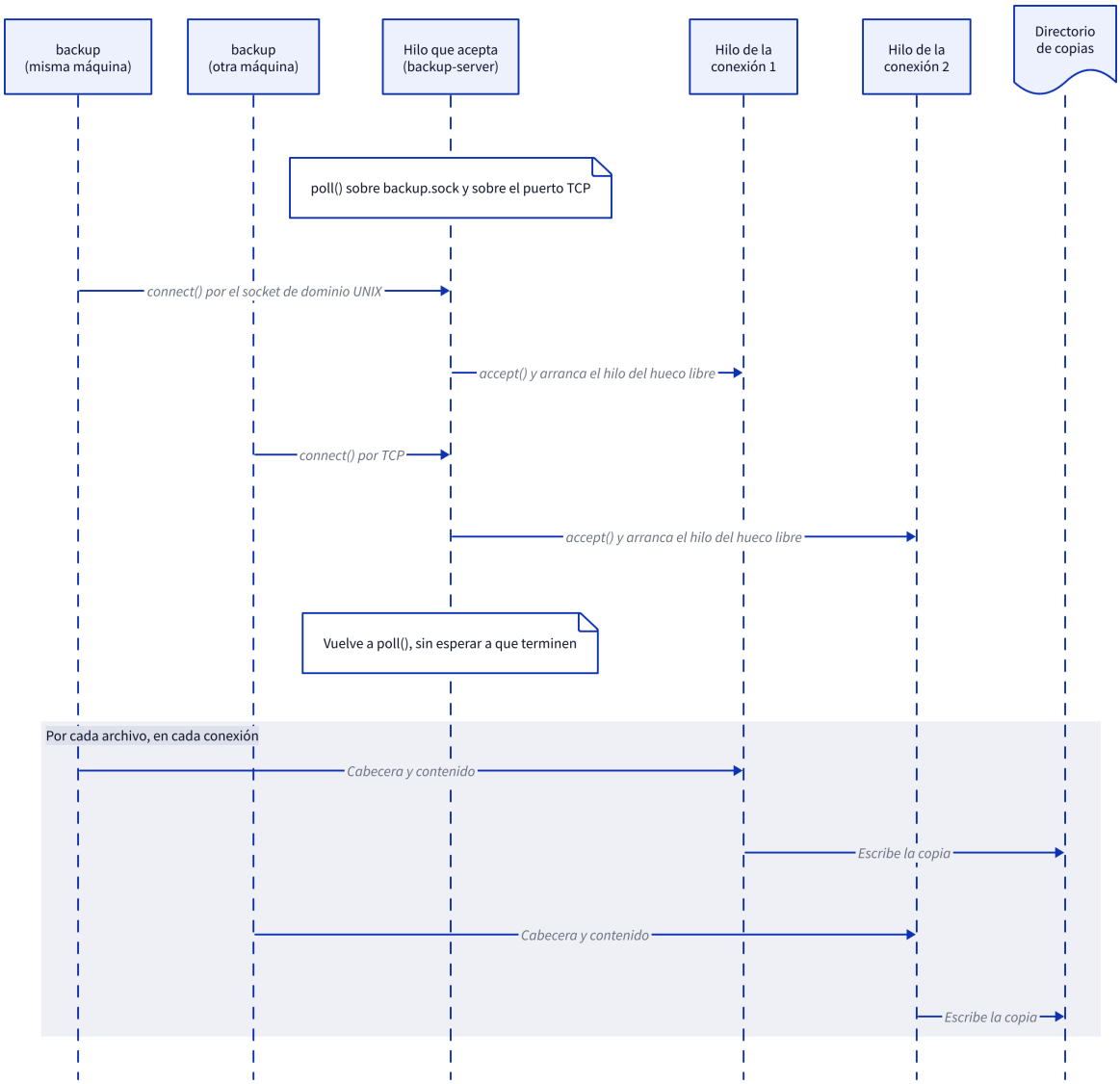


Figura 5.1.: El servidor atendiendo por sus dos *sockets* a dos clientes a la vez

Esperar por los dos a la vez es exactamente para lo que sirve `poll()`, al que se le pasa un `pollfd` por descriptor en lugar de uno solo (Sección I.6). Cuando retorna hay que mirar **cuál** de ellos está listo, porque de eso depende sobre qué descriptor se llama a `accept()`.

> **!** Importante
>
> Escuchar en `0.0.0.0` –la constante `INADDR_ANY`– significa aceptar conexiones de cualquiera que alcance la máquina por la red. Por eso el valor por omisión es `127.0.0.1` (`INADDR_LOOPBACK`). Escuchar públicamente tiene que pedirse explícitamente, tal y como advierte la Sección I.2.1.

5.3.2. El protocolo

Por la conexión ya no viaja solo la ruta. Por cada archivo, el cliente envía:

1. Una **cabecera**, que es una línea de texto terminada en `\n` con el tamaño del archivo en bytes, un espacio y su ruta absoluta.
2. El **contenido**, que son exactamente esos bytes, sin nada más.

```
2411520 /home/user/informe.odt\n
<2411520 bytes del archivo>
```

La cabecera es un mensaje como los de la Sección 4.3.2, así que `send_line()` y `recv_line()` valen tal cual para enviarla y recibirla. El tamaño va **antes** que la ruta porque una ruta puede contener espacios y así el receptor sabe siempre dónde termina el número: lo que va detrás del primer espacio es la ruta entera, hasta el `\n`.

> **!** Importante
>
> El tamaño no es un adorno del que se pueda prescindir. Sin él, el que recibe no tiene forma de saber dónde acaba un archivo y empieza la cabecera del siguiente, porque el flujo de un `SOCK_STREAM` no tiene marcas de inicio y fin de mensaje (Sección I.1) y por esa misma conexión van a viajar varios archivos, así que tampoco sirve usar el cierre de la conexión como delimitador.

Saber el tamaño de antemano tiene además dos ventajas que se aprovechan enseguida: el servidor puede informar del progreso de lo que está recibiendo, y puede detectar que una transferencia se quedó a medias –le prometieron más bytes de los que llegaron– y borrar el archivo incompleto.

La ruta sigue siendo absoluta y sigue sirviendo para lo mismo: el servidor se queda con su último componente para construir el nombre de la copia, con `make_dest_path()` (Sección 3.7.1). Lo que ya no hace, y es la diferencia de fondo con la Unidad 4, es abrir ese archivo.

5.3.3. El servidor concurrente

Transferir un archivo grande por la red lleva su tiempo, y atender una conexión hasta el final antes de aceptar la siguiente –como hacía el servidor de la Unidad 4– deja esperando a todo el mundo. Así que el hilo que acepta conexiones ya no se encarga de copiar. En su lugar, en cuanto tiene un descriptor para una nueva conexión, arranca un hilo para dicha conexión y vuelve a esperar.

El número de conexiones simultáneas está limitado a `max_connections`, y esa limitación tiene varias ventajas:

- Un servidor que arranca un hilo por cada conexión que le llegue es un servidor al que se tumba abriendo más conexiones de las que puede manejar.
- El hilo de señales tiene que recorrer el progreso de todas las conexiones activas para responder a `SIGUSR1`. Si el conjunto creciera y menguara dinámicamente durante la ejecución –por ejemplo, usando métodos como `std::vector::push_back`– habría que protegerlo con un mecanismo de sincronización de los que menciona la Sección G.3. Con un número fijo de huecos, reservados antes de crear ningún hilo, el problema desaparece: es la misma regla de la Sección G.2, y la misma que obligaba en la Sección 3.5 a dimensionar el vector de antemano.

Cada hueco guarda el progreso de su conexión y el `std::jthread` que la atiende. Su posición en el vector, contada desde uno, es además el número con el que el servidor identifica esa conexión en todos sus mensajes –el `[1]` y el `[2]` de la Sección 5.2–.

Cuántos huecos hay es una constante del programa:

```
constexpr std::size_t max_connections = 4;
```

Cuando no queda ninguno libre, las conexiones que llegan esperan en la cola de `listen()` (Sección I.5.1).

5.4. Manejo de errores

Se mantienen las pautas de la Sección 4.4, y a ellas se añaden las situaciones propias de esta actividad.

Impiden arrancar y terminan el programa antes de crear ningún hilo:

- Que la dirección o el puerto configurados no sean válidos.
- En el servidor, que falle la preparación de **cualquiera** de los dos *sockets*. Si uno de los dos no se puede poner a escuchar, el servidor no arranca a medias. El caso más frecuente es un `bind()` que falla con `EADDRINUSE` porque el puerto está ocupado.

- En el cliente, que falle `connect()`. Sobre TCP los errores típicos son distintos de los del dominio UNIX. Por ejemplo, son muy comunes errores como `ECONNREFUSED`, si no hay nadie escuchando en ese puerto, y `ETIMEDOUT` o `EHOSTUNREACH` si no se puede llegar a la máquina del servidor (Sección I.5.3).

Y no terminan el programa:

- Que una transferencia falle o se cancele. El hilo que la estuviera recibiendo cierra la conexión, **elimina la copia incompleta** con `unlink()` –como se hacía en la Sección 3.4.2– y termina, dejando libre su hueco. El servidor sigue aceptando conexiones mientras tanto, porque de eso se ocupa otro hilo.
- Que un cliente se muera a mitad del envío. El servidor lo ve como un `recv()` que retorna `0` antes de haber reunido los bytes prometidos en la cabecera, y lo trata como cualquier otra transferencia incompleta.
- Que la cabecera venga mal formada, sin un número delante o con un tamaño que no es un número. El servidor cierra esa conexión, porque a partir de ahí ya no sabe dónde empieza el mensaje siguiente.

> **Nota**
>
> Con varios hilos escribiendo por su cuenta en la salida de error, sigue valiendo la advertencia de la Sección 3.7.6: compón cada mensaje entero con `std::format` y escríbelo en una sola operación, o las líneas de unas conexiones y otras acabarán entremezcladas. Por eso, además, cada mensaje del servidor lleva delante el número de su conexión.

5.5. Implementación

5.5.1. La configuración

La dirección y el puerto se pueden indicar de dos maneras, y se resuelven siempre en el mismo orden:

1. La opción de la línea de comandos, si se indicó.
2. La variable de entorno que le corresponde, si está definida.
3. El valor por omisión.

La Tabla 5.1 recoge de dónde sale el valor de cada uno:

Tabla 5.1.: Los parámetros configurables y de dónde sale su valor

Parámetro	Opción	Variable	Por omisión
Dirección en la que escucha el servidor	`-l`	`BACKUP_LISTEN_ADDR`	`127.0.0.1`

Parámetro	Opción	Variable	Por omisión
Puerto en el que escucha el servidor	`-p`	`BACKUP_LISTEN_PORT`	`9000`
Dirección del servidor a la que se conecta el cliente	`-s`	`BACKUP_SERVER`	sin TCP

Esa precedencia es la habitual en las herramientas de línea de comandos, y resuelve dos necesidades distintas: la variable de entorno fija el valor que se usa casi siempre –en el arranque automático de un servicio, o en la sesión de quien copia siempre al mismo sitio– y la opción permite apartarse de él en una ejecución concreta –por ejemplo, durante el desarrollo– sin tocar el entorno.

Para implementarla basta una función por parámetro, o una que reciba las tres piezas:

```
std::string get_setting(const std::optional<std::string>& option,
    const std::string& variable, const std::string& fallback);
```

Si `option` tiene valor, lo devuelve; si no, consulta `variable` con `get_environment_variable()` (Sección B.1); y si tampoco está definida, devuelve `fallback`. Que el valor de la opción sea un `std::optional` es lo que permite distinguir «no se indicó» de «se indicó una cadena vacía», así que `parse_arguments()` (Sección 2.4.1) debe guardarlo así en `Options`.

> **Nota**
>
> `BACKUP_RUN_DIR` no entra en esta tabla y sigue siendo solo una variable de entorno, sin opción que la acompañe. Es una convención local que cliente y servidor tienen para encontrarse, y no algo que cambie de una ejecución a otra, tal y como se explicó en la Sección 4.3.1.

El puerto y la dirección hay que convertirlos antes de usarlos. El primero, de texto a número con `std::from_chars()`, como ya se hizo con el PID en la Sección 4.5.7, comprobando además que cae entre 1 y 65535. La segunda, con `inet_aton()`, que informa de que la cadena no contiene una dirección válida retornando `0` (Sección I.2.1).

La dirección del cliente, además, hay que partirla antes por el `:` para separar la del servidor del puerto, teniendo en cuenta que el puerto puede no estar.

> **Importante**
>
> `inet_aton()` solo entiende direcciones escritas en notación decimal con puntos, como `192.168.1.5`. Un nombre como `localhost` o `servidor.ull.es` **no vale**, y debe rechazarse como cualquier otra dirección inválida. Traducir nombres a direcciones es trabajo de `getaddrinfo()`, que queda fuera de esta práctica.

5.5.2. El servidor

El `main()` del servidor es el de la Sección 4.5.1 con dos *sockets* donde había uno y con los huecos de las conexiones preparados antes de crear los hilos:

```
options = parse_arguments(argc, argv)
if has_error(options):
    error("argumentos inválidos")

# Igual que en la unidad anterior: el directorio de copias, el comando
# de compresión, BACKUP_RUN_DIR y la comprobación de que no hay ya
# otro servidor en marcha...

listen_addr = get_setting(options.listen_addr,                          ①
                          "BACKUP_LISTEN_ADDR", "127.0.0.1")
listen_port = get_setting(options.listen_port,
                          "BACKUP_LISTEN_PORT", "9000")

unix_fd = create_unix_socket(run_dir)                                   ②
if has_error(unix_fd):
    error("no se pudo crear el socket de dominio UNIX")

inet_fd = create_inet_socket(listen_addr, listen_port)
if has_error(inet_fd):
    error("no se pudo crear el socket TCP")

if has_error(write_pid_file(run_dir)):                                  ③
    error("no se pudo escribir el archivo del PID")

signal_set = block_signals()
if has_error(signal_set):
    error("no se pudieron bloquear las señales")

backup_stop = stop_source()
connections = [Connection()                                             ④
               for _ in range(max_connections)]

signal_thread = start_thread(wait_for_signals,
                             signal_set, backup_stop, connections)

accept_thread = start_thread(accept_worker, options,                    ⑤
                             compression_path, [unix_fd, inet_fd],
                             connections, backup_stop.get_token())

accept_failed = accept_thread.wait()
```

```
close(unix_fd)                                                    ⑥
close(inet_fd)
remove_socket_file(run_dir)
remove_pid_file(run_dir)

if accept_failed:
    return EXIT_FAILURE
```

① La configuración se resuelve antes de crear nada, para que un puerto mal escrito no deje a medias el arranque.
② Si el segundo *socket* falla hay que cerrar el primero antes de retornar, junto con el resto de la limpieza de la Sección 4.4.
③ El archivo del PID se escribe después de los dos *sockets*, y no antes, para que un `bind()` que falle no deje atrás un PID anunciando un servidor que no llegó a arrancar.
④ Los huecos de las conexiones se reservan aquí, antes que ningún hilo, por el motivo que explica la Sección 5.3.3. Como contienen variables atómicas, ni se copian ni se mueven, así que el vector se crea ya con su tamaño definitivo (Sección G.3).
⑤ El hilo que acepta ya no copia nada: solo reparte conexiones entre los huecos.
⑥ Antes de llegar aquí hay que esperar a los hilos de las conexiones, que pueden seguir recibiendo archivos aunque el que acepta ya haya terminado. Esa espera tampoco se programa: ocurre al destruirse `connections`, así que este debe declararse en un ámbito que se cierre **antes** de la limpieza, con el mismo recurso que la Sección 3.7.7 usaba para ordenar los hilos de `copy`.

5.5.3. El cliente

El del cliente cambia todavía menos, porque la diferencia entre los dos tipos de *socket* queda dentro de una función:

```
options = parse_arguments(argc, argv)
if has_error(options):
    error("argumentos inválidos")

server = get_setting(options.server, "BACKUP_SERVER", "")

conn_fd = connect_to_server(server)                               ①
if has_error(conn_fd):
    error("no se pudo conectar con el servidor")

signal_set = block_signals()                                      ②
if has_error(signal_set):
    error("no se pudieron bloquear las señales")

send_stop = stop_source()
```

```
signal_thread = start_thread(wait_for_signals,
                             signal_set, send_stop)

send_thread = start_thread(send_worker, options, conn_fd,
                           send_stop.get_token())

send_failed = send_thread.wait()

close(conn_fd)

if send_failed:
    return EXIT_FAILURE
```

① Con `server` vacío se conecta por el *socket* de dominio UNIX, leyendo `BACKUP_RUN_DIR` como en la Sección 4.5.4, y con cualquier otro valor por TCP. A partir del descriptor que retorna, el resto del cliente es idéntico en los dos casos.
② El conjunto del cliente sigue siendo el de la Sección 4.5.11, sin `SIGUSR1`. Su hilo de señales solo cancela el envío, y por eso tampoco recibe ninguna estructura de progreso.

5.5.4. Los sockets del servidor

La `create_server_socket()` de la Sección 4.5.3 se queda como está, con el nombre `create_unix_socket()` para distinguirla de la nueva:

```
std::expected<int, std::system_error> create_inet_socket(
    const std::string& address, const std::string& port);
```

Hace lo mismo que su pareja –`socket()`, `bind()` y `listen()`– con tres diferencias:

- El dominio es `AF_INET` y la dirección se construye en un `sockaddr_in`, con `htons()` para el puerto y `inet_aton()` para la IP, tal y como se explica en la Sección I.2.1.
- No hay ningún archivo que borrar antes ni después, porque la dirección no está en el sistema de archivos.
- Hay que activar `SO_REUSEADDR` **antes** del `bind()`, o reiniciar el servidor fallará con `EADDRINUSE` durante unos minutos, por el motivo que explica la Sección I.5.1. Esta es la diferencia que más desconcierta al probar, porque en el dominio UNIX no ocurre.

5.5.5. La conexión del cliente

```
std::expected<int, std::system_error> connect_to_server(
    const std::string& server);
```

Decide el transporte y conecta. Con `server` vacío hace lo de la Sección 4.5.4; en otro caso separa la dirección del puerto, construye la dirección `AF_INET` y llama a `connect()` (Sección I.5.3).

Retorna el descriptor de la conexión, y es el único sitio del cliente donde se menciona una familia de direcciones.

5.5.6. Esperar por varios descriptores

La `wait_for_input()` de la Sección 4.5.6 se generaliza para esperar por varios descriptores y decir cuál está listo:

```
std::expected<std::optional<int>, std::system_error> wait_for_input(
    std::span<const int> fds, std::stop_token stop_token);
```

Retorna el descriptor por el que ya se puede leer sin bloquearse, o `std::nullopt` si se solicitó la cancelación mientras esperaba. Construye un `pollfd` por cada descriptor de `fds`, todos con `POLLIN`, y al volver de `poll()` busca el primero cuyo campo `revents` tenga ese bit.

`std::span`, declarada en `<span>`, es a las secuencias lo que `std::string_view` –la que se sugirió en la Sección 2.4.3– es a las cadenas: no es dueña de nada, solo apunta a unos elementos contiguos que ya existen en otro contenedor y sabe cuántos son. Aquí eso resuelve un problema concreto. El servidor tiene dos descriptores en un `std::array<int, 2>`, porque tiene que escuchar por dos *sockets*:

```
std::array<int, 2> listen_fds{unix_fd, inet_fd};                    ①
auto ready = wait_for_input(listen_fds, stop_token);
```

① El `std::array` se convierte solo en el `std::span`, y lo mismo haría un `std::vector`.

Mientras que los hilos de las conexiones solo esperan por un descriptor, dentro de `recv_line()`:

```
auto ready = wait_for_input({&conn_fd, 1}, stop_token);             ①
```

① Un descriptor suelto se pasa como la dirección del primer elemento y el número de elementos, sin construir ningún contenedor.

> **Nota**
>
> Si hay varios descriptores listos a la vez, esta función retorna siempre el primero de la lista, así que el *socket* que esté antes se atiende siempre primero. Con dos y este volumen de conexiones da igual, pero conviene saber que la solución general sería repartir el turno con un mecanismo de *round-robin*.

5.5.7. Enviar y recibir archivos

El contenido de un archivo no es una línea ni cabe en una, así que no lo mueven `send_line()` ni `recv_line()`, sino un par de funciones propio. Son las dos mitades en que queda partida la `copy_file()` de la Unidad 1 cuando el origen y el destino están en máquinas distintas: una lee del archivo y escribe en la conexión, y la otra lee de la conexión y escribe en el archivo.

```
std::expected<void, std::system_error> send_file(int conn_fd,
    const std::string& src_path, std::uintmax_t size,
    std::stop_token stop_token);
```

Abre `src_path`, lee bloques de 128 KiB y los envía por la conexión con `send()`, hasta completar los `size` bytes que anunció la cabecera. Comprueba el *token* entre bloque y bloque, exactamente donde lo hacía `copy_file()` (Sección 3.4.1). No lleva ninguna cuenta del progreso, a diferencia de su pareja, que se usa en el servidor.

```
std::expected<void, std::system_error> recv_file(int conn_fd,
    const std::string& dest_path, std::uintmax_t size,
    CopyProgress& progress, std::stop_token stop_token);
```

Crea `dest_path` y recibe de la conexión exactamente `size` bytes, escribiéndolos conforme llegan. Si se cancela, si falla o si el cliente cierra antes de tiempo –`recv()` retorna `0` con bytes pendientes por leer–, la función cierra el archivo de destino y lo **elimina** con `unlink()`, según la regla de la Sección 3.4.2.

> **Importante**
>
> Ni `send()` ni `recv()` mueven necesariamente todos los bytes que se les piden, y con los *sockets* eso es la norma, no la excepción (Sección I.5.4). Las dos funciones tienen que llevar la cuenta de lo que llevan transferido y seguir pidiendo el resto.
> Cuidado además con el final. `recv()` no puede pedir más de lo que queda del archivo, o se llevaría por delante la cabecera del siguiente. El tamaño del bloque que se pide es el menor entre los 128 KiB del búfer y lo que falte.

Cuando el servidor guarda las copias comprimidas hace falta la variante correspondiente:

```
std::expected<void, std::system_error> recv_file_compressed(int conn_fd,
    const std::string& dest_path, std::uintmax_t size,
    const std::string& compression_path,
    CopyProgress& progress, std::stop_token stop_token);
```

Es la `copy_file_compressed()` de la Sección 2.4.5 con el origen cambiado: en lugar de abrir un archivo y leerlo hasta el final, lee `size` bytes de la conexión y los escribe en la tubería del compresor. Todo lo demás –el `pipe()`, el `fork()`, las redirecciones, el `execv()`, el `waitpid()` y los códigos de salida de la Sección 2.4.5.1– se queda igual, incluida la forma de cancelarla que describe la Sección 3.4.3.

5.5.8. El hilo que acepta conexiones

Sustituye al `backup_worker()` de la Sección 4.5.8 y ya no copia nada:

```
void accept_worker(const Options& options,
    const std::string& compression_path,
    std::span<const int> listen_fds,
    std::vector<Connection>& connections, bool& failed,
    std::stop_token stop_token);
```

Su bucle es más corto que el de la unidad anterior, porque el trabajo se lo lleva otro hilo:

```
while True:
    slot = find_free_slot(connections, stop_token)                    ①
    if not has_value(slot):
        break

    ready_fd = wait_for_input(listen_fds, stop_token)
    if has_error(ready_fd):
        print(stderr, "error al esperar conexiones: {error}")
        failed = True
        break

    if not has_value(ready_fd):
        break

    conn_fd, client_name = accept(ready_fd)                           ②
    if has_error(conn_fd):
        print(stderr, "error al aceptar la conexión: {error}")
        continue

    start_connection(connections[slot], conn_fd,                      ③
```

```
                    client_name, slot + 1, options,
                    compression_path, stop_token)
```

① Buscar el hueco **antes** de esperar la conexión evita aceptarla para tener que dejarla parada enseguida. Si no hay ninguno libre, la función espera con un plazo y vuelve a mirar, comprobando el *token* como todas las esperas de la Sección 4.3.3, y retorna `std::nullopt` si lo que ha ocurrido es que se ha pedido la parada.
② En `accept()` conviene recoger también la dirección del cliente, que es lo que el servidor muestra en sus mensajes (Sección I.5.2).
③ Asignarle al hueco su `std::jthread` nuevo destruye el anterior, que es el de la conexión que ya terminó, y esperarlo no cuesta nada porque hace rato que acabó (Sección G.1).

Las dos funciones que manejan los huecos son:

```
std::optional<std::size_t> find_free_slot(
    const std::vector<Connection>& connections,
    std::stop_token stop_token);
```

Retorna el índice del primer hueco cuyo `active` está a `false`, esperando a que se libere alguno si están todos ocupados, o `std::nullopt` si se solicitó la parada mientras esperaba.

```
void start_connection(Connection& connection, int conn_fd,
    const std::string& client_name, std::size_t number,
    const Options& options, const std::string& compression_path,
    std::stop_token stop_token);
```

Marca el hueco como ocupado y le asigna un `std::jthread` nuevo que ejecuta `connection_worker()`, al que le pasa el progreso y el `active` del hueco junto con todo lo demás. El orden importa: `active` se pone a `true` **antes** de arrancar el hilo, porque si se hiciera después el hilo que acepta podría entregarle el mismo hueco a la conexión siguiente.

5.5.9. El hilo de cada conexión

Es el bucle interior del `backup_worker()` de la Sección 4.5.8, sacado a su propia función y con el contenido de los archivos añadido:

```
void connection_worker(int conn_fd, const std::string& client_name,
    std::size_t number, const Options& options,
    const std::string& compression_path,
    CopyProgress& progress, std::atomic<bool>& active,
    std::stop_token stop_token);
```

`client_name` es la dirección que devolvió `accept()`, ya convertida en texto con `inet_ntoa()`, o el nombre del *socket* de dominio UNIX cuando la conexión llegó por ahí. Solo sirve para hacer más informativos los mensajes, pero es lo que permite saber de dónde vino cada copia. Y `number` es el número de la conexión (Sección 5.3.3), que encabeza esos mismos mensajes.

Recibe cabeceras hasta que el cliente cierra la conexión y, por cada una:

1. La parte en el tamaño y la ruta, según la Sección 5.3.2. Una cabecera que no se pueda interpretar termina la conexión.
2. Construye la ruta de destino con `make_dest_path()`, marca `in_progress` y pone `total_bytes` al tamaño anunciado.
3. Llama a `recv_file()` o a `recv_file_compressed()`, según las opciones con las que se arrancó el servidor.
4. Muestra el resultado, con el número de su conexión delante y con `client_name`.

La comprobación de `is_same_file()` que hacía el bucle de la Sección 4.5.8 ya no tiene sentido y desaparece: el origen no es un archivo que el servidor pueda abrir, sino una conexión.

Al terminar cierra `conn_fd` y pone `active` a `false`, que es lo que le dice al hilo que acepta que su hueco vuelve a estar libre. Como cualquier función principal de un hilo, todo su cuerpo va dentro de un `try/catch`, por lo que se explicó en la Sección 3.6, y ese `active` tiene que quedar a `false` también cuando se sale por una excepción.

5.5.10. El hilo de envío del cliente

Es el de la Sección 4.5.9 con un paso más por archivo. Después de comprobar que es un archivo regular y de convertir su ruta en absoluta, ahora:

1. Consulta su tamaño con `stat()`, como se hacía en la Sección 3.5, porque es lo que anuncia la cabecera.
2. Envía la cabecera con `send_line()`.
3. Envía el contenido con `send_file()`.

Un fallo en cualquiera de los dos envíos deja la conexión en un estado que no permite seguir adelante, así que el hilo lo informa, marca el fallo y deja de enviar los archivos que quedaran.

> **Nota**
>
> El tamaño se consulta una vez, antes de enviar la cabecera, y es el que se promete. Si el archivo cambia de tamaño mientras se envía, lo que vale es lo prometido: hay que enviar exactamente esos bytes, rellenando de menos nunca y de más tampoco. Un archivo que encoge a mitad del envío es, por tanto, un error de la transferencia.

5.5.11. Informe de progreso

La estructura es exactamente la de la Sección 4.5.11, sin la ruta del archivo y por el mismo motivo que allí. Lo que cambia es que ya no hay una, sino una por conexión, guardada en el hueco que la atiende:

```
struct CopyProgress
{
    std::atomic<bool> in_progress = false;
    std::atomic<std::uintmax_t> total_bytes = 0;
    std::atomic<std::uintmax_t> copied_bytes = 0;
};

struct Connection
{
    std::atomic<bool> active = false;                                    ①
    CopyProgress progress;
    std::jthread thread;                                                 ②
};
```

① Lo pone a `true` el hilo que acepta al arrancar la conexión y a `false` el de la conexión al terminar. Es lo único que comparten los dos, y por eso basta con que sea atómico.
② El hueco es el dueño del hilo, así que destruir el vector en `main()` espera por todas las conexiones que sigan vivas.

Estas estructuras son solo para el servidor. En su `wait_for_signals()` entra el vector de `Connection` y al recibir `SIGUSR1` muestra una línea por cada una que esté activa, con su número delante. El cliente conserva la versión de `wait_for_signals()` de la Sección 4.5.11, sin parámetro de progreso y sin el caso de `SIGUSR1`.

Sigue valiendo la advertencia de la Sección 3.7.3 sobre el porcentaje cuando `total_bytes` es `0`.

5.6. Comprobación

Lo primero es comprobar que lo de la unidad anterior sigue funcionando, ahora que el contenido viaja por la conexión:

```
$ export BACKUP_RUN_DIR=/tmp/backup-run
$ mkdir -p /tmp/backup-run /tmp/copias
$ dd if=/dev/urandom of=/tmp/bigfile.dat bs=1M count=1024 iflag=fullblock
$ backup-server /tmp/copias &
$ backup /tmp/bigfile.dat
$ cmp /tmp/bigfile.dat /tmp/copias/bigfile.dat
```

Y a continuación, que por TCP se obtiene exactamente el mismo resultado:

```
$ backup -s 127.0.0.1:9000 /tmp/bigfile.dat
$ cmp /tmp/bigfile.dat /tmp/copias/bigfile.dat
```

Para probarlo de verdad entre dos máquinas hace falta arrancar el servidor con `-l 0.0.0.0` y usar desde la otra la IP del equipo, que se averigua con `ip address`. Si no hay una segunda máquina a mano, dos terminales y `127.0.0.1` sirven para todo lo demás.

Finalmente, que el servidor atiende de verdad a varios a la vez, que es lo que esta unidad añade. Lanzando dos clientes con archivos grandes y preguntando por el progreso deben aparecer **las dos** transferencias avanzando:

```
$ backup /tmp/bigfile.dat & backup -s 127.0.0.1:9000 /tmp/bigfile.dat &
$ kill -USR1 $(cat /tmp/backup-run/backup-server.pid)
backup-server: [1] 268435456 de 1073741824 bytes (25%)
backup-server: [2] 159383552 de 1073741824 bytes (14%)
```

Si solo aparece una, el servidor las está atendiendo en fila y no en paralelo.

5.6.1. Casos de prueba adicionales

Adicionalmente, es importante probar los siguientes escenarios:

- **Arrancar el servidor con el puerto ocupado**, que debe informar y no arrancar:

  ```
  $ backup-server -p 9000 /tmp/copias &
  $ backup-server -p 9000 /tmp/copias
  backup-server: error: no se pudo crear el socket TCP: Address already
  ↪ in use
  ```

- **Reiniciar el servidor inmediatamente**, que debe poder volver a escuchar en el mismo puerto gracias a `SO_REUSEADDR` (Sección 5.5.4).

- **Pedir una copia a un puerto en el que no hay nadie**:

  ```
  $ backup -s 127.0.0.1:9999 /tmp/bigfile.dat
  backup: error: no se pudo conectar con el servidor: Connection refused
  ```

- **Comprobar la precedencia de la configuración**, que la opción gana a la variable de entorno (Sección 5.5.1):

  ```
  $ export BACKUP_SERVER=127.0.0.1:9999
  $ backup -s 127.0.0.1:9000 /tmp/bigfile.dat
  ```

- **Matar al cliente a mitad de una transferencia grande**, comprobando que el servidor detecta la copia incompleta, la borra y sigue aceptando conexiones:

```
$ backup /tmp/bigfile.dat &
$ kill -9 %1
$ ls /tmp/copias
```

- **Cancelar el envío desde el cliente** con `Ctrl-C` durante una transferencia grande, que debe dejar el directorio de copias igual de limpio.
- **Terminar el servidor mientras recibe de varios clientes**, comprobando que ninguna copia a medias se queda en el directorio y que no queda ningún proceso del compresor sin recoger.
- **Superar el límite de conexiones simultáneas**, lanzando más clientes que `max_connections` y comprobando que todos acaban siendo atendidos, aunque unos esperen a otros.
- **Guardar comprimidas las copias que llegan por la red**:

```
$ backup-server -z /tmp/copias &
$ backup -s 127.0.0.1:9000 /tmp/texto.txt
$ gunzip -c /tmp/copias/texto.txt.gz | cmp - /tmp/texto.txt
```

- **Copiar un archivo vacío**, cuya cabecera anuncia `0` bytes y cuyo contenido no ocupa nada, que debe crear igualmente la copia.
- **Indicar una dirección o un puerto que no valen**, como `-p 70000` o `-s localhost` –que es un nombre, no una dirección–, que deben rechazarse al arrancar con un mensaje claro.

6. Gestión automática de los recursos del sistema

Vamos a cambiar la forma en que `backup` y `backup-server` de la Unidad 5 gestionan sus descriptores de archivo, sin modificar el comportamiento de ninguno de los dos programas. Aprenderemos qué es un *handle*, qué consecuencias tiene copiarlo y cómo envolverlo en una clase que lo cierre por nosotros, igual que `std::jthread` lleva desde la Unidad 3 esperando a nuestros hilos sin que se lo pidamos.

6.1. Objetivo

En todas las unidades anteriores los descriptores se han cerrado a mano. Cada `open()`, `pipe()`, `socket()` y `accept()` obliga a añadir un `close()` en **todos** los caminos por los que se puede salir de la función: el que copia el archivo entero, el que se encuentra un error a mitad y el que descubre que se ha cancelado la copia. Olvidarlo en uno solo de ellos provocaría una fuga de descriptores, y el problema es que en un servidor que atiende conexiones indefinidamente, una fuga acaba agotando los descriptores disponibles.

Vamos a hacer que ese trabajo lo haga el lenguaje por nosotros. El programa debe cumplir con los siguientes requisitos:

- Los descriptores de archivo se gestionarán con una clase `SafeFD`, escrita por nosotros, cuyo destructor cierre el descriptor que contenga.
- Ninguna otra parte del programa llamará a `close()`. Las llamadas al sistema seguirán recibiendo el `int` que esperan, obtenido del objeto que lo guarda.
- `SafeFD` **no se podrá copiar**, y el intento de hacerlo tendrá que ser un error de compilación, no un fallo en ejecución, por los motivos que se explican en la Sección 6.3.1.
- `SafeFD` **sí se podrá mover**, porque mover es lo que ocurre cuando un descriptor cambia de dueño.
- Las funciones que crean un descriptor lo retornarán como `SafeFD`; las que solo lo usan lo recibirán como `const SafeFD&`; y las que se quedan con él lo recibirán por valor.
- Los dos programas se comportarán exactamente igual que en la Unidad 5, opción por opción y mensaje por mensaje.
- Al terminar seguirán sin quedar descriptores abiertos, ni archivos a medias, ni procesos del compresor sin recoger, igual que se exigía en las unidades anteriores.

> **!** Importante
>
> Esta unidad no añade ninguna llamada al sistema. Todo lo que se pide se resuelve con mecanismos del lenguaje –constructores, destructores y semántica de movimiento– aplicados a las llamadas que ya se venían usando.
> Tampoco se pueden usar en su lugar clases de la librería estándar o de terceros que resuelvan el problema ya hecho, porque el objetivo es entender por qué la clase es como es.

6.2. Qué cambia y qué no

Nada de lo que el usuario ve debe cambiar –ni la línea de comandos, ni los mensajes, ni el protocolo, ni el directorio de ejecución, ni el comportamiento ante las señales–. Ese es precisamente el criterio para saber si esta unidad está bien resuelta.

Lo que cambia es un detalle de implementación que se reduce a tres aspectos:

1. Una clase nueva, `SafeFD`, en una cabecera que comparten los dos programas.
2. El tipo con el que los descriptores viajan por las funciones que ya existen.
3. La desaparición de las llamadas a `close()` repartidas por el programa.

A un cambio así –que reorganiza el código sin alterar lo que hace– se le llama **refactorización**, y conviene hacerlo por partes, comprobando después de cada una que el programa sigue funcionando.

6.3. Los recursos tienen dueño

6.3.1. El descriptor es un *handle*

Un descriptor de archivo no es el recurso, sino un número que lo identifica ante el núcleo, tal y como se explica en la Sección A.1. A un identificador así se le llama *handle*, un nombre que aparece en todas partes: son *handles* los descriptores de POSIX, los `HANDLE` y `HWND` de Windows, los `pthread_t` de los hilos y los identificadores que devuelven las librerías que gestionan recursos por su cuenta.

Un *handle* no se comporta como una variable cualquiera que contiene un valor. De ahí surge la dificultad de esta unidad, porque, como se ilustra en la Figura 6.1, copiar la variable que contiene el *handle* copia el número, no el recurso. Es decir, `int otro = conn_fd` no abre una segunda conexión, sino que ahora `otro` y `conn_fd` son dos nombres de la misma conexión. Por eso cerrar cualquiera de los dos descriptores deja al otro inservible, y por eso quien recibe una de esas copias no puede saber, mirando el `int`, si le toca cerrarla o si es otra parte del código la que tiene esa responsabilidad.

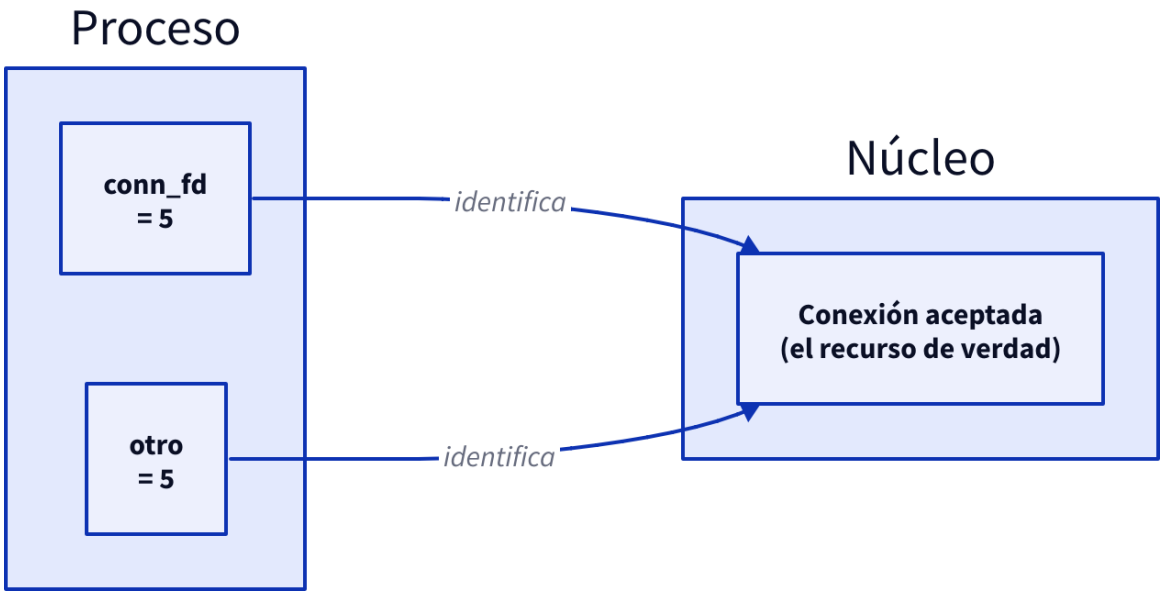


Figura 6.1.: Dos variables con el mismo descriptor no son dos recursos, sino dos nombres del mismo

> ⚠ Advertencia
>
> Además los identificadores se reutilizan. Cuando se cierra un descriptor, el núcleo puede entregar ese mismo número en la siguiente llamada que cree otro. Así que una variable que se quedó con un descriptor cerrado no siempre provoca un error cuando se utiliza, pues es posible que en ese momento identifique un recurso diferente. Este tipo de errores son especialmente difíciles de encontrar, porque el programa no falla donde está la equivocación, sino más tarde y en otro punto.

En POSIX, la operación que sí crea un segundo *handle* para el mismo recurso es `dup2()`, que ya se usó para las redirecciones de la Unidad 2. Y ni siquiera entonces se copia el recurso. Lo que se obtiene son dos identificadores que el núcleo tiene en cuenta, de forma que el recurso se libera cuando se cierra el último de ellos.

Así que lo que hace falta no es prohibir que haya dos o más copias de un *handle*, sino dejar claro **quién es el dueño**: quién tiene la obligación de cerrarlo y, por tanto, quién no debe hacerlo. Cuando se usan `int` –como en los descriptores de archivo– esa información no está en ninguna parte, solo en la cabeza de los programadores.

6.3.2. Lo que `std::jthread` hace

Un `std::jthread` es también un *handle*: el objeto no es el hilo, que vive fuera del programa, sino lo que permite referirse a él. La diferencia es lo que la librería estándar hace para facilitarnos su gestión:

- Su destructor pide la parada del hilo y lo espera, así que desde la Unidad 3 no hemos escrito ni un solo `join()` (Sección G.1), porque es algo que ocurre automáticamente.
- No se puede copiar, solo mover, exactamente por lo que dice la Sección 6.3.1: dos objetos para un mismo hilo significarían dos destructores esperando por él.

Estos mecanismos son los que nos permiten apoyarnos en el ámbito de las variables para ordenar el final del programa, como hacen la Sección 3.7.7 y el `main()` del servidor de la Sección 5.5.2.

Los descriptores no heredaron nada de esto porque vienen de una API de C, en la que un descriptor es un `int` y un `int` no tiene destructor. La clase que vamos a escribir les da a los descriptores lo mismo que `std::jthread` les da a los hilos.

> **Nota**
>
> Esta forma de gestionar los recursos –adquirirlos al construir un objeto y liberarlos en su destructor– se conoce por sus siglas en inglés, **RAII** (*Resource Acquisition Is Initialization*), y es la razón por la que en C++ bien escrito casi nunca aparece una liberación explícita –un `delete`, `free()` o `close()`–, sino que se confía en el destructor de un objeto para que lo haga. Es la misma idea que hay detrás de `std::string` o de `std::vector`, que liberan la memoria que reservan por sí mismos cuando se destruyen.

6.4. La clase `SafeFD`

La clase guarda un descriptor y se encarga de cerrarlo:

```
class SafeFD
{
public:
    SafeFD() noexcept = default;                                    ①

    explicit SafeFD(int fd) noexcept                                ②
        : fd_{fd}
    {}

    SafeFD(const SafeFD&) = delete;                                 ③
    SafeFD& operator=(const SafeFD&) = delete;

    SafeFD(SafeFD&& other) noexcept                                 ④
        : fd_{other.fd_}
    {
        other.fd_ = invalid_fd;
    }
```

```
    SafeFD& operator=(SafeFD&& other) noexcept
    {
        if (this != &other)                                          ⑤
        {
            reset(other.fd_);
            other.fd_ = invalid_fd;
        }
        return *this;
    }

    ~SafeFD() noexcept
    {
        reset();
    }

    void reset(int fd = invalid_fd) noexcept                         ⑥
    {
        if (fd_ != invalid_fd)
        {
            close(fd_);
        }
        fd_ = fd;
    }

    [[nodiscard]] bool is_valid() const noexcept
    {
        return fd_ != invalid_fd;
    }

    [[nodiscard]] int get() const noexcept                           ⑦
    {
        return fd_;
    }

private:
    static constexpr int invalid_fd = -1;                            ⑧
    int fd_ = invalid_fd;
};
```

① Un `SafeFD` construido así no es dueño de nada y su destructor no cierra nada. Es el estado en el que queda un objeto al que le han movido el descriptor, y también el que permite declarar la variable antes de saber cuál va a guardar.

② Construir un `SafeFD` a partir de un `int` significa hacerse cargo de cerrarlo, y `explicit` impide que un `int` se convierta en un `SafeFD` sin que el programador lo haya solicitado.

③ Copiar el objeto duplicaría el *handle* sin duplicar el recurso, así que los dos destructores cerrarían el mismo descriptor: el segundo `close()` fallaría, o peor, cerraría el descriptor

que el núcleo ya le hubiera entregado a otro (Sección 6.3.1). Eliminando el constructor de copia, quien lo intente se encuentra con un error de compilación en lugar de con un programa que falla de vez en cuando.

④ Mover sí tiene sentido, porque es lo que ocurre cuando el descriptor cambia de dueño: el objeto de origen deja de serlo, y por eso hay que dejarlo inválido para que su destructor no cierre nada.

⑤ Sin esta comprobación, una sentencia como `fd = std::move(fd)` cerraría el descriptor y dejaría el objeto inválido, destruyendo un recurso que nadie había cedido a nadie.

⑥ `reset()` cierra el descriptor manualmente, que es lo que a veces hace falta cuando el cierre forma parte del funcionamiento del programa –por ejemplo, el cierre de una conexión– y no solo de la limpieza antes de terminar (Sección 6.4.1).

⑦ Las llamadas al sistema siguen recibiendo un `int`, y esta es la única forma de dárselo. Indicamos `[[nodiscard]]` para que el programador recuerde usar el valor devuelto (Sección B.4).

⑧ El descriptor inválido es `-1`, que es lo que retornan las llamadas al sistema cuando fallan. No vale `0`, que es un descriptor tan válido como cualquier otro –el de la entrada estándar (Sección A.1)–.

La clase entera cabe en una cabecera –`safe_fd.hpp`, por ejemplo– que deben incluir los dos programas. La clase necesita `<unistd.h>` por `close()`.

> **!** Importante
>
> La clase no obliga a nadie a ser cuidadoso. Guardar en un `int` aparte el descriptor que retorna `get()` devuelve al programa todos los problemas comentados al principio, y usar ese `int` para cerrarlo a mano le añade uno nuevo, porque el destructor cerrará más tarde un descriptor que ya no le pertenece. La regla es que el descriptor solo salga del objeto para dárselo a una llamada al sistema y siempre en la misma línea de código.

6.4.1. Cerrar antes de tiempo

Casi siempre basta con que el descriptor se cierre al terminar el ámbito en el que se declaró. De eso se encarga el destructor de `SafeFD`. La excepción está en `copy_file_compressed()`, donde cerrar un extremo de la tubería es la forma de decirle al compresor que debe terminar (Sección 2.4.5.1):

- El hijo cierra el extremo que no va a usar antes del `execv()`. Mientras quede algún proceso con ese extremo abierto, la tubería no se ve cerrada por el otro lado.
- El padre cierra el extremo de escritura en cuanto termina de enviar el contenido, y es eso lo que hace que el comando vea el final de su entrada y termine. Si ese cierre esperase al final de la función, el padre se quedaría en `waitpid()` esperando a un proceso que a su vez espera datos que no van a llegar nunca.

En ambos casos necesitamos llamar a `reset()` para hacer un cierre manual, justo donde antes había un `close()`.

> **Advertencia**
>
> En el hijo, además, los destructores no llegan a ejecutarse nunca: `execv()` sustituye la imagen del proceso, y el `_exit()` de los caminos de error termina sin desenrollar la pila (Sección D.1.1). Así que los descriptores que el hijo no debe heredar –el del archivo de origen, el del destino una vez redirigido y el extremo de la tubería que le sobra– siguen teniendo que cerrarse con `reset()` antes del `execv()`, igual que se cerraban en la Sección 2.3. Olvidarlo no rompe nada a simple vista, pues el programa sigue funcionando y los descriptores de más solo se descubren mirando el `/proc/PID/fd` del compresor.

6.5. Manejo de errores

Se mantienen las pautas de la Sección 5.4, y el cambio de esta unidad las simplifica en dos sitios.

El primero es el de las funciones que crean un descriptor y fallan después. Un error a mitad de `create_inet_socket()` ya no obliga a cerrar nada antes de retornar, porque de eso se encarga el destructor del `SafeFD` local, y eso deshace además la precaución que advertía la Sección B.3.1:

> **Nota**
>
> `system_error_from_errno()` lee `errno`, así que había que llamarla antes de cualquier `close()` y guardar el error en una variable para poder retornarlo después. Con `SafeFD` esa precaución sobra: el destructor se ejecuta cuando el valor de retorno ya está construido, es decir, después de haber leído `errno`.

El segundo son las excepciones. Hasta ahora una excepción que atravesara una función dejaba abiertos sus descriptores, porque nadie los cerraba por el camino. Al deshacer la pila, en cambio, se ejecutan los destructores de las variables locales (Sección B.5.1), así que los descriptores se cierran solos también por ese camino. Lo que no cambia es la regla de la Sección 3.6: de la función principal de un hilo no puede escapar ninguna excepción.

> **Advertencia**
>
> Lo que sí perdemos con este cambio es la información de error de `close()`. Como otras llamadas al sistema, `close()` puede fallar, y su error suele ser indicio de que los datos de un `write()` anterior no llegaron a escribirse de verdad. Lamentablemente, un destructor no puede retornar un valor ni debe dejar escapar una excepción, así que ese posible error se pierde.
> Los programas de esta práctica nunca han comprobado el resultado de `close()` porque el descriptor se cierra aunque la llamada falle, así que realmente no perdemos ninguna funcionalidad. Pero si un programa necesita detectar ese error, tendría que añadir a

> la clase un método que cerrase e informara del resultado, y llamarlo explícitamente antes de que el objeto se destruyera. Lo que nos daría el destructor es asegurar que el descriptor se cierra aunque el programador se olvide de llamar a ese método.

6.6. Implementación

El flujo de los dos programas es el de la Sección 5.5. Lo que cambia es el tipo con el que los descriptores viajan por él, que depende únicamente del papel que tenga cada función respecto al descriptor que maneja: crearlo, usarlo o quedárselo. La única que cambia algo más es `wait_for_input()`, y en la Sección 6.6.2 se ve por qué.

6.6.1. Una función que crea un descriptor

Es el caso de `create_unix_socket()`, `create_inet_socket()` (Sección 5.5.4) y `connect_to_server()` (Sección 5.5.5). Todas retornan el descriptor que acaban de crear, así que todas entregan con el valor de retorno la responsabilidad de cerrarlo:

```
std::expected<SafeFD, std::system_error> create_inet_socket(
    const std::string& address, const std::string& port);
```

Internamente el objeto `SafeFD` se crea y se mueve al retornarlo, transfiriendo la obligación de cerrarlo a la función que lo llamó:

```
SafeFD listen_fd{socket(AF_INET, SOCK_STREAM, 0)};                    ①
if (!listen_fd.is_valid())
{
    return system_error_from_errno("Error al crear el socket");
}

int reuse = 1;
if (setsockopt(listen_fd.get(), SOL_SOCKET, SO_REUSEADDR,
        &reuse, sizeof(reuse)) == -1)
{
    return system_error_from_errno(                                   ②
        "Error al activar SO_REUSEADDR");
}

// bind(), listen()...

return listen_fd;                                                     ③
```

① Entre la llamada al sistema y el objeto que se hace cargo del descriptor no queda ni una línea en la que el programa pueda salir por otro sitio. Que `socket()` haya fallado se consulta después en el objeto, porque `is_valid()` comprueba exactamente lo mismo que se comprobaba antes en el `int`.

② Aquí está la diferencia con la versión de la Sección 5.5.4. Aquella tenía que cerrar el descriptor antes de cada uno de estos `return`, y ahora lo cierra el destructor de `listen_fd` al salir de la función.

③ El descriptor sale de la función sin cerrarse, porque el objeto se **mueve** al `std::expected` que se retorna. Al ser `listen_fd` una variable local que se está retornando, el compilador lo hace sin que haya que escribir `std::move()`.

6.6.2. Una función que solo usa el descriptor

Es el caso de `send_line()`, `recv_line()` (Sección 4.5.5), `send_file()`, `recv_file()` y `recv_file_compressed()` (Sección 5.5.7). Ninguna de ellas creó la conexión ni va a cerrarla: se la prestan para escribir o leer por ella, así que la reciben por referencia constante:

```
std::expected<void, std::system_error> send_line(const SafeFD& conn_fd,
    const std::string& message);
```

Dentro, la única diferencia es de dónde sale el `int` que espera la llamada al sistema:

```
ssize_t sent = send(conn_fd.get(), data + total, size - total, 0);
```

Aquí lo importante es lo que **no** se puede escribir. Si la función recibiera el `SafeFD` por valor, significaría que se ha quedado con la conexión, así que su destructor la cerraría al retornar y quien la llamó seguiría intentando enviar por un descriptor cerrado. Como la clase no se puede copiar, pasar `SafeFD` por valor no llega a compilar, y el error se detecta en ese momento y no en mitad de una transferencia.

> **Nota**
>
> Que la referencia sea constante no dice nada sobre si la función lee o escribe por el *socket*. Enviar datos cambia el estado del recurso, que está en el núcleo, pero no cambia el objeto, que solo guarda su identificador. `const` significa aquí que la función no toca la propiedad del descriptor.

La `wait_for_input()` de la Sección 5.5.6 es del mismo tipo –mira descriptores que son de otros– pero tiene que decir cuál de ellos está listo, y ahí hay que elegir con cuidado qué queremos retornar:

```
std::expected<std::optional<std::size_t>, std::system_error>
    wait_for_input(std::span<const SafeFD> fds,
        std::stop_token stop_token);
```

Lo que haremos es retornar el **índice** del descriptor que está listo, en lugar del descriptor mismo. Porque retornar un `SafeFD` sería ceder una propiedad que la función nunca tuvo, y retornar el `int` del descriptor volvería a sacar el descriptor del envoltorio que lo protege. Retornando el índice sabemos cuál de los descriptores que le dieron es, sin ninguno de los problemas anteriores.

La función se llama así:

```
std::array<SafeFD, 2> listen_fds{std::move(unix_fd),
    std::move(inet_fd)};                                                    ①

auto ready = wait_for_input(listen_fds, stop_token);                        ②
```

① Después de moverlos, los objetos `unix_fd` e `inet_fd` ya no son dueños de nada y el array es quien cierra los dos *sockets* al destruirse.
② En `recv_line()`, donde solo hay un descriptor, sigue valiendo lo que explicaba la Sección 5.5.6: `{&conn_fd, 1}`, sin construir ningún contenedor.

6.6.3. Una función que se queda con el descriptor

Este es el caso de `connection_worker()` (Sección 5.5.9). El hilo que acepta conexiones no vuelve a saber nada de la conexión que acaba de aceptar, y quien la cierra al terminar es el hilo que la atiende, así que la conexión tiene que cambiar de dueño:

```
void connection_worker(SafeFD conn_fd, const std::string& client_name,
    std::size_t number, const Options& options,
    const std::string& compression_path,
    CopyProgress& progress, std::atomic<bool>& active,
    std::stop_token stop_token);
```

Por valor, y no por referencia, porque es justamente lo que se quiere decir: la función se hace cargo del descriptor y su destructor lo cerrará, que es lo que sustituye al `close(conn_fd)` del final del original.

Quien la llama tiene que cederlo explícitamente con `std::move()`:

```
SafeFD conn_fd{accept(listen_fds[index].get(),                              ①
    reinterpret_cast<sockaddr*>(&client_addr), &addr_len)};

// ...

connection.thread = std::jthread(connection_worker,                         ②
    std::move(conn_fd), client_name, number, std::cref(options),
    std::cref(compression_path), std::ref(connection.progress),
    std::ref(connection.active), stop_token);
```

① `index` es lo que retornó `wait_for_input()`, ya sacado del `std::expected` y del `std::optional` (Sección 6.6.2). El descriptor de la conexión entra en un `SafeFD` en el mismo sitio en que aparece, como en la Sección 6.6.1.

② Los argumentos de un `std::jthread` se copian antes de pasárselos al hilo (Sección G.1), y un `SafeFD` no se puede copiar. `std::move()` es lo que permite construirlo, y a la vez lo que deja constancia en el código de que a partir de esta línea la conexión es del hilo nuevo.

> **Importante**
>
> Después de `std::move()` la variable de origen sigue existiendo, pero ya no es dueña de nada: su descriptor es `invalid_fd`. Usarla a partir de ahí no es un error de compilación, así que es el único caso en el que el compilador no nos cubre las espaldas. Lo que sí ocurre es que el fallo es evidente en cuanto se prueba, porque la llamada al sistema recibe un `-1` como descriptor y falla siempre, en lugar de funcionar a veces.

6.6.4. El resto del programa

Con esas tres reglas, lo que queda es mecánico:

- En `copy_file()` (Sección 1.4.5) y en `copy_file_compressed()` (Sección 2.4.5) los descriptores son variables locales, así que desaparecen todos sus `close()`, incluidos los del camino de la cancelación (Sección 3.4.2). Los dos cierres de la tubería que no son limpieza son la excepción que explica la Sección 6.4.1.
- En `accept_worker()` (Sección 5.5.8) el parámetro pasa a ser un `std::span<const SafeFD>`.
- En los dos `main()` desaparecen los `close()` de la limpieza final, y con ellos el cuidado de no olvidarse de ninguno en los caminos de error.

El orden en que se destruye todo importa, y es el que ya organizaban los ámbitos de la Sección 5.5.2. El `std::array` con los *sockets* que escuchan tiene que declararse **antes** del ámbito del vector de conexiones, para que se destruya después de que los hilos hayan terminado. Al revés, el servidor cerraría los *sockets* mientras alguien podría seguir usándolos.

> **Nota**
>
> La limpieza que no desaparece es la del sistema de archivos. El archivo del *socket* de dominio UNIX y el del PID siguen borrándose con `unlink()` al final del `main()`, porque cerrar un *socket* no elimina su archivo (Sección I.7). Son otro recurso con dueño, aunque su identificador sea una ruta en lugar de un número, por lo que, si quisiéramos, se podría aplicar la misma lógica de gestión de recursos.

6.7. Comprobación

La primera comprobación es la de siempre: repetir enteras las pruebas de la Sección 5.6 y obtener exactamente los mismos resultados.

Lo propio de esta unidad es comprobar que los descriptores se cierran de verdad, y para eso el sistema los enseña. Cada proceso tiene en `/proc/PID/fd` un enlace por cada descriptor que tiene abierto:

```
$ backup-server /tmp/copias &
[1] 4123
$ ls /proc/4123/fd
0  1  2  3  4
```

Son la entrada, la salida y el error estándar, más los dos *sockets* que escuchan. Después de atender unas cuantas conexiones, tienen que seguir siendo esos cinco:

```
$ for i in 1 2 3 4 5; do backup /tmp/texto.txt; done
$ ls /proc/4123/fd
0  1  2  3  4
```

Si la lista crece con cada cliente, hay algún descriptor de conexión que no se está cerrando.

6.7.1. Casos de prueba adicionales

Adicionalmente, es importante probar los siguientes escenarios:

- **Cancelar una copia con compresión** con `Ctrl-C`, que es donde el cierre del extremo de escritura de la tubería deja de ser limpieza (Sección 6.4.1). Si se ha dejado que lo cierre el destructor, el programa se queda colgado en `waitpid()` en lugar de terminar.
- **Terminar el servidor mientras atiende conexiones**, comprobando que sigue sin dejar copias a medias y que el directorio de ejecución queda limpio.
- **Arrancar el servidor con el puerto ocupado**, que es el camino en el que el primer *socket* se crea y el segundo falla, y por el que ya no pasa ningún `close()` escrito a mano. Debe seguir informando del error y no arrancar, sin afectar al servidor que ya estaba en marcha.
- **Destruir un `SafeFD` que nunca llegó a recibir un descriptor**, que no debe cerrar nada. Con `strace -e trace=close` no tiene que aparecer ninguna llamada con el argumento `-1`.
- **Matar al cliente a mitad de una transferencia**, comprobando que el hueco de esa conexión vuelve a quedar libre y su descriptor cerrado.
- **Recorrer el código buscando `close()`**, que no debe aparecer en ningún sitio que no sea el `reset()` de la clase.

A. Operaciones con archivos con la API POSIX

Los archivos son uno de los mecanismos fundamentales para el almacenamiento persistente de información en los sistemas operativos. La API POSIX define un conjunto de funciones de bajo nivel que permiten a los programas interactuar con el sistema de archivos de manera estandarizada: abrir, leer, escribir y cerrar archivos, así como consultar y modificar sus atributos. En esta sección exploraremos estas funciones básicas y aprenderemos cómo los procesos solicitan recursos mediante descriptores de archivo, cómo gestionar los errores y cómo desarrollar programas robustos.

A.1. Descriptores de archivo

Antes de poder realizar cualquier operación sobre un archivo –como leer su contenido o escribir datos en él–, es necesario comprender el mecanismo mediante el cual los procesos identifican y acceden a los recursos del sistema operativo.

A.1.1. Identificación de recursos

Los procesos necesitan solicitar recursos al sistema operativo para realizar su trabajo: acceder a archivos, comunicarse con otros procesos, utilizar dispositivos de hardware, etc. Sin embargo, por razones de seguridad y gestión eficiente, **los procesos no pueden acceder directamente a la mayoría de estos recursos**.

En su lugar, el sistema operativo actúa como intermediario:

1. Cuando un proceso solicita un recurso (por ejemplo, abrir un archivo), el sistema operativo reserva y prepara ese recurso.
2. En lugar de dar acceso directo al recurso, el sistema operativo devuelve al proceso un **identificador** único que representa ese recurso.
3. Cada vez que el proceso quiere realizar una operación sobre el recurso (leer, escribir, cerrar, etc.), debe proporcionar ese identificador al sistema operativo para indicar sobre qué recurso desea actuar.
4. El sistema operativo verifica que el proceso tiene permisos para realizar la operación y la ejecuta en su nombre.

5. Cuando el proceso termina de usar el recurso, debe notificarlo al sistema operativo –generalmente cerrando el recurso mediante su identificador– para que pueda liberarlo.

> **Nota**
>
> Este modelo sería similar a si en programación orientada a objetos, en lugar de llamar directamente a los métodos de un objeto, tuviéramos que pedirle a un **intermediario** que lo haga por nosotros. En ese caso, el intermediario necesitaría saber qué objeto queremos usar (el identificador) y qué método queremos invocar (la operación).

A.1.2. Descriptores de archivo en POSIX

En los sistemas POSIX –Linux, macOS, BSD, etc.– estos identificadores se llaman **descriptores de archivo** y son números enteros no negativos (`int`): `0`, `1`, `2`, `3`, `4`, etc.

Por ejemplo:

```
int fd = open("archivo.txt", O_RDONLY);                    ①
read(fd, buffer, tamaño);
close(fd);                                                  ②
```

① Obtener un descriptor para identificar el archivo abierto (`fd`).
② Cerrar el descriptor para liberar el recurso.

Hay que tener en cuenta que copiar un descriptor no duplica el recurso al que identifica. Por eso debemos tener cuidado al copiar las variables que los contienen y al cerrar descriptores que se usan en otros lugares del programa.

A.1.3. Descriptores estándar

Todo proceso se inicia con tres descriptores que están siempre disponibles y no es necesario abrirlos explícitamente:

`0` **(STDIN_FILENO)** Entrada estándar (*standard input*). Por defecto, conectado al teclado. Se puede leer de él usando `read()`.
`1` **(STDOUT_FILENO)** Salida estándar (*standard output*). Por defecto, conectado a la terminal. Se puede escribir en él usando `write()`.
`2` **(STDERR_FILENO)** Salida de error estándar (*standard error*). Por defecto, también conectado a la terminal. Se usa para escribir mensajes de error y diagnóstico.

> **Nota**
>
> En C y C++, las macros `STDIN_FILENO`, `STDOUT_FILENO` y `STDERR_FILENO` están definidas en el archivo de cabecera `<unistd.h>`.

Estos descriptores permiten al proceso interactuar con el entorno (terminal, consola, etc.) y se pueden usar como cualquier otro descriptor. Por ejemplo:

```
int main()
{
    const char* mensaje = "Hola, mundo!\n";
    write(STDOUT_FILENO, mensaje, strlen(mensaje));
    return 0;
}
```

A.1.4. Descriptores y otros recursos del sistema

Aunque los ejemplos anteriores muestran operaciones con archivos, el mismo concepto de identificadores se aplica a otros recursos del sistema operativo, como:

- **Tuberías** (*pipes*): Creadas con `pipe()`.
- **Sockets**: Creados con `socket()`.
- **Dispositivos**: Archivos especiales en `/dev/` que representan hardware.

Todos estos recursos se manipulan usando descriptores de archivo y las mismas funciones como `read()`, `write()` y `close()`, aunque el sistema operativo también ofrece otras funciones para realizar operaciones específicas según el tipo de recurso.

A.2. Gestión de errores en la API POSIX

Las funciones del sistema operativo pueden fallar por múltiples razones: permisos insuficientes, archivos inexistentes, falta de espacio en disco, interrupciones del sistema o memoria insuficiente, entre otras. Por ello, el manejo correcto de errores es fundamental para desarrollar programas robustos y fiables.

A.2.1. Convención de valores de retorno

Muchas funciones de las API de los sistemas operativos siguen convenciones similares para indicar éxito o error:

1. Terminan devolviendo un valor específico para indicar que ocurrió un error, como: `false`, `NULL`, `-1`, etc.
2. Proveen un mecanismo para obtener información detallada sobre el error ocurrido, como una variable global o una función que devuelve un mensaje descriptivo. En los sistemas POSIX ese mecanismo es la variable `errno`, cuyo valor es un código numérico que identifica el tipo de error.

Entre las funciones de más bajo nivel de la API POSIX, la convención más común es que **devuelvan `-1` para indicar un error**. Por ejemplo:

```
// Ejemplo de manejo de error al abrir un archivo
int fd = open("archivo.txt", O_RDONLY);
if (fd == -1)
{
    // Ha ocurrido un error al abrir el archivo
    std::println(stderr, "Error ({}) al abrir el archivo: {}",
        errno,                                                    ①
        std::strerror(errno)                                      ②
    );
    return /* ... */;
}
```

① `errno` contiene el código numérico del error ocurrido.
② `strerror(errno)` devuelve un mensaje descriptivo del error.

> **!** Importante
>
> **Es fundamental verificar el valor de retorno de todas las llamadas al sistema.** No comprobar errores es una de las causas más comunes de bugs difíciles de diagnosticar y de vulnerabilidades de seguridad.

A.2.2. Valores de `errno`

`errno` se define en el archivo de cabecera `<errno.h>` (o `<cerrno>` en C++) y contiene un valor de tipo `int`. Los valores posibles de `errno` son constantes definidas en `<errno.h>`, como:

`ENOENT` (*No such file or directory*) El archivo o directorio especificado no existe.
`EACCES` (*Permission denied*) Permisos insuficientes para realizar la operación.
`EEXIST` (*File exists*) El archivo ya existe (usado con `O_CREAT | O_EXCL`).
`EISDIR` (*Is a directory*) Se intentó abrir un directorio con `O_WRONLY` o `O_RDWR`.
`ENOSPC` (*No space left on device*) No hay espacio disponible en el dispositivo.
`EMFILE` (*Too many open files*) El proceso ha alcanzado su límite de descriptores abiertos.
`EINTR` (*Interrupted system call*) La llamada fue interrumpida por una señal.

Cada función –como `open()` o `read()`– documenta en su página del manual los posibles códigos de error que puede generar.

Por tanto, el patrón estándar para verificar errores en llamadas al sistema POSIX es:

```
ssize_t result = read(...);
if (result == -1)   // Verificar si hubo error
{
    // En este punto, errno contiene el código de error

    if (errno == ENOENT)
    {
        // Manejar error de archivo no encontrado
    }
    else if (errno == EACCES)
    {
        // Manejar error de permisos insuficientes
    }
    // ... otros errores específicos

    // Acción genérica en caso de error
    // ...
    return /* ... */;
}

// Continuar con el código normal si no hubo error
```

A.2.3. Lectura inmediata de `errno`

Un detalle importante es que **resulta crucial leer el valor de `errno` inmediatamente después de detectar un error**, porque:

1. **`errno` solo tiene un valor significativo justo después de que una función falle.** Nadie lo pone a cero cuando una función tiene éxito, por lo que en cualquier otro momento puede contener el valor de un error anterior.
2. **Cualquier llamada posterior a funciones del sistema puede modificar `errno`**, incluso si esas funciones tienen éxito.

Por ejemplo, este código es **incorrecto**:

```
int fd = open(filename, O_RDONLY);
if (fd == -1)
{
    // Hacer otras operaciones...
    std::println("Archivo: {}", filename);                    ①

    // Para cuando llegamos aquí, errno puede haber cambiado
    if (errno == ENOENT)                                      ②
    {
```

```
        std::println(stderr, "El archivo no existe");
    }
}
```

① La llamada a `std::println` puede llamar internamente a funciones del sistema que modifiquen `errno`.
② Por tanto, al llegar a esta línea, `errno` puede no contener el código de error original de `open()`.

El código correcto debe guardar `errno` inmediatamente:

```
int fd = open(filename, O_RDONLY);
if (fd == -1)
{
    int error_code = errno;                                          ①

    // Ahora podemos hacer otras operaciones de forma segura
    std::println("Error al abrir archivo: {}", filename);

    if (error_code == ENOENT)                                        ②
    {
        std::println(stderr, "El archivo no existe");
    }
}
```

① Guardar el valor de `errno` en una variable local inmediatamente después de detectar el error.
② Usar la copia guardada para verificar el tipo de error.

A.3. Abrir archivos

La función `open()` abre el archivo cuya ruta se pasa como primer argumento (`pathname`) y devuelve, en caso de éxito, un descriptor de archivo (ver la Sección A.1) que puede ser usado posteriormente para leer, escribir o realizar otras operaciones sobre el archivo abierto.

```
int open(const char *pathname, int flags);
int open(const char *pathname, int flags, mode_t mode);
```

Si la llamada falla, `open()` retorna `-1` y `errno` contiene el código de error (ver la Sección A.2).

A.3.1. Opciones de apertura

El segundo argumento (`flags`) es un conjunto de opciones que indican cómo se va a abrir el archivo. Algunas de estas opciones son:

- `O_RDONLY` abre el archivo para lectura.
- `O_WRONLY` abre el archivo para escritura.
- `O_RDWR` abre el archivo para lectura y escritura.
- `O_TRUNC` trunca el archivo a 0 bytes al abrirlo.
- `O_APPEND` abre el archivo para escritura y coloca el puntero de escritura al final del archivo. Esto es útil cuando se quieren añadir datos al final del archivo.
- `O_CREAT` crea el archivo si no existe. Si no se indica esta opción, y el archivo no existe, la función `open()` fallará con `ENOENT`.
- `O_EXCL`, en combinación con `O_CREAT`, hace que la llamada a `open()` falle con `EEXIST` si el archivo ya existe.

Se pueden combinar varias de estas opciones usando el operador `|` –*or* bit a bit–:

```
int fd = open("ruta/del/archivo", O_RDONLY | O_CREAT, 0644);
if (fd == -1)
{
    // Error al abrir el archivo...
    return /* ... */;
}
```

A.3.2. Permisos de los nuevos archivos

El tercer argumento (`mode`) indica los permisos del nuevo archivo, si es que va a ser creado porque no existía previamente. Por tanto, esta opción solo tiene sentido si se pasa la opción `O_CREAT` en `flags` porque, de lo contrario, el archivo abierto ya existe y conserva sus permisos originales.

Si se indica `O_CREAT` y no se pasa `mode`, el comportamiento es indefinido: el archivo se crea con los permisos que resulten de leer un argumento no definido, por lo que puede acabar con cualquier conjunto de permisos. Así que lo adecuado es indicar siempre los permisos que se quiere que tenga el archivo, para lo que se puede usar la misma notación octal que se utiliza con el comando `chmod` en la *shell*:

```
int fd = open("ruta/del/archivo", O_RDONLY | O_CREAT, 0666);
if (fd == -1)
{
    // Error al abrir el archivo...
    return /* ... */;
}
```

En el ejemplo anterior, el nuevo archivo hereda del proceso que llama a `open()` el propietario y el grupo. Respecto a los permisos, el deseo del programador es que el archivo tenga permisos de lectura y escritura para el propietario, para el grupo y para otros.

A.3.3. Permisos y máscara umask

Los permisos indicados en el argumento `mode` de `open()` son modificados por la máscara **umask** del proceso antes de ser aplicados al archivo, de tal forma que los permisos reales del archivo serán `mode & ~umask`.

La **umask** puede ser configurada por cada usuario mediante el comando `umask` de la *shell* y se hereda por todos los procesos lanzados por ese usuario, lo que permite a los usuarios controlar los permisos que tendrán los archivos que creen los programas que ejecuten. Además, cada proceso puede cambiar su propia **umask** mediante la función `umask()`.

Por ejemplo, para establecer la **umask** a `0022` durante una sesión, se puede usar el siguiente comando en la *shell*:

```
$ umask 0022
```

El comando `umask` también se puede usar para comprobar el valor actual de la máscara:

```
$ umask
0022
```

Si un programa crea un nuevo archivo con `open(..., 0666)`, pero la `umask` heredada por el proceso desde la *shell* del usuario es `0022`, el archivo tendrá permisos `0666 & ~0022 = 0644`. Es decir, con esa máscara, por defecto, los nuevos archivos no tendrán permisos de escritura ni para el grupo ni para otros.

> **Nota**
>
> No suele tener sentido indicar permisos de ejecución en `mode` –por ejemplo, `0777`– a menos que se esté desarrollando un compilador y se sepa que el archivo que se va a crear va a ser ejecutable.

A.4. Cerrar descriptores de archivo

Cuando un archivo ya no es necesario, es importante cerrarlo para liberar los recursos del sistema asociados a ese archivo. Los descriptores de archivo se cierran usando la función `close()`.

```
int close(int fd);
```

El motivo de error más común al llamar a `close()` es intentar cerrar un descriptor inválido o que ya ha sido cerrado.

A.5. Leer datos de archivos

La función `read()` permite leer datos de un archivo abierto y avanzar el puntero de lectura.

```
ssize_t read(int fd, void* buf, size_t count);
```

`fd` Descriptor del archivo abierto del que se quieren leer datos. Este descriptor se puede haber obtenido previamente con `open()`, `pipe()`, `dup()` o cualquier otra función que genere un descriptor de archivo.

`buf` Puntero a la región de memoria donde se almacenarán los datos leídos. Es responsabilidad del programador reservar la región de memoria y asegurarse de que tiene suficiente espacio para almacenar los datos.

`count` Número máximo de bytes que se desean leer del archivo. La función intentará leer como máximo hasta este número de bytes, pero puede leer menos.

Si la llamada tiene éxito, `read()` retorna el número de bytes realmente leídos, que puede ser:

- **Mayor que 0**, si se han leído datos correctamente. El número retornado indica cuántos bytes se han almacenado en `buf`.
- **Igual a 0**, si se ha alcanzado el final del archivo (*end-of-file*), por lo que no hay más datos que leer.

Si la llamada falla, `read()` retorna `-1` y `errno` contiene el código de error (ver la Sección A.2).

A.5.1. Consideraciones importantes

La función `read()` no garantiza leer todos los bytes solicitados, incluso cuando el archivo contiene suficientes datos. El número de bytes leídos puede ser menor que `count` por diversos motivos:

- Se ha alcanzado el final del archivo antes de leer `count` bytes.
- La lectura fue interrumpida por una señal.
- En el caso de descriptores de archivos de *sockets* o tuberías, puede haber menos datos disponibles de los solicitados.

Por tanto, **es frecuente tener que llamar a `read()` múltiples veces en un bucle** hasta que se hayan leído todos los datos necesarios o se alcance el final del archivo.

Por ejemplo:

```
void read_data(int fd, std::vector<uint8_t>& buffer, size_t bytes_needed)
{
    ssize_t bytes_read;
    size_t total_read = 0;

    buffer.resize(bytes_needed);

    while (total_read < bytes_needed)
    {
        bytes_read = read(fd, buffer.data() + total_read,
          bytes_needed - total_read);

        if (bytes_read == -1)
        {
            // Error al leer...
            break;
        }
        else if (bytes_read == 0)
        {
            // Final del archivo alcanzado
            break;
        }

        total_read += static_cast<size_t>(bytes_read);                    ①
    }

    buffer.resize(total_read);                                            ②
}
```

① Acumular el número total de bytes leídos hasta ahora. Se hace un *cast* a `size_t` porque `bytes_read` es de tipo `ssize_t`, que es un tipo con signo, mientras que `total_read` es de tipo `size_t`, que es sin signo.
② Ajustar el tamaño del búfer al número real de bytes leídos.

A.6. Escribir datos en archivos

La función `write()` permite escribir datos en un archivo abierto y avanzar el puntero de escritura.

```
ssize_t write(int fd, const void* buf, size_t count);
```

Sus argumentos son los mismos que los de `read()`, pero en sentido contrario: `fd` es el descriptor del archivo abierto en el que se quieren escribir los datos –que debe tener permisos de escritura–, `buf` apunta a la región de memoria que los contiene y `count` es el número de bytes que se quieren escribir desde `buf`. Los datos se escriben a partir de la posición actual del puntero de escritura del descriptor.

Si la llamada tiene éxito, `write()` retorna el número de bytes realmente escritos; si falla, retorna `-1` y `errno` contiene el código de error (ver la Sección A.2).

Hay varios motivos por los que la función `write()` puede escribir menos bytes de los que se le han indicado, cuando se usa para escribir en un *socket* o en una tubería. Con archivos regulares no suele ocurrir. Cuando ocurre, normalmente se debe a que no queda espacio en el dispositivo.

Lo correcto es comprobar el valor de retorno de `write()` y si es menor que el tamaño del búfer indicado, volver a llamar a `write()` con el resto de bytes que quedan por escribir. Si el problema se repite, entonces `write()` retornará `-1` y el valor de `errno` indicará el error –por ejemplo, `ENOSPC` si el espacio en el dispositivo no es suficiente–.

A.7. Acceder a los atributos de un archivo

Las funciones `stat()`, `lstat()` y `fstat()` ofrecen una manera de comprobar la existencia de un archivo y de obtener acceso a sus atributos para obtener información como:

- Tipo de archivo.
- Tamaño y número de bloques ocupados.
- Número de inodo.
- Propietario y grupo.
- Permisos.
- Fechas de acceso y modificación.

```
int stat(const char* file_name, struct stat* buf);
int fstat(int filedes, struct stat* buf);
int lstat(const char* file_name, struct stat* buf);
```

Si `stat()` falla con el error `ENOENT`, es que el archivo por el que se pregunta no existe.

Todas las versiones de `stat()` reciben un puntero a una estructura `stat`, que se rellena con la información del archivo al volver de la llamada. La única diferencia entre `stat()` y `fstat()` es que `stat()` recibe la ruta del archivo y `fstat()` recibe el descriptor de archivo que se debe haber abierto previamente con `open()`. Mientras que `lstat()` es igual que `stat()`, pero si la ruta apunta a un enlace simbólico, no sigue el enlace y devuelve información sobre el propio enlace.

La estructura `stat` contiene información sobre el archivo, como el número de inodo, los permisos, el tamaño, el número de enlaces, el propietario, el grupo o las fechas de acceso y modificación:

```
struct stat {
    dev_t         st_dev;      /* dispositivo */
    ino_t         st_ino;      /* inodo */
    mode_t        st_mode;     /* protección y tipo */
    nlink_t       st_nlink;    /* número de enlaces físicos */
    uid_t         st_uid;      /* ID del usuario propietario */
    gid_t         st_gid;      /* ID del grupo propietario */
    dev_t         st_rdev;     /* tipo dispositivo (si es
                                  dispositivo inodo) */
    off_t         st_size;     /* tamaño total, en bytes */
    blksize_t     st_blksize;  /* tamaño de bloque para el
                                  sistema de ficheros de E/S */
    blkcnt_t      st_blocks;   /* número de bloques asignados */
    time_t        st_atime;    /* hora último acceso */
    time_t        st_mtime;    /* hora última modificación */
    time_t        st_ctime;    /* hora último cambio */
};
```

A.7.1. Comprobar si dos archivos son el mismo

Cada archivo tiene un número de inodo único que lo identifica en el sistema de archivos. Un mismo archivo puede ser accesible por medio de varias rutas, por lo que se pueden comparar los números de inodos de archivos en rutas diferentes para determinar si son el mismo archivo.

El inconveniente es que el número de inodo solo es único dentro del mismo sistema de archivos, es decir, que dos archivos pueden tener el mismo número de inodo en sistemas de archivos diferentes. Por eso se necesita comparar tanto `st_dev` –que identifica el dispositivo donde se encuentra el archivo– como `st_ino` –que identifica el archivo dentro del dispositivo– de la estructura `stat`, para saber si dos rutas conducen al mismo archivo.

A.7.2. Acceso a los permisos y al tipo de archivo

Tanto los permisos como el tipo del archivo se almacenan en el campo `st_mode` de la estructura `stat`. Para comprobar si un archivo es de un tipo en particular, se puede utilizar alguna de las siguientes macros:

- `S_ISLNK(m)`: ¿Es un enlace simbólico? `stat()` sigue estos enlaces, por lo que nunca devuelve `true`, pero `lstat()` sí puede hacerlo.
- `S_ISREG(m)`: ¿Es un archivo regular?
- `S_ISDIR(m)`: ¿Es un directorio?

- `S_ISCHR(m)`: ¿Es un dispositivo de caracteres?
- `S_ISBLK(m)`: ¿Es un dispositivo de bloques?
- `S_ISFIFO(m)`: ¿Es una tubería con nombre (FIFO)?
- `S_ISSOCK(m)`: ¿Es un *socket* de dominio UNIX con nombre?

Por ejemplo, para comprobar si una ruta es un directorio:

```
struct stat st;
if (stat(filepath, &st) == -1)
{
    // Error al obtener los atributos del archivo...
}
else
{
    if (S_ISDIR(st.st_mode))
    {
        // 'filepath' es un directorio
    }
    else
    {
        // 'filepath' no es un directorio
    }
}
```

La macro `S_IFMT` contiene una máscara con todos los bits que sirven para indicar el tipo de archivo a 1. Es decir, que se puede usar `S_IFMT` para extraer por separado los bits de tipo y los permisos del archivo:

```
mode_t file_type = st.st_mode & S_IFMT;
mode_t file_permissions = st.st_mode & ~S_IFMT;
```

A.7.3. Comprobar el acceso a un archivo

Cuando solo hace falta saber si un archivo existe o si se puede leer, escribir o ejecutar, sin consultar el resto de sus atributos, se puede usar `access()`:

```
int access(const char *pathname, int mode);
```

En `mode` se indica qué se quiere comprobar, combinando con el operador `|` una o varias de las siguientes constantes:

- `F_OK`: ¿Existe el archivo?
- `R_OK`: ¿Se puede leer?
- `W_OK`: ¿Se puede escribir?

- `X_OK`: ¿Se puede ejecutar? O, si es un directorio, ¿se puede acceder a su contenido?

Retorna `0` si el archivo existe y se tienen todos los permisos indicados, o `-1` estableciendo `errno` en caso contrario. Por ejemplo con `ENOENT` si el archivo no existe o `EACCES` si existe pero no se tienen los permisos.

Es decir, así podemos comprobar si un comando existe y se puede ejecutar:

```
if (access("/usr/bin/ls", X_OK) == 0)
{
    // El comando existe y se puede ejecutar...
}
```

> **Advertencia**
>
> Comprobar y actuar después son dos operaciones distintas, por lo que entre la llamada a `access()` y el momento en que se abre o se ejecuta el archivo, este puede haber sido borrado, sustituido o haber recibido otros permisos. Por eso `access()` no sirve para decidir si una operación va a funcionar –para eso se intenta la operación y se comprueba su error–, sino para poder avisar de que algo falta antes de empezar.

A.8. Eliminar un archivo

La llamada al sistema `unlink()` elimina un nombre del sistema de archivos:

```
int unlink(const char *pathname);
```

Retorna `0` si tiene éxito, o `-1` estableciendo `errno` en caso contrario. Por ejemplo, con `ENOENT` si el archivo no existe o `EACCES` si no se tienen permisos de escritura en el directorio que lo contiene. Para eliminar directorios se utiliza `rmdir()`, porque `unlink()` falla con el código de error `EISDIR` si se le indica uno.

Lo que se elimina realmente es el nombre del archivo en el directorio, no necesariamente el archivo. El contenido del archivo se libera cuando no queda ningún nombre que lo referencie **y** ningún proceso lo tenga abierto:

```
int fd = open("/tmp/copia.dat", O_WRONLY | O_CREAT | O_TRUNC, 0644);
// ...
unlink("/tmp/copia.dat");                                          ①
// ...
close(fd);                                                         ②
```

① El archivo desaparece del directorio inmediatamente, pero se puede seguir leyendo y escribiendo mediante `fd`.

② Aquí es cuando el sistema libera de verdad el espacio que ocupaba.

Es la forma habitual de crear archivos temporales: se crean, se abren y se eliminan inmediatamente, de manera que ningún otro proceso puede acceder a ellos, pero el proceso que los creó puede seguir usándolos hasta cerrarlos.

B. Gestión de errores en C++

La gestión adecuada de errores es fundamental en cualquier aplicación robusta. En C++ moderno, disponemos de varias estrategias para manejar errores, desde el retorno de códigos de error hasta el uso de excepciones, siendo cada una apropiada para diferentes situaciones. En este apéndice vamos a centrarnos en cómo gestionar errores usando `std::expected`, introducida en C++23 y declarada en `<expected>`, en cómo representar los valores que pueden faltar con `std::optional` y en cómo tratar las excepciones que pueden llegar desde la librería estándar.

En la actualidad, se recomienda usar `std::expected` en la mayoría de situaciones de manejo de errores en C++ moderno, dejando las excepciones para los casos verdaderamente excepcionales. Por eso la regla que vamos a seguir en esta práctica para decidir entre uno u otro mecanismo depende de lo que puede hacer el programa a continuación.

Se retorna el error cuando falla la operación que hemos pedido realizar. Que `open()` falle con `ENOENT` porque el archivo no existe, o que `write()` falle con `ENOSPC` porque no queda espacio en el disco, no son situaciones extrañas en un programa que copia archivos. Son situaciones normales, que forman parte de su trabajo. En ambos casos es quien llamó a la función –y no la función– quien tiene el contexto necesario para decidir qué hacer, así que esta le devuelve el error en lugar de tomar la decisión por él.

Se lanza una excepción cuando lo que falla indica que el programa o su entorno están rotos y no tiene sentido seguir adelante. Por ejemplo, quedarse sin memoria, incumplir una precondición o cualquier error de lógica del propio programa.

> **!** Importante
>
> En esta práctica **no vamos a lanzar excepciones**. Todos los errores se retornan usando `std::expected`. Pero sí conviene saber reconocerlas, porque algunas funciones de la librería estándar informan de sus errores lanzándolas, y preparar un lugar donde capturar las que puedan escaparse, tal y como se explica en la Sección B.5.

B.1. Valores que pueden faltar con `std::optional`

No todo lo que puede salir mal es un error. A veces una función simplemente no tiene un valor que retornar, sin que eso signifique que algo haya fallado. O nos sirve con informar del error con la ausencia de valor, sin necesidad de dar más detalles.

Para esos casos está `std::optional<T>`, declarada en `<optional>`, que contiene un valor de tipo `T` o nada en absoluto. Es la forma habitual de envolver las funciones de C que indican la ausencia de valor retornando un puntero nulo, como `getenv()`, que retorna el valor de una variable de entorno o `nullptr` si no está definida:

```
std::optional<std::string> get_environment_variable(
    const std::string& name)
{
    char* value = getenv(name.c_str());
    if (value)
    {
        return std::string(value);                    ①
    }
    else
    {
        return std::nullopt;                          ②
    }
}
```

① El valor se convierte solo en un `std::optional<std::string>` que lo contiene.
② `std::nullopt` es el valor que representa la ausencia de valor.

> **Nota**
>
> En el ejemplo anterior podría parecer más sencillo retornar un `std::string` vacío cuando la variable no está definida, pero entonces se perdería la diferencia entre que la variable no exista y que exista con un valor vacío.

Quien llama a la función comprueba si hay valor evaluando el objeto como una condición –o llamando al método `std::optional::has_value()`– y puede acceder a él con el operador `*`:

```
auto work_dir = get_environment_variable("HOME");
if (!work_dir)
{
    // La variable no está definida...
}

std::println("El directorio de trabajo es {}", *work_dir);    ①
```

① El operador `*` desreferencia el `std::optional` y retorna el valor que contiene, que en este caso es un `std::string` con el valor de la variable de entorno.

> **!** Importante
>
> Acceder al valor de un `std::optional` vacío con el operador `*` es un comportamiento indefinido, igual que desreferenciar un puntero nulo. Hay que comprobar siempre que hay valor antes de usarlo.

También existe el método `value()`, que en ese caso lanza una excepción `std::bad_optional_access` en lugar de provocar un comportamiento indefinido. Y `value_or()`, que retorna el valor que se le indique cuando no hay ninguno en el objeto:

```
std::string dir = get_environment_variable("HOME")
    .value_or("/tmp");                                                    ①
```

① `value_or()` es la forma más compacta de dar un valor por defecto cuando la variable de entorno no está definida.

La diferencia con `std::expected` está en lo que se puede contar sobre la ausencia del valor. Si no hay nada útil que explicar –la variable de entorno no está definida, el comando no aparece en ningún directorio del `PATH`– basta con `std::optional`. Si hay que decir **por qué** ha fallado la operación, para que quien llamó a la función pueda decidir qué hacer o mostrar un mensaje, entonces se usa `std::expected`.

B.2. Propagación de errores con `std::expected`

`std::expected<T, E>` es una clase *template* introducida en C++23 que puede contener un valor de tipo `T` en caso de éxito o un error de tipo `E` en caso de fallo. Es decir, permite que una función retorne un valor exitoso, si tuvo éxito, o un objeto con información del error, si la operación falló. En caso de error, el objeto que contiene la información del error, a su vez, puede ser propagado a las funciones que realizaron la llamada, sucesivamente, hasta que se maneje el error en un nivel superior.

B.2.1. Funciones con valor de retorno

Cuando una función necesita retornar un valor en caso de éxito y un código de error en caso de fallo, `std::expected` es ideal. Por ejemplo, consideremos una función `open_file()` que envuelve la llamada al sistema `open()` y retorna su descriptor:

```
std::expected<int, int> open_file(const std::string& path,                ①
    int flags, mode_t mode = 0666)
{
    int fd = open(path.c_str(), flags, mode);
    if (fd == -1)
    {
```

```
        return std::unexpected(errno);                                    ②
    }

    return fd;                                                            ③
}
```

① El primer parámetro de la plantilla es el tipo del valor que se retorna en caso de éxito –aquí el descriptor– y el segundo, el del error –aquí el código que deja `errno`–. Que ambos sean `int` en este ejemplo es casualidad; no tienen por qué coincidir.
② `std::unexpected` envuelve el valor para marcarlo como error, de forma que no se confunda con el descriptor que retorna la función cuando todo va bien.
③ El descriptor, en cambio, se convierte solo en el `std::expected` que retorna la función.

El uso de esta función es directo y al volver de la llamada se puede verificar fácilmente el resultado:

```
auto result = open_file("datos.txt", O_RDONLY);
if (!result.has_value())                                                  ①
{
    int error_code = result.error();                                      ②
    std::println(stderr, "error ({}) al abrir el archivo: {}",
        error_code, std::strerror(error_code));

    // Manejar error...
    return /* ... */;
}

int fd = result.value();                                                  ③
// Usar el descriptor de archivo...
```

① `has_value()` retorna `true` si la operación fue exitosa y `false` si hubo un error.
② El método `error()` recupera el objeto de error que envuelve el `std::expected`, que aquí es el código de `errno` que dejó `open()`.
③ Y el método `value()`, el valor devuelto en caso de éxito: el descriptor del archivo abierto.

B.2.2. Funciones sin valor de retorno

Para funciones que no necesitan retornar un valor en caso de éxito, podemos usar `std::expected<void, E>`. Esto es especialmente útil para operaciones que solo pueden fallar o tener éxito:

```
std::expected<void, int> write_data(int fd,
    const std::vector<uint8_t>& data)
{
```

```
    size_t total_written = 0;

    while (total_written < data.size())
    {
        ssize_t written = write(fd,
            data.data() + total_written,
            data.size() - total_written);

        if (written == -1)
        {
            return std::unexpected(errno);
        }

        total_written += static_cast<size_t>(written);
    }

    return {};                                                    ①
}
```

① En caso de éxito se retorna un `std::expected` vacío, que es como `std::expected<void, E>` indica que la operación se completó.

Esta función se usaría de manera similar:

```
std::vector<uint8_t> datos = {/* ... */};
auto result = write_data(fd, datos);

if (!result.has_value())
{
    std::println(stderr, "error ({}) al escribir: {}",
        result.error(), std::strerror(result.error()));
    // Manejar error...
    return /* ... */;
}

// Continuar con la siguiente operación...
```

B.3. Información de error enriquecida

En los ejemplos anteriores, usamos `int` como tipo de error para representar códigos de error del sistema (`errno`), pero podemos definir nuestros propios tipos de error para proporcionar información más rica y específica sobre los errores que pueden ocurrir en nuestra aplicación. Por ejemplo, podríamos definir un `enum` para representar diferentes tipos de errores o una

clase que encapsule información adicional sobre el error, como el tipo de error, un mensaje descriptivo, la línea de código donde ocurrió el error, etc.

C++ ya proporciona varias clases de error estándar que también se pueden utilizar como excepciones, como `std::runtime_error`, `std::system_error`, `std::logic_error`, entre otras.

En particular, `std::system_error` es útil para representar errores relacionados con llamadas al sistema y puede encapsular códigos de error del sistema junto con mensajes descriptivos:

```
std::expected<void, std::system_error> write_data(int fd,
    const std::vector<uint8_t>& data)
{
    size_t total_written = 0;

    while (total_written < data.size())
    {
        ssize_t written = write(fd,
            data.data() + total_written,
            data.size() - total_written);

        if (written == -1)
        {
            return std::unexpected(
                std::system_error(errno,                                    ①
                    std::system_category(),
                    "Error al escribir datos")
            );
        }

        total_written += static_cast<size_t>(written);
    }

    return {};
}
```

① `std::system_error` se construye con tres datos: el código de error del sistema –`errno`–, su categoría –`std::system_category()` para los errores del sistema– y un mensaje que añade contexto.

El uso de esta función sería similar al ejemplo anterior, pero ahora podemos acceder al objeto `std::system_error` para obtener tanto el código de error como el mensaje descriptivo:

```
std::vector<uint8_t> datos = {/* ... */};
auto result = write_data(fd, datos);

if (!result.has_value())
```

```
{
    std::println(stderr, "error ({}) al escribir: {}",
        result.error().code().value(),                    ①
        result.error().what());                           ②
    // Manejar error...
    return /* ... */;
}

// Continuar con la siguiente operación...
```

① `result.error().code().value()` retorna el valor de `errno` encapsulado en el objeto `std::system_error`.
② `result.error().what()` retorna el mensaje descriptivo del error, que incluye tanto el mensaje personalizado como la descripción del código de error que proporciona el sistema.

B.3.1. Una función auxiliar para no repetirse

Construir el objeto `std::system_error` en cada punto de error es repetitivo. Por eso conviene definir una pequeña función auxiliar que capture `errno` en el mismo instante en que se detecta el error:

```
std::unexpected<std::system_error> system_error_from_errno(
    std::string context)
{
    return std::unexpected(
        std::system_error(errno, std::system_category(),
            std::move(context)));
}
```

Al retornar un `std::unexpected`, el resultado sirve tanto para funciones que devuelven `std::expected<void, std::system_error>` como para las que devuelven algún valor, y cada punto de error del programa se reduce a una línea:

```
int src_fd = open(src_path.c_str(), O_RDONLY);
if (src_fd == -1)
{
    return system_error_from_errno(
        std::format("Error al abrir '{}'", src_path));
}
```

> **Advertencia**
>
> `system_error_from_errno()` lee `errno`, así que debe llamarse **antes** de cualquier otra llamada al sistema, incluidas las llamadas a `close()` para cerrar los descriptores que hubiéramos abierto hasta ese momento. Si hay que cerrar descriptores antes de retornar, se debe guardar primero el error:
>
> ```
> auto error = system_error_from_errno(
>     std::format("Error al abrir '{}'", dest_path));
> close(src_fd);                                                    ①
> return error;
> ```
>
> ① Si se llamase a `system_error_from_errno()` después de este `close()`, el valor de `errno` ya no sería el del error que queremos informar, sino el que hubiera dejado `close()`.

B.4. Forzar la comprobación de errores con `[[nodiscard]]`

Una de las principales ventajas de usar `std::expected` es que podemos obligar al programador a comprobar si la operación tuvo éxito antes de usar el resultado. Esto se logra usando el atributo `[[nodiscard]]` en la declaración de la función:

```
[[nodiscard]]
std::expected<int, std::system_error> open_file(const std::string& path,
    int flags, mode_t mode = 0666)
{
    int fd = open(path.c_str(), flags, mode);
    if (fd == -1)
    {
        return system_error_from_errno(
            std::format("Error al abrir '{}'", path));
    }

    return fd;
}
```

Con `[[nodiscard]]`, el compilador emitirá una advertencia si intentamos llamar a la función sin usar su valor de retorno. Por ejemplo, en el siguiente caso, el compilador emitirá una advertencia porque el valor de retorno de `open_file()` se está ignorando.

```
open_file("archivo.txt", O_RDONLY);
```

Mientras que en este otro caso, el valor de retorno se almacena en `result` y se comprueba correctamente, por lo que no habrá advertencia:

```
auto result = open_file("archivo.txt", O_RDONLY);
if (!result.has_value())
{
    // Manejar error...
    return /* ... */;
}
```

Esto es especialmente útil para funciones que pueden fallar, ya que obliga a los programadores a manejar las condiciones de error de manera explícita. Si realmente queremos ignorar el resultado, debemos hacerlo de forma explícita:

```
// Ignorar explícitamente el resultado (no recomendado)
[[maybe_unused]] auto result = open_file("archivo.txt", O_RDONLY);

// O usando std::ignore
std::ignore = open_file("archivo.txt", O_RDONLY);
```

> **Nota**
>
> El atributo `[[nodiscard]]` es especialmente valioso durante el desarrollo, ya que ayuda a detectar errores donde olvidamos comprobar si una operación tuvo éxito. Recomendamos usar este atributo en todas las funciones que retornen `std::expected`.

B.5. Excepciones

Como se ha dicho, en esta práctica todos los errores se retornan a quien llamó a la función, usando `std::expected`. Sin embargo, C++ dispone de un segundo mecanismo para informar de errores –las excepciones– que conviene conocer por dos motivos: porque algunas funciones de la librería estándar lo usan, aunque nosotros no lo hagamos, y porque necesitamos un lugar donde capturar las excepciones que puedan llegar a escaparse.

B.5.1. Qué ocurre cuando se lanza una excepción

Cuando una función lanza una excepción con `throw`, deja de ejecutarse inmediatamente en ese punto y el control vuelve hacia atrás, función a función, hasta encontrar un bloque `try/catch` que capture ese tipo de excepción. Durante ese recorrido –llamado *desenrollado de la pila*– se destruyen los objetos locales de cada función que se abandona. Si ninguna función captura la excepción, el programa es abortado llamando a `std::terminate()`.

> **Advertencia**
>
> Que se destruyan los objetos locales **no** significa que se liberen los recursos del sistema que estuviéramos usando. Un descriptor de archivo es un simple `int`, así que no tiene destructor y nadie llamará a `close()` por nosotros.
> En una función como `copy_file()`, que abre descriptores y los cierra explícitamente en cada camino de salida, dejar escapar una excepción significa dejar sin cerrar todos los descriptores que estuvieran abiertos en ese momento. Es una de las razones por las que en esta práctica los errores se retornan en lugar de lanzarse.

B.5.2. De dónde salen las excepciones si no las lanzamos

Aunque nuestro código no use `throw`, hay funciones de la librería estándar que informan de sus errores lanzando excepciones:

`std::stoi()`, `std::stol()` y similares Lanzan `std::invalid_argument` si el texto no es un número y `std::out_of_range` si el valor no cabe en el tipo de destino. Por eso, para convertir texto a números, es preferible `std::from_chars()`, que informa del error retornándolo.

Las funciones de `std::filesystem` Casi todas tienen dos versiones. Una que lanza `std::filesystem::filesystem_error` –que deriva de `std::system_error`– y otra que recibe un `std::error_code` como último argumento y no lanza nada. De `std::filesystem` solo usamos `path` –que manipula cadenas de texto y no lanza nada– y `canonical()`, de la que siempre debemos usar la versión que devuelve los errores con `std::error_code`.

Reserva de memoria Crear un `std::vector` o usar `new` puede lanzar `std::bad_alloc` si no hay memoria suficiente.

Por ejemplo, `std::stoi()` lanza una excepción cuando el texto que recibe no representa un número válido:

```
int numero = std::stoi(contenido);                                    (1)
```

(1) Si `contenido` no contiene un número, `std::stoi()` lanza `std::invalid_argument` y la ejecución **no continúa** en la línea siguiente, sino que sale de la función.

B.5.3. Capturar excepciones

Para capturar una excepción hay que encerrar el código que puede lanzarla en un bloque `try` y añadir a continuación uno o más bloques `catch`, uno por cada tipo de excepción que queramos tratar:

```
try
{
    int numero = std::stoi(contenido);
    // ...
}
catch (const std::exception& e)                                    ①
{
    std::println(stderr, "error inesperado: {}", e.what());        ②
}
```

① Las excepciones se capturan **siempre por referencia constante**, para evitar que se haga una copia del objeto de la excepción.

② El método `what()` devuelve el mensaje descriptivo del error. Cuando la excepción es un `std::system_error`, se puede obtener además el código de error con `e.code()`, como se explica en la Sección B.3.

Los bloques `catch` se comprueban en orden, así que hay que poner siempre los tipos más específicos antes que los más generales. Como todas las excepciones de la librería estándar derivan de `std::exception`, un `catch` de ese tipo captura cualquiera de ellas.

B.5.4. El patrón `protected_main()`

Un patrón habitual para no mezclar el manejo de excepciones con el resto del código consiste en trasladar todo el contenido de `main()` a una función `protected_main()`, y dejar en `main()` únicamente el bloque `try/catch`:

```
int protected_main(int argc, char* argv[])
{
    // Todo el código de la aplicación...

    return EXIT_SUCCESS;
}

int main(int argc, char* argv[])
{
    try
    {
        return protected_main(argc, argv);                         ①
    }
    catch (const std::system_error& e)                             ②
    {
        std::println(stderr, "error inesperado ({}): {}",
            e.code().value(), e.what());
    }
    catch (const std::exception& e)                                ③
```

```
    {
        std::println(stderr, "error inesperado: {}", e.what());
    }

    return EXIT_FAILURE;
}
```

① `main()` se limita a llamar a `protected_main()`, pasándole los argumentos de la línea de comandos, y a devolver su resultado.
② Se captura primero `std::system_error`, el tipo más específico, del que se puede obtener el código de error.
③ Y después `std::exception`, que captura cualquier otra excepción de la librería estándar, como `std::bad_alloc`.

De esta forma se puede escribir el programa en `protected_main()` sabiendo que cualquier excepción que se escape será interceptada en `main()`, que mostrará un mensaje de error y terminará ordenadamente, en lugar de que el programa aborte.

> **!** Importante
>
> En esta práctica, `protected_main()` es **un seguro**. Si llega a capturar algo, es que hay un fallo grave en el programa o en el sistema. Los errores previsibles –que un archivo no exista, que no haya permisos, que no quede espacio– no llegan hasta aquí, porque se retornan con `std::expected` y se tratan donde corresponde.

C. Tuberías anónimas

Las tuberías son un mecanismo de comunicación entre procesos que utiliza la misma interfaz que empleamos para manipular archivos: se envían datos mediante `write()` en un extremo y se reciben mediante `read()` en el otro.

Existen dos tipos de tuberías:

Tuberías sin nombre o anónimas Se crean mediante `pipe()`. Se utilizan para la comunicación entre procesos relacionados, como un proceso padre y sus hijos.

Tuberías con nombre (FIFO) Se crean en el sistema de archivos mediante `mkfifo()`. Se utilizan para la comunicación entre procesos no relacionados que se ejecutan de forma independiente.

En esta sección nos centraremos en las **tuberías anónimas** y cómo utilizarlas junto con `fork()` y `exec()` para comunicar procesos.

C.1. Creación de tuberías anónimas

Para crear una tubería anónima, se utiliza la función `pipe()`:

```
int pipe(int pipefd[2]);
```

`pipefd` es un array de dos elementos que se rellenará con los descriptores de la tubería:

- `pipefd[0]`: Descriptor para **leer** de la tubería
- `pipefd[1]`: Descriptor para **escribir** en la tubería

Retorna `0` si tiene éxito, o `-1` estableciendo `errno`.

Por ejemplo, el siguiente código crea una tubería anónima e imprime los descriptores obtenidos:

```
int pipefd[2];

int result = pipe(pipefd);
if (result == -1)
{
    // Error al crear la tubería...
    return /* ... */;
```

```
}

std::println("Descriptor para lectura: {}", pipefd[0]);
std::println("Descriptor para escritura: {}", pipefd[1]);
```

C.1.1. Uso básico de tuberías anónimas

Una vez creada la tubería, se puede escribir datos en un extremo y leerlos desde el otro. Por ejemplo:

```
int pipefd[2];
int result = pipe(pipefd);
if (result == -1)
{
    // Error al crear la tubería...
    return /* ... */;
}

// Escribir datos en la tubería
std::string_view mensaje = "Hola desde la tubería";
ssize_t bytes_written = write(pipefd[1], mensaje.data(), mensaje.size());
if (bytes_written == -1)
{
    // Error al escribir en la tubería...
    return /* ... */;
}

// Leer datos de la tubería
std::array<char, 100> buffer;
ssize_t bytes_read = read(pipefd[0], buffer.data(), buffer.size() - 1);   ①
if (bytes_read == -1)
{
    // Error al leer de la tubería...
    return /* ... */;
}

buffer[bytes_read] = '\0';                                                ②
std::println("Mensaje leído: {}", buffer.data());

// Cerrar los descriptores
close(pipefd[0]);
close(pipefd[1]);
```

① Restamos 1 al tamaño del búfer para dejar espacio para el terminador nulo.

② Añadir un terminador nulo para tratar el búfer como una cadena. El terminador nulo no se envió porque solo se enviaron los bytes de la cadena de texto.

> **Nota**
>
> El ejemplo da por hecho que el mensaje cabe en una sola operación, pero `read()` y `write()` sobre una tubería pueden transferir menos bytes de los solicitados, igual que sobre un archivo, tal y como se explica en la Sección A.5.1: en un programa real hay que llamarlas dentro de un bucle.
> Sobre una tubería, además, ambas se bloquean: `read()` mientras no haya datos que leer, y `write()` mientras no quede sitio en el búfer.

C.2. Comunicación entre procesos con tuberías

Los descriptores de archivo abiertos se **heredan de procesos padre a procesos hijo** cuando se utiliza `fork()` (ver la Sección D.1).

Esto significa que si se crea una tubería antes de llamar a `fork()`, tanto el proceso padre como el proceso hijo tendrán acceso a los descriptores de la tubería y podrán comunicarse entre ellos, permitiendo a uno de los procesos escribir datos en la tubería y al otro leerlos. Para ilustrarlo, vamos a trasladar el ejemplo anterior a un escenario con `fork()`:

```
int pipefd[2];
int result = pipe(pipefd);                                          ①
if (result == -1)
{
    // Error al crear la tubería...
    return /* ... */;
}

pid_t pid = fork();

if (pid == 0)
{
    // En el proceso hijo...

    close(pipefd[1]);                                               ②

    std::array<char, 100> buffer;
    ssize_t bytes_read = read(pipefd[0],
        buffer.data(), buffer.size() - 1);
    if (bytes_read == -1)
    {
        // Error al leer de la tubería...
        _exit(/* ... */);
```

```
    }

    buffer[bytes_read] = '\0';
    std::println("El hijo recibió: {}", buffer.data());

    close(pipefd[0]);
    _exit(EXIT_SUCCESS);
}
else if (pid > 0)
{
    // En el proceso padre...

    close(pipefd[0]);

    // Enviar mensaje al hijo
    std::string_view mensaje = "Hola desde el padre";
    ssize_t bytes_written = write(pipefd[1],
        mensaje.data(), mensaje.size());
    if (bytes_written == -1)
    {
        // Error al escribir en la tubería...
        // Cerrar descriptores, esperar al hijo y...
        return /* ... */;
    }

    close(pipefd[1]);
    wait(nullptr);                                          ③
}
else
{
    // Error al crear el proceso hijo...
    // Cerrar descriptores y...
    return /* ... */;
}
```

① Crear la tubería antes de llamar a `fork()`, para que ambos procesos la hereden.
② Cada proceso cierra el extremo de la tubería que no va a usar (el hijo cierra el de escritura y el padre cierra el de lectura).
③ El padre espera a que el hijo termine con `wait()`. El puntero `nullptr` indica que no nos interesa el estado de salida del hijo en este caso.

! Importancia de cerrar extremos no usados

Es **fundamental** que cada proceso cierre los extremos de la tubería que no va a usar:

- El proceso que solo escribe debe cerrar `pipefd[0]` (lectura).
- El proceso que solo lee debe cerrar `pipefd[1]` (escritura).

Esto es necesario para que:

1. **Se detecte el fin de archivo (EOF):** Cuando todos los descriptores de escritura se cierran, `read()` en el extremo de lectura retorna `0`, indicando EOF.
2. **Se eviten fugas de descriptores:** Los descriptores abiertos consumen recursos del sistema.
3. **Funcione la detección de tubería rota:** Si el lector cierra su extremo y el escritor intenta escribir, recibirá la señal `SIGPIPE`.

C.3. Escribir en una tubería rota

Si el proceso que lee de la tubería termina o cierra su extremo de lectura antes de que el escritor haya terminado, las llamadas a `write()` en el extremo de escritura fallan con `EPIPE` y el proceso escritor recibe la señal `SIGPIPE`, que lo termina si no está manejada o bloqueada.

Ese es el comportamiento que se espera en la *shell* –si `wc -w` termina, el comando que escribe a la izquierda de `|` debe terminar también– pero no en un programa que debe seguir funcionando aunque falle el proceso con el que se comunica. Para evitarlo, se bloquea `SIGPIPE` antes de empezar a escribir en la tubería:

```
sigset_t sigpipe_set;

sigemptyset(&sigpipe_set);                                          ①
sigaddset(&sigpipe_set, SIGPIPE);

if (sigprocmask(SIG_BLOCK, &sigpipe_set, nullptr) == -1)            ②
{
    // Error al bloquear la señal...
    return /* ... */;
}
```

① Un conjunto de señales (`sigset_t`) que contiene solo `SIGPIPE`.
② Bloquear las señales del conjunto. A partir de aquí, escribir en una tubería cuyo extremo de lectura está cerrado ya no termina el proceso: `write()` retorna `-1` con `errno` igual a `EPIPE`, que se trata como cualquier otro error.

Lo habitual es bloquear `SIGPIPE` una sola vez, al principio del programa, y dejarla bloqueada mientras este se ejecuta. Solo si el bloqueo debe afectar únicamente a una parte del programa es necesario guardar la máscara de señales anterior –en el segundo argumento de `sigprocmask()`– para reponerla después con `SIG_SETMASK`.

Hay que tener en cuenta que la máscara de señales se hereda al crear un proceso hijo con `fork()` y se conserva al reemplazar su imagen con `exec()`, así que los comandos que ejecute el programa también lo harán con `SIGPIPE` bloqueada.

Las funciones `sigemptyset()`, `sigaddset()` y `sigprocmask()` están declaradas en la cabecera `<signal.h>`.

C.4. Redirección con tuberías

Como los descriptores de tuberías son descriptores de archivo normales, se pueden usar con `dup2()` (ver la Sección D.4) para redirigir la entrada o salida estándar de un proceso hacia o desde una tubería. Esto es especialmente útil cuando se combina con `fork()` y `exec()` (ver la Sección D.1 y la Sección D.2) para ejecutar programas externos que leen de `stdin` o escriben en `stdout`.

C.4.1. Redirigir la entrada y la salida estándar

En este patrón, el hijo redirige su salida estándar a la tubería para que el padre pueda leer lo que el hijo imprime. La estructura es la misma del ejemplo anterior –crear la tubería, llamar a `fork()` y que cada proceso cierre el extremo que no va a usar– pero con los papeles intercambiados: ahora escribe el hijo y lee el padre. Lo único que se añade es la redirección, que se hace en el hijo:

```
if (pid == 0)
{
    // En el proceso hijo, su salida estándar irá a la tubería

    close(pipefd[0]);                                                    ①

    dup2(pipefd[1], STDOUT_FILENO);                                      ②
    close(pipefd[1]);                                                    ③

    // Todo lo que el hijo escriba a stdout irá a la tubería
    std::println("Mensaje del hijo");

    _exit(EXIT_SUCCESS);
}
```

① El hijo cierra el extremo de lectura, que no va a usar.
② El hijo redirige su salida estándar al extremo de escritura de la tubería.
③ Y cierra el descriptor original, porque una vez redirigido ya no hace falta.

El padre, por su parte, cierra `pipefd[1]` y lee de `pipefd[0]` igual que leía el hijo en el ejemplo anterior.

La redirección en sentido contrario –que el hijo lea de la tubería a través de su entrada estándar– es simétrica: basta con `dup2(pipefd[0], STDIN_FILENO)` en el hijo, y que sea el padre quien escriba. Es lo que hace el ejemplo de la sección siguiente.

C.5. Patrón completo: Pipe + Fork + Exec + Redirección

El patrón más potente en programación Unix combina tuberías, `fork()`, `exec()` y redirecciones para ejecutar un programa externo y comunicarse con él. Este patrón es la base del operador `|` en la *shell*, que conecta la salida de un comando con la entrada de otro.

```
int pipefd[2];
int result = pipe(pipefd);                                                    ①
if (result == -1)
{
    // Error al crear la tubería...
    return /* ... */;
}

pid_t pid = fork();                                                           ②

if (pid == 0)
{
    // En el proceso hijo, su entrada estándar vendrá de la tubería

    close(pipefd[1]);                                                         ③

    dup2(pipefd[0], STDIN_FILENO);                                            ④
    close(pipefd[0]);                                                         ⑤

    // Ejecutar en el proceso hijo el comando que leerá de stdin
    execlp("wc", "wc", "-w", nullptr);                                        ⑥

    // Si llegamos aquí, exec() falló
    std::println(stderr, "Error: {}", std::strerror(errno));                  ⑦
    _exit(127);
}
else if (pid > 0)
{
    // En el proceso padre, escribimos en la tubería

    close(pipefd[0]);                                                         ③

    std::string_view texto = "Hola mundo desde una tubería\n";
    ssize_t bytes_written = write(pipefd[1], texto.data(), texto.size());
  ↪                                                                           ⑧
```

```
    if (bytes_written == -1)
    {
        // Error al escribir en la tubería...
        // Cerrar descriptores, esperar al hijo y...
        return /* ... */;
    }

    close(pipefd[1]);                                                    ⑨

    int status;
    waitpid(pid, &status, 0);                                            ⑩

    if (WIFEXITED(status) && WEXITSTATUS(status) == 0)
    {
        std::println("Comando ejecutado con éxito");
    }
    else
    {
        std::println(stderr, "Comando falló");
    }
}
else
{
    // Error al crear el proceso hijo...
    // Cerrar descriptores y...
    return /* ... */;
}
```

① Crear la tubería antes de llamar a `fork()`, para que ambos procesos tengan acceso a ella.
② Crear un proceso hijo con `fork()`.
③ Como antes, cada proceso cierra el extremo de la tubería que no va a utilizar.
④ El hijo redirige su `stdin` al extremo de lectura de la tubería.
⑤ El hijo cierra el descriptor original (ya no es necesario).
⑥ El hijo ejecuta `wc -w`, que leerá de su entrada estándar; por la redirección, esa entrada es el extremo de lectura de la tubería.
⑦ Si `exec()` falla, se imprime un mensaje de error y el hijo termina con el código `127`, para que el padre pueda distinguir el fallo al lanzar el comando de un código de salida del propio comando (ver la Sección D.3.2).
⑧ El padre escribe texto en el extremo de escritura de la tubería.
⑨ El padre cierra el extremo de escritura. Esto envía EOF al hijo, al no haber más descriptores de escritura abiertos.
⑩ El padre espera a que el hijo termine y verifica su estado de salida.

Este patrón es equivalente al uso de tuberías en la *shell*:

```
echo "Hola mundo desde una tubería" | wc -w
```

C.6. Redirección bidireccional

Para comunicación bidireccional entre padre e hijo se necesitan **dos tuberías**: una para enviar datos del padre al hijo y otra para las respuestas del hijo al padre. Con una sola no basta, porque los datos que un proceso escribe en ella también los puede leer él mismo.

Lo único que cambia respecto a los ejemplos anteriores es el reparto de los cuatro extremos entre los dos procesos:

```
int pipe_to_child[2];
int pipe_from_child[2];

// Crear ambas tuberías, comprobando el resultado de cada pipe()...

pid_t pid = fork();

if (pid == 0)
{
    // En el proceso hijo...

    close(pipe_to_child[1]);                                          ①
    close(pipe_from_child[0]);

    // Leer de pipe_to_child[0] y responder por pipe_from_child[1]...
}
else if (pid > 0)
{
    // En el proceso padre...

    close(pipe_to_child[0]);                                          ②
    close(pipe_from_child[1]);

    // Escribir en pipe_to_child[1] y leer de pipe_from_child[0]...
}
```

① El hijo se queda con el extremo de lectura de `pipe_to_child` y con el de escritura de `pipe_from_child`, y cierra los otros dos.
② El padre hace justo lo contrario.

El resto –las llamadas a `read()` y `write()`, el cierre de los extremos de escritura para enviar EOF y la espera con `wait()`– es igual que en los ejemplos anteriores.

D. Gestión de procesos

D.1. Creación de procesos con `fork()`

La función `fork()` se utiliza para crear un nuevo proceso (hijo) duplicando el proceso que la llama (proceso padre):

```
pid_t fork();
```

El nuevo proceso tiene un espacio de direcciones independiente y comienza a ejecutarse desde el punto donde se llamó a `fork()`, pero tiene respecto al padre:

- El mismo código
- La misma memoria (copiada, no compartida)
- Los mismos descriptores de archivo abiertos
- El mismo directorio de trabajo
- Las mismas variables de entorno

Según el valor retornado por `fork()`, se puede determinar si el código se está ejecutando en el proceso padre o en el proceso hijo:

- En el **proceso padre**, retorna el PID del hijo (valor positivo).
- En el **proceso hijo**, retorna `0`.
- En caso de **error**, no se crea el hijo, retorna `-1` y establece `errno` en el padre.

A continuación se muestra un ejemplo básico de uso de `fork()`:

```
pid_t pid = fork();

if (pid == -1)
{
    // Error al crear el proceso...
    return /* ... */;
}
else if (pid == 0)
{
    // En el proceso hijo...

    std::println("Soy el hijo, mi PID es {}", getpid());
```

```
    std::println("Mi padre tiene PID {}", getppid());

    _exit(EXIT_SUCCESS);                                                    ①
}
else
{
    // En el proceso padre...

    std::println("Soy el padre, mi PID es {}", getpid());
    std::println("Creé un hijo con PID {}", pid);

    wait(nullptr);                                                          ②
}
```

① El proceso hijo termina con `_exit()` para evitar que salga del `if` y siga ejecutando el código del padre. Tiene que ser `_exit()`, y nunca `std::exit()` ni un `return`, por los motivos que se explican en la Sección D.1.1.

② El proceso padre espera a que el hijo termine con `wait()`. Esto es importante para evitar que el proceso hijo quede como zombie cuando termine.

D.1.1. Cómo debe terminar el proceso hijo

La forma recomendada de terminar un programa en C++ es retornando desde `main()` o llamando a `std::exit()`. Sin embargo, en el caso de un proceso hijo creado con `fork()`, ninguna de estas dos formas es adecuada. Las dos ejecutan los manejadores registrados con `atexit()`, destruyen los objetos globales y vacían y cierran los flujos de salida con búfer. Todo ese estado es una copia del que tenía el padre al llamar a `fork()`, lo que en teoría es inofensivo, porque ocurre en la memoria del hijo, pero los efectos de hacerlo no lo son, ya que actúan sobre recursos que los dos procesos comparten. Por ejemplo, un destructor que borre un archivo lo borra para ambos, y vaciar los búferes heredados escribe en el mismo recurso que usa el padre, duplicando una salida que el segundo todavía no había escrito.

Por eso, en este caso conviene terminar el hijo de forma inmediata, sin tocar nada de lo anterior. Y para eso está la función `_exit()` –declarada en `<unistd.h>`– o su equivalente en C++ `std::_Exit()` –declarada en `<cstdlib>`–. Los descriptores de archivo se cierran y la memoria se libera igualmente, porque de eso se encarga el núcleo cuando termina cualquier proceso.

Como `_exit()` tampoco ejecuta los destructores de las variables locales, si queremos asegurarnos de que se llaman, conviene poner el cuerpo del hijo en su propia función y dejar un único punto de salida:

```
pid_t pid = fork();

if (pid == -1)
```

```
{
    // Error al crear el proceso...
    return /* ... */;
}

if (pid == 0)
{
    _exit(child_main(/* ... */));                                    ①
}

// A partir de aquí solo sigue el proceso padre...
```

① `child_main()` retorna el código de salida del hijo. Al retornar se destruyen sus variables locales, con lo que no se pierde RAII, y solo entonces se llama a `_exit()`.

La regla es fácil de recordar: del camino del hijo no se retorna nunca –hacerlo lo devolvería al flujo del padre a partir de `fork()`– y siempre se sale por ese único `_exit()`.

Hay que tener en cuenta, además, estos dos detalles:

- Si el padre tiene salida pendiente en los búferes, el hijo se lleva una copia de ella. Conviene vaciarlos con `std::fflush(nullptr)` **antes** de `fork()`. Y dentro del hijo, usar `write()` en lugar de la salida con búfer o, si se usa esta, llamar a `std::fflush()` antes de terminar para asegurarse de que se escribe.
- Ninguna excepción debe escapar del cuerpo del hijo. Si lo hiciera, el desenrollado de la pila la llevaría más allá del `fork()`, hasta el `try`/`catch` del padre, y tendríamos dos procesos ejecutando el mismo manejador de errores. Por eso `child_main()` debe capturar con `catch (...)` y terminar con el código que corresponda.

Si el hijo va a llamar a `exec()`, nada de lo anterior le afecta en cuanto `exec()` tiene éxito, porque sustituye la imagen del proceso entera. Pero sigue haciendo falta para terminar en el tramo que va desde `fork()` hasta `exec()`, y cuando el propio `exec()` falla.

D.1.2. Descriptores de archivo compartidos

Como hemos comentado, los descriptores de archivo abiertos antes de `fork()` son **compartidos** entre padre e hijo en el sentido de que ambos tienen copias de los descriptores que apuntan al **mismo recurso subyacente** (archivo, tubería, *socket*, etc.).

Esto significa que:

- Ambos pueden leer/escribir en el mismo archivo.
- La posición de lectura/escritura es compartida, por lo que una operación de lectura/escritura en uno mueve la posición para el otro.
- Ambos deben cerrar sus descriptores independientemente.

Por ejemplo, si ejecutamos el siguiente código el archivo contendrá ambos mensajes escritos por padre e hijo, según el orden de ejecución. Padre e hijo no se sobrescriben mutuamente porque usan el mismo descriptor compartido, por lo que cada escritura avanza la posición en el archivo para los dos:

```
int fd = open("archivo.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);          ①

pid_t pid = fork();

if (pid == 0)
{
    // El proceso hijo escribe en el archivo...

    write(fd, "Hijo\n", 5);
    close(fd);

    _exit(EXIT_SUCCESS);
}
else if (pid > 0)
{
    // El proceso padre escribe en el archivo...

    write(fd, "Padre\n", 6);
    close(fd);

    wait(nullptr);
}
```

① El archivo se abre **antes** de `fork()`. Por eso padre e hijo comparten el descriptor y, con él, la posición de escritura.

D.2. Reemplazo de programa con `exec()`

La familia de funciones `exec()` se utiliza para **reemplazar** el código del proceso actual con un nuevo programa.

```
int execl(const char *path, const char *arg, ...);
int execlp(const char *file, const char *arg, ...);
int execle(const char *path, const char *arg, ...
    /*, (char *) nullptr, char *const envp[] */);
int execv(const char *path, char *const argv[]);
int execvp(const char *file, char *const argv[]);
int execve(const char *path, char *const argv[], char *const envp[]);
```

Estas funciones:

- **No retornan si tiene éxito:** El programa actual es completamente reemplazado por el nuevo programa, por lo que no vuelven al código que las llamó.
- **Solo retornan si hay error:** Retornan `-1` y establecen `errno`.
- **Los descriptores abiertos se mantienen:** Los descriptores de archivo abiertos antes de `exec()` permanecen abiertos en el nuevo programa (a menos que se obtuvieran con el flag `O_CLOEXEC`).
- **Variables de entorno heredadas:** El nuevo programa hereda las variables de entorno del proceso actual (excepto en las variantes de `exec()` con `e`).

D.2.1. Diferencias entre las variantes de `exec()`

Las distintas variantes de la función `exec()` permiten pasar argumentos y variables de entorno de diferentes maneras:

D.2.1.1. Por forma de pasar argumentos

`execl()`, `execlp()`, `execle()` Permiten pasar los argumentos al programa como una **lista variable** de argumentos separados por coma. El último argumento debe ser `nullptr`.

```
execl("/bin/ls", "ls", "-l", "/etc", nullptr);
```

`execv()`, `execvp()`, `execve()` Permiten pasar los argumentos al programa como un **array de cadenas** de caracteres. Esta versión es útil cuando el número de argumentos no se conoce en tiempo de compilación. El array debe terminar con `nullptr`.

```
char* args[] = {
    const_cast<char*>("ls"),
    const_cast<char*>("-l"),
    const_cast<char*>("/etc"),
    nullptr
};
execv("/bin/ls", args);
```

Los literales de cadena como `"ls"` son de tipo `const char*` porque no son modificables, pero los argumentos de `exec()` están declarados como `char*` por razones históricas. Por eso es necesario usar el operador `const_cast<char*>()` para quitar el `const`.

D.2.1.2. Por búsqueda en `PATH`

`execl()`, `execv()`, `execle()`, `execve()` Requieren la **ruta completa** (absoluta o relativa) del programa a ejecutar.

```
execl("/bin/ls", "ls", "-l", nullptr);
```

```
execv("/usr/bin/gcc", args);
```

`execlp()`, `execvp()` Si no se especifica la ruta completa, buscan el programa en los directorios especificados en la variable de entorno `PATH`.

```
execlp("ls", "ls", "-l", nullptr);                    ①
```

① Busca el ejecutable `ls` en `PATH` y lo ejecuta si lo encuentra.

```
execvp("gcc", args);                                  ①
```

① Busca el ejecutable `gcc` en `PATH` y lo ejecuta si lo encuentra.

D.2.1.3. Por variables de entorno

`execl()`, `execlp()`, `execv()`, `execvp()` El nuevo programa **hereda** las variables de entorno del proceso actual.

`execle()`, `execve()` Permiten especificar un array de **variables de entorno personalizadas** para el nuevo programa.

```
char* env[] = {                                       ①
  const_cast<char*>("LANG=en_US.UTF-8"),
  const_cast<char*>("PATH=/usr/bin:/bin"),
  nullptr
};
execle("/bin/ls", "ls", "-l", nullptr, env);
```

① Array con las variables de entorno con las que se ejecutará `/bin/ls`.

```
char* args[] = {
    const_cast<char*>("ls"),
    const_cast<char*>("-l"),
    nullptr
};
char* env[] = {                                       ①
  const_cast<char*>("LANG=en_US.UTF-8"),
  const_cast<char*>("PATH=/usr/bin:/bin"),
  nullptr
};
```

```
execve("/bin/ls", args, env);
```

① Array con las variables de entorno con las que se ejecutará `/bin/ls`.

D.2.2. Ejemplo con execvp()

La variante más común y conveniente es `execvp()` porque busca en `PATH` y acepta arrays, que se pueden construir dinámicamente durante la ejecución.

```
pid_t pid = fork();

if (pid == 0)
{
    // El proceso hijo ejecuta el comando ls -l /tmp

    char* args[] = {
        const_cast<char*>("ls"),                                    ①
        const_cast<char*>("-l"),
        const_cast<char*>("/tmp"),
        nullptr
    };

    execvp("ls", args);

    std::println(stderr, "Error al ejecutar ls: {}",
      std::strerror(errno));
    _exit(127);                                                     ②
}
else
{
    // El proceso padre espera a que el hijo termine

    int status;
    waitpid(pid, &status, 0);

    if (WIFEXITED(status))
    {
        std::println("El comando terminó con código {}",
          WEXITSTATUS(status));                                     ③
    }
}
```

① `argv[0]` debe ser el nombre del comando porque, por convención, en los sistemas POSIX el primer argumento es siempre el nombre del programa.

② El hijo termina con el código `127`, que por convención indica que no se pudo ejecutar el comando (ver la Sección D.3.2).
③ El valor que recoge `WEXITSTATUS()` es el que retornó `ls` al terminar. Por convención, `0` indica que el comando se ejecutó correctamente y cualquier otro valor, algún tipo de error.

D.3. Espera de procesos hijos con wait()

Las funciones `wait()` y `waitpid()` se utilizan para hacer que el proceso padre espere a que termine uno de sus procesos hijos.

```
pid_t wait(int *status);
pid_t waitpid(pid_t pid, int *status, int options);
```

En concreto:

`wait(status)` Bloquea el proceso hasta que **cualquier** proceso hijo termine. Retorna el PID del hijo que terminó, o `-1` si ocurrió un error.

`waitpid(pid, status, options)` Espera a que termine el hijo **específico** con el PID indicado.

- El argumento `options` permite modificar el comportamiento de la espera. Si `options` es `0`, se comporta igual que `wait()`, bloqueando el proceso y retornando el PID del hijo que terminó.
- Con `options = WNOHANG`, el proceso no se bloquea si el hijo no ha terminado, retornando `0` en ese caso. Esto sirve para comprobar periódicamente si un hijo ha terminado sin bloquear el proceso padre.

D.3.1. Interpretar el estado de salida

Al volver de `wait()` o `waitpid()`, el parámetro `status` contiene información sobre la terminación del proceso hijo. Se usan las siguientes macros para interpretarlo:

`WIFEXITED(status)` Se evalúa como cierto si el hijo terminó normalmente (con `_exit()`, `std::exit()` o retorno de `main()`).
`WEXITSTATUS(status)` Si `WIFEXITED` es cierto, retorna el código de salida del hijo. Del valor con el que terminó solo se conserva el byte menos significativo, así que el código que recibe el padre siempre está entre `0` y `255`. Por ejemplo, un hijo que termine con `256` le llega al padre como `0`, que significa éxito.
`WIFSIGNALED(status)` Se evalúa como cierto si el hijo fue terminado por una señal.
`WTERMSIG(status)` Si `WIFSIGNALED` es cierto, retorna el número de señal que terminó el hijo.

Por ejemplo, para capturar el código de salida de un hijo que termina normalmente:

```
pid_t pid = fork();

if (pid == 0)
{
    // En el proceso hijo...
    sleep(300);

    _exit(42);
}
else
{
    // En el proceso padre...

    int status;
    waitpid(pid, &status, 0);

    if (WIFEXITED(status))
    {
        int exit_code = WEXITSTATUS(status);
        std::println("Hijo terminó con código {}", exit_code);
    }
    else if (WIFSIGNALED(status))                                    ①
    {
        int signal = WTERMSIG(status);
        std::println("Hijo terminado por señal {}", signal);
    }
}
```

① Los dos casos se excluyen: o el hijo llegó a terminar por su cuenta con un código de salida, o lo terminó una señal antes de que pudiera hacerlo. Por ejemplo, si se le enviara `kill -15 <pid>` mientras espera en `sleep(300)`, se mostraría "Hijo terminado por señal 15".

D.3.2. Códigos de salida al ejecutar un comando

Cuando el hijo ejecuta un comando externo con `exec()`, lo único que el padre recibe de vuelta es su código de salida, así que ese número tiene que servir para dos cosas: informar de cómo terminó el comando y, si nunca llegó a ejecutarse, informar de por qué.

Para no confundir ambos casos, podemos seguir una convención similar a la que usa la *shell*, que reserva los códigos a partir de `125`:

`127` No se encontró el comando –`exec()` falló con `ENOENT`–.
`126` El comando existe, pero no se pudo ejecutar –`exec()` falló con `EACCES` o `ENOEXEC`–.

`125` El hijo falló antes de llegar a ejecutar el comando.

Por eso, en el ejemplo de la Sección D.2, el hijo termina con el código `127` si `execvp()` retorna, y el padre puede distinguir con `WEXITSTATUS()` un fallo al lanzar el comando de un código de salida del propio comando.

D.3.3. Importancia de `wait()`

Es **fundamental** llamar a `wait()` o `waitpid()` por cada hijo creado para:

1. **Evitar procesos zombie:** Si el padre no hace `wait()`, los hijos terminados quedan como zombies, ocupando entradas en la tabla de procesos del sistema.
2. **Liberar recursos:** El sistema libera completamente los recursos del hijo después de `wait()`.
3. **Verificar el éxito:** El código de salida indica si el comando ejecutado tuvo éxito.

D.4. Redirección de entrada/salida

La función `dup2()` duplica un descriptor de archivo en un número de descriptor específico, lo que es útil para redirigir la entrada/salida estándar de un proceso a un archivo o tubería:

```
int dup2(int oldfd, int newfd);
```

`oldfd` Descriptor del archivo o recurso origen al que se quiere redirigir.
`newfd` Número de descriptor destino, que es el descriptor redirigido.

`dup2()` cierra `newfd` si ya está abierto y luego copia `oldfd` a `newfd`, de modo que ambos descriptores apunten al mismo recurso. Después de la llamada, cualquier operación de lectura/escritura en `newfd` afectará al mismo recurso que `oldfd`. Si `oldfd` no es válido, o si ocurre algún otro error, `dup2()` retorna `-1` y establece `errno` con el código del error.

D.4.1. Ejemplos de redirección a archivos

El uso principal de `dup2()` es redirigir la entrada estándar (`stdin`), salida estándar (`stdout`), o salida de error (`stderr`) de un proceso a archivos:

D.4.1.1. Redirigir la salida estándar a un archivo

```
int fd = open("salida.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);
if (fd == -1)
{
    // Error al abrir archivo...
    return /* ... */;
}

dup2(fd, STDOUT_FILENO);
close(fd);                                                          ①

std::println("Este mensaje se guarda en salida.txt");               ②
write(STDOUT_FILENO, "También este\n", 14);
```

① Una vez redirigido, ya no es necesario el descriptor original `fd`, por lo que se cierra.
② Cualquier salida a `stdout` (como `std::println` o `write`) se escribirá en `salida.txt` en lugar de en la consola.

D.4.1.2. Redirigir la entrada estándar desde un archivo

```
int fd = open("entrada.txt", O_RDONLY);
if (fd == -1)
{
    // Error al abrir archivo...
    return /* ... */;
}

dup2(fd, STDIN_FILENO);
close(fd);                                                          ①

std::string line;
std::getline(std::cin, line);                                       ②
std::println("Leído de 'entrada.txt': {}", line);
```

① Como antes, el descriptor original sobra en cuanto la redirección está hecha.
② Cualquier lectura de `stdin` (como `std::cin` o `read`) leerá desde `entrada.txt` en lugar del teclado.

D.4.2. Redirección con `fork()` y `exec()`

La redirección es especialmente útil combinada con `fork()` y `exec()` para ejecutar comandos con entrada/salida redirigida:

D.4.2.1. Ejecutar comando con salida redirigida

Por ejemplo, para ejecutar `ls` de forma equivalente al comando de *shell* `ls -l > listado.txt`:

```
pid_t pid = fork();

if (pid == 0)
{
    // En el proceso hijo, se hará la redirección y se ejecuta el comando

    int fd = open("listado.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);      ①

    dup2(fd, STDOUT_FILENO);
    close(fd);

    execlp("ls", "ls", "-l", nullptr);                                      ②

    _exit(127);                                                             ③
}
else
{
    // El proceso padre espera a que el hijo termine

    int status;
    waitpid(pid, &status, 0);

    //...
}
```

① La redirección se hace en el hijo, después de `fork()`, para que la salida estándar del padre siga siendo la suya.
② `ls` hereda la redirección a través de `exec()`, porque los descriptores sobreviven al reemplazo de la imagen del proceso. Por eso su salida acaba en `listado.txt` sin que `ls` sepa nada de ello.
③ Si `execlp()` falla, el hijo termina con código `127` para indicar el error al ejecutar el comando.

D.4.2.2. Ejecutar comando con entrada redirigida

La redirección de la entrada es simétrica. Para ejecutar `wc` de forma equivalente al comando de *shell* `wc -l < datos.txt`, el hijo hace lo mismo que en el ejemplo anterior pero abriendo el archivo para lectura y redirigiendo `stdin`:

```
if (pid == 0)
{
    // En el proceso hijo, se hará la redirección y se ejecuta el comando

    int fd = open("datos.txt", O_RDONLY);                                    ①

    dup2(fd, STDIN_FILENO);                                                  ②
    close(fd);

    execlp("wc", "wc", "-l", nullptr);

    _exit(127);
}
```

① Ahora el archivo se abre para lectura, porque va a ser la entrada del comando.
② Y lo que se redirige es `stdin`, de modo que `wc -l` leerá de `datos.txt` en lugar del teclado.

El proceso padre no cambia: espera con `waitpid()` y comprueba el estado de salida del hijo.

E. Procesamiento de la línea de comandos

Casi todos los programas admiten opciones y argumentos en la línea de comandos, así que necesitan código para interpretarlos antes de comenzar a trabajar. En este apéndice vemos dos formas de hacerlo: recorrer a mano el vector de argumentos, que no necesita nada más que la librería estándar, y usar `getopt()`, que sigue las convenciones habituales de los programas en los sistemas tipo UNIX. En ambos casos el resultado es el mismo: una estructura con las opciones encontradas o un código de error que explica por qué las opciones no se pudieron obtener.

E.1. Argumentos de la línea de comandos

Como ya sabemos, los argumentos de la línea de comandos se reciben en `main()` mediante `argv`, un array de punteros a `char`:

```
int main(int argc, char* argv[])
{
    // ...
}
```

Para facilitar su procesamiento, `argv` se puede convertir en un vector de `std::string_view`:

```
int main(int argc, char* argv[])
{
    std::vector<std::string_view> args(argv + 1, argv + argc);

    // ...
}
```

Usamos `std::string_view` porque nos ofrece una funcionalidad similar a la de `std::string` pero sin hacer una copia de cada cadena. Es decir, los argumentos de la línea de comandos están en `argv`, pero accedemos a ellos a través de los objetos `std::string_view` en `args`.

Con este vector, es muy sencillo iterar para procesar los argumentos de la línea de comandos uno tras otro:

```
for (auto it = args.begin(), end = args.end(); it != end; ++it)
{
    if (*it == "-h" || *it == "--help")                                    ①
    {
        show_help = true;
    }
    else if (*it == "-o" || *it == "--output")                             ②
    {
        if (++it != end)                                                   ③
        {
            output_filename = *it;
        }
        else
        {
            // Error por falta de argumento...
            return /* ... */;
        }
    }
    else if (/* otra opción */)
    {
        // ...
    }
    else if (!it->starts_with("-"))                                        ④
    {
        additional_args.emplace_back(*it);
    }

    // ...
}
```

① Se compara cada cadena con las opciones conocidas: `-h` y `--help` son equivalentes y activan la misma bandera.

② `-o` no es una opción activa o inactiva, sino una que debe ir seguida de su propio argumento. Por eso no se puede resolver en la misma línea, hay que mirar también el argumento siguiente.

③ Al avanzar el iterador, ese argumento siguiente pasa a ser el valor de `-o` y el bucle ya no lo vuelve a ver como un argumento independiente, que es justo lo que se busca. Pero si llegamos a `end`, es que `-o` era el último argumento de la línea de comandos y la opción está incompleta.

④ Lo que no empieza por `-` no es una opción, sino uno de los argumentos del programa –nombres de archivo, por ejemplo–. Como pueden ser varios y su número no se conoce de antemano, se van guardando según aparecen para procesarlos cuando se hayan leído todos.

> **Nota**
>
> A primera vista, podría parecer más sencillo usar un índice entero para recorrer el array `argv` en lugar de un iterador. Sin embargo, usar iteradores nos permite avanzar fácilmente con `++it` cuando una opción tiene un argumento asociado (como en el caso de `-o`/`--output`).

E.2. Función `parse_arguments()`

Para separar mejor la responsabilidad, se puede meter el código anterior en una función `parse_arguments()` que procese los argumentos de la línea de comandos y retorne una estructura con los valores de las opciones encontradas:

```
enum class ParseArgsError                                                    ①
{
    missing_argument,
    unknown_option,
    // ...
};

struct Options
{
    bool show_help = false;
    std::string output_filename;
    // ...
    std::vector<std::string> additional_args;
};

std::expected<Options, ParseArgsError>
parse_arguments(int argc, char* argv[])
{
    std::vector<std::string_view> args(argv + 1, argv + argc);
    Options options;

    for (auto it = args.begin(), end = args.end(); it != end; ++it)
    {
        if (*it == "-h" || *it == "--help")
        {
            options.show_help = true;
        }
        else if (*it == "-o" || *it == "--output")
        {
            if (++it != end)
            {
```

```
                options.output_filename = *it;
            }
            else
            {
                return std::unexpected(                                    ②
                    ParseArgsError::missing_argument);
            }
        }
        else if (/* otra opción */)
        {
            // ...
        }
        else if (!it->starts_with("-"))
        {
            options.additional_args.emplace_back(*it);
        }
        else
        {
            return std::unexpected(
                ParseArgsError::unknown_option);
        }
    }

    return options;                                                        ③
}
```

① Los errores de la línea de comandos no vienen de ninguna llamada al sistema, así que no hay ningún `errno` que propagar y basta con definir un `enum` propio con los códigos de error.

② `std::unexpected` marca el valor como error, para que quien llame a la función pueda distinguirlo del `Options` que retorna cuando todo va bien.

③ La estructura se convierte sola en el `std::expected` que retorna la función.

Antes de usar `parse_arguments()` desde `main()` conviene tener a mano una función que recuerde cómo se invoca el programa, porque la convención es mostrar esa información tanto cuando se pide ayuda con `-h` como cuando la línea de comandos es incorrecta:

```
void print_usage()
{
    std::println(stderr, "uso: copy [-h] [-o ARCHIVO] ORIGEN DESTINO");
}
```

Con todo ello, el procesamiento de la línea de comandos en `main()` quedaría así:

```cpp
int main(int argc, char* argv[])
{
    auto arguments_result = parse_arguments(argc, argv);

    if (!arguments_result.has_value())                                    ①
    {
        switch (arguments_result.error())
        {
            case ParseArgsError::missing_argument:
                std::println(stderr,
                    "error: falta el argumento de una opción");
                break;

            case ParseArgsError::unknown_option:
                std::println(stderr,
                    "error: opción desconocida");
                break;
        }

        print_usage();                                                    ②
        return EXIT_FAILURE;
    }

    Options options = arguments_result.value();                           ③

    if (options.show_help)
    {
        print_usage();
        return EXIT_SUCCESS;
    }

    if (!options.output_filename.empty())
    {
        std::println("Archivo de salida: {}", options.output_filename);
    }

    // Procesar argumentos adicionales
    for (const auto& arg : options.additional_args)
    {
        std::println("Procesando: {}", arg);
    }

    return EXIT_SUCCESS;
}
```

① Si `has_value()` retorna `false`, lo que guarda `arguments_result` no es un `Options` sino

el `ParseArgsError` que explica qué salió mal.

② Ante cualquier error en la línea de comandos, mostrar cómo se usa el programa y terminar con un código distinto de `0`.

③ `value()` da acceso al `Options` que retornó `parse_arguments()` con las opciones encontradas. Solo tiene sentido llamarlo después de comprobar que la operación no terminó con un error.

E.3. Procesamiento con `getopt()`

Como alternativa al procesamiento manual de argumentos, POSIX proporciona la función `getopt()`, que simplifica el análisis de opciones de línea de comandos siguiendo las convenciones estándar de UNIX, lo que permite ofrecer una interfaz más consistente con la de otros programas.

```
int getopt(int argc, char *const argv[], const char *optstring);
```

E.3.1. Variables globales de `getopt()`

La función `getopt()` utiliza y modifica varias variables globales definidas en el archivo `<unistd.h>`:

`optind` Índice del siguiente elemento a procesar en `argv`. Inicialmente tiene el valor `1` (el primer argumento después del nombre del programa). Después de procesar todas las opciones, `optind` apunta al primer argumento que no es una opción.

`optopt` Carácter de la opción que causó un error (cuando `getopt()` retorna `'?'` o `':'`).

`optarg` Puntero al argumento de una opción que requiere argumento. Por ejemplo, si la opción es `-o archivo`, `optarg` apuntará a `"archivo"`.

`opterr` Vale `1` por omisión, lo que hace que `getopt()` imprima sus propios mensajes de error por la salida de error. Se puede poner a `0` para silenciarlos y encargarnos de los errores manualmente.

E.3.2. Cadena de opciones `optstring`

El parámetro `optstring` especifica las opciones válidas y si requieren argumentos:

- **Letra sola:** La opción no requiere argumento.
  - Ejemplo: `"abc"` acepta `-a`, `-b`, `-c`.
- **Letra seguida de `:`** La opción requiere un argumento.
  - Ejemplo: `"a:bc"` acepta `-a archivo`, `-b`, `-c`.
- **Letra seguida de `::`** La opción tiene un argumento opcional.

  - Ejemplo: `"a::bc"` acepta `-a`, `-a archivo`, `-b`, `-c`.

- **Dos puntos al principio:** Facilita distinguir entre cuando falta un argumento y cuando una opción es desconocida. Con `:` al principio, `getopt()` retorna `':'` cuando falta el argumento de una opción y reserva `'?'` para las opciones desconocidas. Además silencia los mensajes que `getopt()` imprimiría por su cuenta, sin necesidad de asignarle un 0 a `opterr`.

Por ejemplo, tanto `"ho:"` como `":ho:"` aceptan `-h` y `-o archivo`.

> **Nota**
>
> Para soportar opciones largas (`--help`, `--output`, etc.) hay que usar `getopt_long()`, una extensión GNU de `getopt()`.

E.3.3. Valor de retorno

La función `getopt()` retorna:

- El **carácter de la opción** encontrada (por ejemplo, `'h'`, `'o'`).
- `':'` si falta el argumento de una opción que lo requiere, siempre que `optstring` empiece por dos puntos.
- `'?'` si encuentra una opción desconocida.
- `-1` cuando no quedan más opciones por procesar.

E.3.4. Uso básico de `getopt()`

El patrón típico es llamar a `getopt()` en un bucle hasta que retorne `-1`:

```
for (int opt; (opt = getopt(argc, argv, ":ho:")) != -1;)                    ①
{
    switch (opt)
    {
        case 'h':
            show_help = true;
            break;

        case 'o':
            output_filename = optarg;                                       ②
            break;

        case ':':                                                           ③
            return EXIT_FAILURE;

        case '?':
```

```
            return EXIT_FAILURE;
    }
}

for (int i = optind; i < argc; i++)                                    ④
{
    // Procesar argumentos adicionales: argv[i]
}
```

① Los dos puntos iniciales de `optstring` reparten los dos errores posibles en dos casos distintos del `switch`, sin tener que consultar `optopt` opción por opción.
② `optarg` apunta al argumento de la opción que se acaba de reconocer, así que hay que usarlo antes de la siguiente iteración del bucle.
③ Si `opt` vale `':'` es que a la opción le falta su argumento, mientras que si vale `'?'` es que la opción indicada es desconocida –no está en `optstring`–. En ambos casos, `optopt` nos dice cuál es la opción culpable.
④ Al salir del bucle, `optind` apunta al primer argumento que no es una opción, así que este bucle recorre todo lo que venga después de ellas.

E.3.5. Implementación completa de `parse_arguments()` con `getopt()`

Ahora podemos adaptar el ejemplo anterior de `parse_arguments()` para usar `getopt()`:

```
std::expected<Options, ParseArgsError>
parse_arguments(int argc, char* argv[])
{
    Options options;

    for (int opt; (opt = getopt(argc, argv, ":ho:")) != -1;)
    {
        switch (opt)
        {
            case 'h':
                options.show_help = true;
                break;

            case 'o':
                options.output_filename = optarg;
                break;

            case ':':
                return std::unexpected(
                    ParseArgsError::missing_argument);
```

```
            case '?':
                return std::unexpected(
                    ParseArgsError::unknown_option);
        }
    }

    for (int i = optind; i < argc; i++)
    {
        options.additional_args.emplace_back(argv[i]);
    }

    return options;
}
```

El uso de esta versión de `parse_arguments()` desde `main()` es idéntico al del ejemplo anterior (Sección E.2). La firma y el `std::expected` que retorna son los mismos. Solo ha cambiado cómo procesan internamente los argumentos.

F. Variables de entorno

El entorno es el conjunto de variables con la forma `NOMBRE=valor` que acompaña a cada proceso. Es la forma habitual de configurar un programa sin pasarle nada en la línea de comandos. Por ejemplo, la variable `PATH` indica en qué directorios se buscan los comandos, `HOME` cuál es el directorio del usuario y `LANG` en qué idioma deben mostrarse los mensajes.

Un proceso hijo recibe una copia del entorno de su padre, así que puede consultarlo y modificarlo sin afectar al proceso que lo creó, y lo conserva al llamar a `exec()` salvo que se use una de las variantes con `e`.

F.1. Consultar una variable

Para consultar el valor de una variable se utiliza `getenv()`:

```
char *getenv(const char *name);
```

Retorna un puntero a la cadena con el valor de la variable, o `nullptr` si no está definida. Esa cadena pertenece al entorno del proceso, por lo que no hay que liberarla ni modificarla, y conviene copiarla si se va a conservar, porque cualquier cambio posterior del entorno puede invalidar el puntero.

Como el puntero puede ser nulo, resulta más cómodo envolver la llamada en una función que retorne un `std::optional<std::string>`, tal y como se muestra en la Sección B.1.

F.2. Definir y eliminar variables

Las funciones `setenv()` y `unsetenv()` cambian el entorno del proceso:

```
int setenv(const char *name, const char *value, int overwrite);
int unsetenv(const char *name);
```

`setenv()` Define la variable `name` con el valor `value`. Si la variable ya existía, solo la sobrescribe cuando `overwrite` es distinto de `0`; si es `0`, la deja como estaba, pero tampoco lo considera un error. Guarda copias del nombre y del valor, así que se le pueden pasar cadenas temporales.

`unsetenv()` Elimina la variable `name` del entorno. No es un error intentar eliminar una variable que no estuviera definida.

Ambas retornan `0` si tienen éxito, o `-1` estableciendo `errno`, que valdrá `EINVAL` si el nombre está vacío o contiene un `=`, o `ENOMEM` si no hay memoria suficiente para el nuevo entorno.

Como el entorno se hereda, definir una variable antes de crear un proceso hijo es la manera de configurar el programa que este vaya a ejecutar:

```
if (setenv("LC_ALL", "C", 1) == -1)
{
    return system_error_from_errno(
        "No se pudo definir LC_ALL");
}
```

También existe `putenv()`, que recibe una única cadena con la forma `NOMBRE=valor`:

```
int putenv(char *string);
```

A diferencia de `setenv()`, la cadena que se le pasa **pasa a formar parte del entorno** en lugar de copiarse. Por eso no se le debe pasar una variable local ni una cadena que se vaya a modificar o liberar después, porque el entorno quedaría apuntando a memoria que ya no es válida. Por tanto, salvo que se necesite ese comportamiento, es preferible usar `setenv()`.

> **!** Importante
>
> Cambiar el entorno no es seguro cuando el programa ya tiene varios hilos en marcha. Al definir una variable nueva, `setenv()` puede tener que sustituir el array del entorno completo, justo mientras otro hilo lo recorre o usa un puntero que le retornó `getenv()`. Si hay que modificar el entorno, lo seguro es hacerlo al principio de `main()`, antes de crear ningún hilo.

F.3. Recorrer todo el entorno

El entorno del proceso es un array de punteros a cadenas `NOMBRE=valor`, terminado en un puntero nulo, al que se accede mediante la variable global `environ`:

```
extern char **environ;
```

Recorrerlo es lo que hace el comando `env` cuando se ejecuta sin argumentos:

```
for (char** variable = environ; *variable != nullptr; ++variable)
{
    std::println("{}", *variable);
}
```

Ese array no debe modificarse directamente. Para eso están `setenv()` y `unsetenv()`. En cambio, sí es el tipo de array que esperan las variantes de `exec()` con `e`, que reciben el entorno con el que debe ejecutarse el nuevo programa en lugar de heredar el actual.

G. Hilos

Un **hilo** es una secuencia de instrucciones que se ejecuta en la CPU dentro de un proceso. Todos los hilos de un proceso comparten su espacio de direcciones y sus recursos –la memoria reservada, los descriptores de archivo abiertos o las señales– pero cada uno tiene su propia pila, su propio contador de programa y su propio juego de registros.

Esa es la diferencia con los procesos creados con `fork()`. Dos procesos son dos espacios de direcciones independientes, así que lo que uno escribe en su memoria el otro no lo ve y comunicarlos exige un mecanismo explícito, como una tubería. Dos hilos, en cambio, ven exactamente la misma memoria: compartir datos entre ellos no cuesta nada, pero tampoco hay nada que impida que se estorben.

POSIX define su propia librería de hilos, POSIX Threads, sobre la que están construidos los hilos de la librería estándar de C++ y de otros lenguajes en los sistemas POSIX. En esta práctica usaremos la librería estándar de C++, porque resuelve por nosotros la espera y la cancelación, y recurriremos a POSIX Threads solo para lo que el estándar de C++ no cubre: la manipulación de la máscara de señales de cada hilo, que se explica en la Sección G.7.

Los ejemplos de este apéndice utilizan hilos para contar cuántas veces aparece un valor en un fragmento de un vector.

```
void count_values(const std::vector<int>& values, std::size_t first,
    std::size_t last, int wanted, std::size_t& result);                    ①
```

① La función principal de un hilo no puede devolver un valor a quien lo creó, así que el resultado se deja en la variable `result` pasada por referencia.

G.1. Crear hilos y esperar a que terminen

Para crear un hilo se construye un objeto `std::jthread`, declarado en la cabecera `<thread>`, indicando la función que ejecutará el nuevo hilo –su **función principal**– y los argumentos que se le deben pasar:

```
void thread_function(int id)
{
    std::println("Hola desde el hilo {}", id);
}
```

```cpp
int main()
{
    std::jthread thread1(thread_function, 1);
    std::jthread thread2(thread_function, 2);

    std::println("Hola desde el hilo principal");
}
```

A partir de la construcción del objeto en el proceso hay tres secuencias de ejecución avanzando a la vez –la de `main()` y las dos de los hilos– así que el orden en que aparecen los mensajes del ejemplo no está determinado y puede cambiar de una ejecución a otra.

Los argumentos que siguen al nombre de la función se **copian** antes de pasárselos al hilo. Cuando lo que se quiere es que el hilo trabaje sobre un objeto de quien lo crea, hay que indicarlo explícitamente con `std::ref()` –o con `std::cref()`, si basta con una referencia constante– declaradas en `<functional>`:

```cpp
std::size_t result = 0;

std::jthread worker(count_values, std::cref(values),                    ①
    0, values.size(), 42, std::ref(result));                            ②
```

① Sin `std::cref()` el hilo recibiría una copia de todo el vector `values`.
② Y sin `std::ref()` recibiría una copia de `result`, de modo que quien creó el hilo no vería nunca el resultado que este escribiera en ella.

> **!** Importante
>
> El objeto al que se refiere `std::ref()` debe seguir existiendo mientras el hilo lo use. Si se destruye antes –por ejemplo, porque era una variable local de una función que ya ha retornado– el hilo seguirá escribiendo en esa dirección de memoria, que ya no le pertenece, posiblemente corrompiendo otros datos o provocando un fallo de segmentación.

Para esperar a que un hilo termine se llama a su método `join()`, que bloquea al hilo que lo llama hasta que el otro retorna de su función principal. Pero en el primer ejemplo no hay ninguna llamada a `join()` y el programa funciona igualmente, porque **el destructor de `std::jthread` llama a `join()` para esperar por su hilo**. Es decir, al terminar `main()` se destruyen `thread1` y `thread2`, y cada destructor espera a que su hilo acabe.

Eso permite escribir el código apoyándose en el ámbito de las variables, sin llamar explícitamente a `join()`:

```
std::size_t result = 0;

{
    std::jthread worker(count_values, std::cref(values),
        0, values.size(), 42, std::ref(result));
}                                                                    ①

std::println("El valor aparece {} veces", result);
```

① Al cerrar el ámbito se destruye `worker` y su destructor espera a que el hilo termine, de modo que `result` ya tiene su valor definitivo en la línea siguiente.

> **Nota**
>
> La clase de hilos original de C++11 es `std::thread`, que se usa igual pero cuyo destructor termina el programa con `std::terminate()` si antes no se ha llamado a `join()` o a `detach()`. `std::jthread`, incorporada en C++20, es la que conviene usar por defecto.

Cuando no interesa el resultado de un hilo ni se piensa esperar por él, se lo marca como separado con `detach()` y entonces es el sistema quien lo libera cuando termina, con las consecuencias que se detallan en la Sección G.7.2.

G.2. Repartir el trabajo entre varios hilos

Cuando hay que hacer varias veces el mismo trabajo sobre datos distintos se puede crear un hilo para cada parte y dejar que trabajen a la vez. Como la función principal de un hilo no devuelve nada, la forma más sencilla de recoger los resultados es reservar de antemano un hueco para cada uno y pasarle a cada hilo una referencia al suyo:

```
constexpr std::size_t num_threads = 4;
const std::size_t chunk_size = values.size() / num_threads;

std::vector<std::size_t> results(num_threads);                       ①

{
    std::vector<std::jthread> workers;
    workers.reserve(num_threads);

    for (std::size_t i = 0; i < num_threads; ++i)
    {
        std::size_t last = (i + 1 < num_threads)
            ? (i + 1) * chunk_size : values.size();                  ②
```

```
        workers.emplace_back(count_values, std::cref(values),
            i * chunk_size, last, 42, std::ref(results[i]));          ③
    }
}                                                                     ④

std::size_t total = 0;
for (std::size_t count : results)
{
    total += count;
}
```

① El vector se dimensiona **antes** de crear los hilos, para que sus elementos no cambien de dirección después.
② El último hilo se queda además con los elementos que sobran de la división entera.
③ Cada hilo escribe únicamente en su propio elemento. De esa forma ningún par de hilos toca el mismo dato y evitamos tener que protegerlo con algún mecanismo de sincronización.
④ Los hilos se destruyen en orden inverso al de creación, esperando por cada uno, y solo entonces –cuando se ha salido del ámbito– se examinan los resultados.

> **!** Importante
>
> Añadir elementos al vector de resultados mientras los hilos se ejecutan sería un error grave. Al crecer, un `std::vector` puede reubicar sus elementos en otra zona de memoria, y las referencias que ya tuvieran los hilos apuntarían a memoria liberada.

Repartir un trabajo entre varios hilos no lo hace necesariamente más rápido. Solo se gana tiempo si hay varios procesadores libres y si el trabajo es lo bastante grande como para compensar el coste de crear los hilos y esperarlos. Cuando lo que limita no es el procesador, sino el dispositivo del que se leen los datos, el reparto puede incluso empeorar el resultado.

G.3. Compartir datos entre hilos

En el ejemplo anterior los hilos no comparten nada porque cada uno escribe en su propio hueco. Pero en cuanto dos hilos acceden al mismo dato y al menos uno de ellos lo modifica, aparece una **condición de carrera**. Es decir, el resultado depende del orden en que el planificador entrelace sus instrucciones, que es distinto en cada ejecución.

Ni siquiera una operación tan simple como incrementar un contador es segura, porque en realidad requiere tres pasos –leer el valor, sumarle uno y escribirlo– y otro hilo puede colarse entre ellos.

Para los datos simples, la forma más sencilla de evitarlo es declararlos `std::atomic`, en la cabecera `<atomic>`. Las operaciones sobre un objeto atómico son indivisibles, así que ningún hilo puede observar una operación a medias. Por ejemplo, si los hilos del ejemplo anterior

llevan la cuenta de cuántos elementos han examinado entre todos usando una variable atómica:

```
std::atomic<std::size_t> processed{0};
```

Cada hilo de trabajo puede incrementar el contador sin miedo a que otro hilo se entrelace:

```
++processed;                                                    ①
```

① El incremento se realiza como una única operación indivisible, no como los tres pasos habituales.

Y un hilo que informa del progreso puede leerlo sin miedo a que otro hilo lo esté modificando al mismo tiempo:

```
std::println("{} de {} elementos procesados",
    processed.load(), values.size());
```

> **Nota**
>
> Un `std::atomic` no se puede copiar ni mover, y una estructura que contenga uno tampoco. Eso obliga a pasarla siempre por referencia –con `std::ref()` al crear el hilo– y a guardarla, si hacen falta varias, en un contenedor que no reubique sus elementos.

Proteger estructuras de datos más complejas, donde hay que mantener coherentes varios campos a la vez, exige mecanismos como `std::mutex`.

G.4. Cancelar hilos

Cancelar un hilo es terminarlo antes de que acabe su trabajo.

POSIX Threads permite pedir que un hilo muera mientras se ejecuta, pero esa forma de cancelar es problemática pues el hilo no llega a ejecutar la sentencia `return` de su función principal, así que no se destruyen sus objetos locales, no se cierran los descriptores de archivo que tuviera abiertos ni se libera la memoria que hubiera reservado. Quien quiera limpiar esos recursos tiene que registrar manejadores de limpieza que la librería ejecuta al morir el hilo.

C++20 incorporó al estándar un mecanismo distinto, llamado **cancelación cooperativa**. En lugar de matar al hilo, se le indica que debería terminar a través de un objeto `std::stop_token`, que el hilo comprueba de vez en cuando y, si la cancelación ha sido solicitada, retorna:

```
void thread_function(std::stop_token stop_token)                    ①
{
    while (!stop_token.stop_requested())
    {
        // Hacer una parte del trabajo...
    }
}

std::jthread thread1(thread_function);                              ②
```

① Si el primer parámetro de la función principal es un `std::stop_token`, `std::jthread` le entrega el suyo al arrancar el hilo.
② Por eso no hay que pasarle ningún argumento aquí.

El hilo **retorna normalmente**, así que se destruyen sus objetos locales y puede cerrar los descriptores y deshacer lo que llevara hecho, sin necesidad de ningún manejador de limpieza. A cambio, un hilo que no compruebe su *token* es incancelable, por mucho que se solicite su cancelación.

La cancelación se solicita llamando a `request_stop()` sobre el objeto `std::jthread`. Y no hace falta llamarlo explícitamente para cancelar un hilo al salir de un ámbito, porque **el destructor de `std::jthread` pide la cancelación de su hilo antes de esperar por él**.

Cuando quien debe pedir la cancelación no es el dueño del objeto `std::jthread`, sino otro hilo, se usa un `std::stop_source` independiente. Todas las copias de un mismo `std::stop_source`, y todos los *tokens* que se obtengan de él con `get_token()`, comparten el mismo estado.

En el siguiente ejemplo, la función de búsqueda recibe un último parámetro `std::stop_token`, que comprueba entre un elemento y el siguiente. Otro hilo cancela la búsqueda si tarda más de cinco segundos:

```
std::stop_source search_stop;

std::jthread timer(cancel_after, search_stop, 5s);                  ①
std::jthread worker(count_values, std::cref(values),
    0, values.size(), 42, std::ref(result),
    search_stop.get_token());                                       ②
```

① Este hilo recibe la fuente, para poder llamar a `request_stop()` sobre ella cuando expire el plazo.
② Y este el *token* que se obtiene de ella. Se pasa en último lugar para dejar claro que el hilo comprueba este *token*, y no el interno que `std::jthread` entrega a las funciones cuyo primer parámetro es un `std::stop_token`.

> **!** Importante
>
> En un programa así conviven dos mecanismos de cancelación independientes y conviene no confundirlos:
>
> - El `std::stop_source` propio, que un hilo usa para cancelar a otro.
> - El *token* que `std::jthread` le entrega a cada hilo, que solo se activa cuando se llama a `request_stop()` sobre ese objeto o cuando se destruye.

Dónde comprobar el *token* es una decisión de diseño. Hacerlo demasiado a menudo cuesta tiempo; hacerlo demasiado poco hace que el hilo tarde en reaccionar. Y hay que elegir puntos en los que el trabajo esté en un estado consistente. Es decir, entre dos operaciones completas, nunca en mitad de una.

G.5. Excepciones en los hilos

Si una excepción escapa de la función principal de un hilo, **no** se propaga al hilo que lo creó –cada hilo tiene su propia pila y no hay ningún camino entre ellas– sino que se llama a `std::terminate()` y el programa entero termina de forma anormal.

Por eso un `try`/`catch` en `main()` solo protege al hilo principal. Cada función principal de hilo necesita el suyo. Para devolver tanto el resultado como los posibles errores, se puede usar `std::expected` como tipo de `result`:

```
void count_values(const std::vector<int>& values,
    std::size_t first, std::size_t last, int wanted,
    std::expected<std::size_t, std::string>& result)
{
    try
    {
        std::size_t count = 0;

        for (std::size_t i = first; i < last; ++i)
        {
            if (values.at(i) == wanted)                    ①
            {
                ++count;
            }
        }

        result = count;
    }
    catch (const std::exception& error)
    {
        result = std::unexpected(error.what());            ②
```

```
    }
}
```

① `at()` lanza una excepción si el índice queda fuera del vector.
② La excepción se recoge y se guarda junto al resto de los resultados, para que quien creó el hilo pueda tratarla como cualquier otro fallo.

G.6. Hilos y procesos

Cuando un proceso con varios hilos llama a `fork()`, el hijo nace con **un solo hilo**: el que hizo la llamada. Pero hereda una copia de toda la memoria del padre, incluida la de los demás hilos y la de cualquier *mutex* que estuvieran usando en ese instante. Si uno de esos *mutex* estaba tomado, en el hijo seguirá tomado para siempre, porque el hilo que debía liberarlo no existe allí.

Por eso, en un programa con varios hilos, la recomendación de que el hijo haga lo imprescindible entre `fork()` y `exec()` se convierte en una regla estricta. Preparar sus descriptores con `dup2()`, restablecer su máscara de señales y llamar a `exec()` es seguro, porque ninguna de esas llamadas necesita tomar *mutex*. Reservar memoria, escribir por la salida estándar o construir objetos de la librería estándar, en cambio, puede bloquear al hijo indefinidamente.

G.7. Hilos y señales

Las señales se dirigen al **proceso**, no a un hilo concreto. Cuando llega una señal, el núcleo se la entrega a cualquiera de los hilos que no la tenga bloqueada, y no hay forma de saber a cuál. En un programa con varios hilos eso significa que la señal puede interrumpir precisamente al hilo que está en mitad de su trabajo.

El patrón habitual para evitarlo es bloquear las señales que interesan en todos los hilos y dedicar uno de ellos a esperarlas de forma síncrona con `sigwaitinfo()` o `sigtimedwait()`, tal y como se explica en la Sección H.5. Al estar bloqueadas en todos los hilos, ninguna señal se entrega de forma asíncrona y todas acaban recogidas por ese hilo, que puede tratarlas dentro del flujo normal del programa.

Cada hilo tiene su **propia máscara de señales**, pero un hilo nuevo hereda una copia de la máscara del hilo que lo creó. De ahí la regla fundamental:

> **!** Importante
>
> Las señales deben bloquearse en `main()` **antes de crear ningún hilo**. Así todos los hilos nacen ya con ellas bloqueadas. Si se bloquean después, existe una ventana de tiempo en la que la señal puede llegar a un hilo que todavía la tiene desbloqueada y terminar el proceso.

Para modificar la máscara de un hilo no se usa `sigprocmask()`, cuyo comportamiento en un programa con varios hilos no está definido por POSIX, sino `pthread_sigmask()`:

```
int pthread_sigmask(int how, const sigset_t *set, sigset_t *oldset);
```

Sus argumentos son exactamente los mismos que los de `sigprocmask()`, descritos en la Sección H.4.2, y actúa sobre el hilo que la llama.

> ⚠ Advertencia
>
> **`pthread_sigmask()` no usa `errno`.** Como el resto de las funciones de POSIX Threads, devuelve `0` si tiene éxito o directamente el código de error si falla, al contrario que la mayoría de las funciones de la librería del sistema. Aplicarle el patrón habitual de comprobar `errno` daría un resultado indefinido.

Así podría ser la preparación de la máscara al principio del programa:

```
sigset_t signal_set;

sigemptyset(&signal_set);
sigaddset(&signal_set, SIGINT);
sigaddset(&signal_set, SIGTERM);

int error = pthread_sigmask(SIG_BLOCK, &signal_set, nullptr);
if (error != 0)
{
    return std::unexpected(std::system_error(error,                    (1)
        std::system_category(),
        "No se pudieron bloquear las señales"));
}
```

(1) El código de error es el que ha devuelto la función, no el valor de `errno`.

La función principal del hilo que espera las señales es un bucle que las recoge una a una y actúa según cuál sea. La dificultad está en cómo termina ese bucle, porque un hilo detenido en `sigwaitinfo()` no sale de ahí hasta que llega una señal, y no mira mientras tanto ningún *token* de cancelación. Si alguien destruye su `std::jthread`, el destructor se quedará esperándolo para siempre y el programa no terminará nunca. Hay dos formas de resolverlo.

G.7.1. Esperar las señales con un plazo

La primera es sustituir `sigwaitinfo()` por `sigtimedwait()`, que abandona la espera al cabo de un plazo aunque no haya llegado ninguna señal. Así el hilo vuelve al principio del bucle cada poco tiempo y puede comprobar si le han pedido que termine:

```
void wait_for_signals(std::stop_token stop_token, sigset_t signal_set)
{
    while (!stop_token.stop_requested())
    {
        siginfo_t signal_info;
        timespec timeout{.tv_sec = 0, .tv_nsec = 100'000'000};

        int signo = sigtimedwait(&signal_set, &signal_info, &timeout);
        if (signo == -1)
        {
            if (errno == EAGAIN || errno == EINTR)                    (1)
            {
                continue;
            }
            // Tratar el error...
        }

        // Actuar según la señal recibida...
    }
}
```

(1) `EAGAIN` significa que expiró el plazo sin que llegara ninguna señal, así que no es un error. En ese caso el hilo vuelve al principio del bucle y aprovecha para comprobar su *token*.

El hilo se crea como cualquier otro y no hace falta nada más para pararlo, porque el destructor de `std::jthread` le pide que termine y lo espera:

```
{
    std::jthread signal_thread(wait_for_signals, signal_set);

    // El resto del trabajo del programa...
}                                                                     (1)
```

(1) Al salir del ámbito, el destructor solicita la parada del hilo y espera, como mucho, a que expire el plazo en curso.

La duración del plazo es un compromiso: cuanto más corto, antes se entera el hilo de que debe terminar, pero más veces se despierta sin motivo. Un valor del orden de décimas de segundo es imperceptible para quien usa el programa y apenas consume procesador.

G.7.2. Separar el hilo

La segunda es aceptar que el hilo se quede esperando en `sigwaitinfo()` y renunciar a esperarlo, marcándolo como separado con `detach()`:

```
std::jthread signal_thread(wait_for_signals, signal_set);
signal_thread.detach();                                          ①
```

① A partir de aquí el objeto ya no representa a ningún hilo, así que su destructor no intentará esperarlo.

El hilo sigue ejecutando su bucle hasta que termina el proceso, momento en el que desaparece con él, esté donde esté. Eso simplifica el código –no hay plazo que elegir ni *token* que comprobar– pero tiene un precio:

- El hilo no puede liberar nada por su cuenta al final, porque no hay cancelación. Nunca llega a retornar de su función principal, de modo que no se ejecutan los destructores de sus variables locales ni se cierran los descriptores que hubiera abierto.
- Quien lo creó no tiene forma de saber si está en mitad de algo. Si el hilo principal termina mientras el hilo separado atiende una señal, el trabajo a medias se pierde.

> **!** Importante
>
> A un hilo separado no se le deben pasar referencias a variables locales de la función que lo creó, ni siquiera a las de `main()`. Como nadie lo espera, esas variables pueden destruirse mientras él las sigue usando.

Por eso conviene separar solo a los hilos que no tengan nada que guardar ni que liberar, y que puedan desaparecer en cualquier instante sin dejar el proceso a medias. Si el hilo de las señales tuviera que hacer algo en el momento de terminar –avisar a los demás, borrar un archivo incompleto o esperar a que otro hilo acabe– la espera con plazo es la única de las dos opciones que lo permite.

H. Señales POSIX

Las señales son un mecanismo de comunicación entre procesos que permite notificar a un proceso que ha ocurrido un evento específico. En los sistemas POSIX, las señales se utilizan tanto para que el sistema operativo notifique a los programas ciertos errores y sucesos críticos, como para que los procesos se comuniquen entre sí.

H.1. Características de las señales

Las señales tienen las siguientes características:

- **Comunicación directa**: Se envían usando el identificador del proceso de destino (PID).
- **Tamaño y formato fijos**: Las señales solo portan la información de que ha ocurrido un evento, identificado por un número de señal. No pueden transportar datos adicionales.
- **Asíncronas por defecto**: Pueden interrumpir la ejecución del proceso en cualquier momento.

Cada señal tiene un efecto particular por defecto en el proceso que la recibe. Por ejemplo, muchas señales, por defecto, terminan el proceso que las recibe. Sin embargo, los procesos pueden cambiar el comportamiento ante una señal mediante:

- **Manejadores de señal**: Funciones que se ejecutan cuando llega la señal.
- **Bloqueo de señales**: Impedir temporalmente que ciertas señales se entreguen al proceso.
- **Espera síncrona**: Usar `sigwait()`, `sigwaitinfo()` o `sigtimedwait()` para esperar señales de forma controlada.

H.2. Señales comunes

Algunas de las señales más comunes del estándar POSIX son:

`SIGALRM` Enviada cuando expira un temporizador configurado con `alarm()` o `setitimer()`.

`SIGHUP` Enviada cuando termina la sesión de terminal asociada al proceso. En servicios del sistema se usa convencionalmente para indicarles que deben reiniciarse o recargar su configuración.

`SIGKILL` Termina inmediatamente el proceso. Esta señal no puede ser bloqueada ni ser manejada.

SIGINT Enviada cuando el usuario pulsa `Ctrl-C` en la terminal. Por defecto termina el proceso.

SIGPIPE Enviada a un proceso cuando intenta escribir en una tubería o *socket* cuyo extremo de lectura ha sido cerrado. Por defecto termina el proceso.

SIGSEGV Enviada cuando un proceso intenta acceder a memoria no permitida (violación de segmento).

SIGTERM Solicita al proceso que termine de forma ordenada. Es la señal por defecto enviada por el comando `kill`. El sistema operativo envía esta señal a todos los procesos cuando se apaga el sistema.

SIGUSR1 y SIGUSR2 Señales reservadas para uso del usuario. No tienen un significado predefinido, por lo que los programadores pueden usarlas con el propósito que deseen.

Se puede consultar la lista completa de señales soportadas en Linux en la página del manual "signal(7)".

H.3. Manejo de señales

Por defecto, cuando una señal llega a un proceso, este interrumpe inmediatamente el código que está ejecutando para ejecutar la acción por defecto de la señal (que frecuentemente es terminar el proceso). Sin embargo, es posible configurar un **manejador de señal**: una función de nuestro programa que será invocada por el sistema operativo cuando llegue una señal determinada.

H.3.1. Configurar manejadores con sigaction()

Para configurar un manejador de señal se utiliza la función `sigaction()`:

```
int sigaction(int signum, const struct sigaction *act,
    struct sigaction *oldact);
```

act Puntero a una estructura `sigaction` que describe cómo tratar la señal.
oldact Puntero donde se guardará la configuración anterior. Si es `NULL`, se ignora.

Retorna `0` si tiene éxito, o `-1` estableciendo `errno`.

La estructura `sigaction` tiene los siguientes campos principales:

sa_handler Puntero a la función de nuestro programa con la que queremos manejar la señal. Puede ser:

- Un puntero a una función con formato `void funct(int)`
- `SIG_DFL`: Para restaurar el comportamiento por defecto
- `SIG_IGN`: Para ignorar la señal

sa_mask Conjunto de señales a bloquear mientras se ejecuta la función manejadora (ver la Sección H.4.1). Esto evita que otras señales interrumpan la función manejadora mientras se está ejecutando.

sa_flags Opciones de configuración. Las más comunes son:

- `SA_RESTART`: Si la señal interrumpe una llamada al sistema, reanudarla automáticamente después del manejador.
- `SA_RESETHAND`: Restaurar el comportamiento por defecto después de ejecutar el manejador una vez.

Por ejemplo, para configurar un manejador para `SIGTERM` y `SIGINT`:

```
void signal_handler(int signum)
{
    switch (signum)
    {
        case SIGTERM:
            write(STDOUT_FILENO, "Recibida señal SIGTERM\n", 24);          ①
            break;

        case SIGINT:
            write(STDOUT_FILENO, "Recibida señal SIGINT\n", 23);
            break;

        default:
            write(STDOUT_FILENO, "Recibida señal desconocida\n", 28);
            break;
    }
}

int main()
{
    struct sigaction sa{};                                                ②
    sa.sa_handler = signal_handler;
    sigemptyset(&sa.sa_mask);                                             ③
    sa.sa_flags = 0;

    if (sigaction(SIGTERM, &sa, NULL) == -1)
    {
        // Error al configurar el manejador...
        return /* ... */;
    }

    if (sigaction(SIGINT, &sa, NULL) == -1)
    {
        // Error al configurar el manejador...
        return /* ... */;
```

```
    }

    // Código principal del programa...
    // Aquí se espera cinco minutos para poder enviarle señales
    // al proceso desde otra terminal
    sleep(300);

    return EXIT_SUCCESS;
}
```

① Se usa `write()` para mostrar el mensaje –en lugar de `std::cout`– porque las funciones de E/S estándar no son seguras en un manejador de señales (ver la Sección H.3.2).
② La estructura se inicializa a cero –con `{}`– para que ningún campo quede con un valor indeterminado.
③ La máscara `sa_mask` se inicializa vacía con `sigemptyset()` (ver la Sección H.4.1), así que ninguna otra señal se bloquea mientras el manejador se ejecuta.

H.3.2. Precauciones con manejadores de señales

Los manejadores de señal se ejecutan de forma asíncrona, interrumpiendo el flujo normal del programa. Esto puede causar problemas si el manejador:

- Modifica variables compartidas sin utilizar mecanismos de sincronización adecuados.
- Llama a funciones no reentrantes.
- Realiza operaciones que no son seguras en contextos de señales.

Por esta razón, se recomienda que los manejadores sean lo más simples posible y solo llamen a funciones marcadas como async-signal-safe. Igualmente, es aconsejable utilizar variables atómicas –como `std::atomic`– o mecanismos de sincronización adecuados para compartir datos entre el manejador y el resto del programa de forma segura.

Una alternativa más segura es usar `sigwait()`, `sigwaitinfo()` o `sigtimedwait()`, para manejar señales de forma síncrona, como se explica en la Sección H.5.

H.3.3. Patrón típico de terminación segura

Generalmente, queremos que la llegada de ciertas señales dispare acciones en el programa –como liberar los recursos y terminar de forma ordenada– pero no podemos realizarlas directamente desde el manejador de señal por las limitaciones mencionadas anteriormente. En su lugar, un patrón común es que el manejador de señal simplemente establezca variables atómicas indicando que se ha solicitado la terminación del programa o cualquier otra acción que nos interese. El programa debe verificar estas variables periódicamente y realizar las acciones necesarias de forma segura cuando detecte que se ha solicitado alguna.

Por ejemplo, aquí tenemos un ejemplo de este patrón para manejar señales de terminación y salir de forma segura del programa:

```
std::atomic<bool> quit_requested{false};                                    ①

void signal_handler(int signum)
{
    char msg[] = "Señal de terminación recibida, cerrando...\n";
    write(STDOUT_FILENO, msg, sizeof(msg) - 1);

    quit_requested = true;
}

int main()
{
    // Registrar signal_handler() para SIGTERM y SIGINT con sigaction(),
    // como en el ejemplo anterior...

    while (!quit_requested)                                                 ②
    {
        // Código principal del programa...
        // Aquí se puede hacer trabajo útil, dormir, etc.
    }

    return EXIT_SUCCESS;
}
```

① La variable debe ser **atómica**, porque la escribe el manejador –que se ejecuta de forma asíncrona– y la lee el bucle principal.
② El bucle comprueba la variable en cada vuelta; cuando el manejador la pone a `true`, sale y el programa libera sus recursos y termina de forma ordenada.

H.3.4. Señales y llamadas al sistema

Una cuestión común al trabajar con señales es cómo afectan a las llamadas al sistema que el proceso pueda estar ejecutando cuando llega una señal. Especialmente porque si el programa está bloqueado en una llamada al sistema (como `read()`, `write()`, `sleep()`, etc.) y llega una señal, puede que queramos que la llamada se interrumpa para que el programa pueda atender la señal. Por ejemplo, en el programa anterior, si el proceso está bloqueado en un `read()` y llega `SIGINT`, seguramente queremos que `read()` sea interrumpido para que el bucle principal pueda comprobar `quit_requested` y salir.

Por defecto, muchas llamadas al sistema se interrumpen cuando llega una señal. Solo en los casos en los que no queremos que esto ocurra, debemos usar el *flag* `SA_RESTART` al configurar el manejador de señal con `sigaction()`.

Las llamadas al sistema que son interrumpidas por una señal, terminan con error y establecen `errno` a `EINTR`. Por tanto, no debemos considerar esto como un error fatal, sino simplemente como una situación que debemos manejar adecuadamente, según las necesidades de nuestro programa. Si `errno` es `EINTR`, puede interesarnos ignorarlo y reintentar la llamada al sistema, o comprobar las variables establecidas por nuestros manejadores de señal –como `quit_requested`– para decidir qué tenemos que hacer a continuación.

H.4. Bloquear señales

Un proceso puede bloquear temporalmente la entrega de ciertas señales. Cuando una señal está bloqueada, queda pendiente hasta que se desbloquea, momento en el que se entrega al proceso.

Para bloquear y desbloquear señales se utiliza la función `sigprocmask()` que necesita un conjunto de señales (`sigset_t`) con las señales a bloquear o desbloquear.

H.4.1. Conjuntos de señales

Un conjunto de señales es una estructura de datos que representa un grupo de señales. Se manipula mediante las siguientes funciones:

- ```
  int sigemptyset(sigset_t *set);
  ```

  Inicializa un conjunto de señales vacío (sin ninguna señal). Retorna `0` en éxito, `-1` en error.

- ```
  int sigfillset(sigset_t *set);
  ```

  Inicializa un conjunto con todas las señales. Retorna `0` en éxito, `-1` en error.

- ```
  int sigaddset(sigset_t *set, int signum);
  ```

  Añade la señal `signum` al conjunto `set`. Retorna `0` en éxito, `-1` en error.

- ```
  int sigdelset(sigset_t *set, int signum);
  ```

  Elimina la señal `signum` del conjunto `set`. Retorna `0` en éxito, `-1` en error.

- ```
  int sigismember(const sigset_t *set, int signum);
  ```

  Verifica si la señal `signum` está en el conjunto `set`. Retorna `1` si está, `0` si no está, `-1` en error.
  ```

Por ejemplo, para crear un conjunto de señales que contenga solo `SIGTERM`:

```
sigset_t signal_set;

sigemptyset(&signal_set);
sigaddset(&signal_set, SIGTERM);
```

H.4.2. Bloquear y desbloquear con sigprocmask()

La función `sigprocmask()` permite cambiar la máscara de señales bloqueadas del proceso:

```
int sigprocmask(int how, const sigset_t *set, sigset_t *oldset);
```

`how` Especifica cómo modificar la máscara de señales bloqueadas:

- `SIG_BLOCK`: Añade las señales de `set` a las ya bloqueadas.
- `SIG_UNBLOCK`: Elimina las señales de `set` de las bloqueadas.
- `SIG_SETMASK`: Reemplaza la máscara de bloqueadas con `set`.

`set` Puntero al conjunto de señales a usar según `how`. Si es `NULL`, no se modifica la máscara (útil solo para consultar).

`oldset` Puntero donde se guardará la máscara anterior. Si es `NULL`, no se guarda.

Retorna `0` si tiene éxito, o `-1` estableciendo `errno`.

Por ejemplo, para bloquear las señales `SIGTERM` y `SIGINT`:

```
sigset_t signal_set;

sigemptyset(&signal_set);                                                    ①
sigaddset(&signal_set, SIGTERM);
sigaddset(&signal_set, SIGINT);

if (sigprocmask(SIG_BLOCK, &signal_set, NULL) == -1)
{
    // Error al bloquear señales...
    return /* ... */;
}

// Código del proceso que se ejecuta con SIGTERM y SIGINT bloqueadas...
```

① Se prepara el conjunto con las dos señales, como se vio en la Sección H.4.1.

Mientras que se pueden desbloquear todas las señales bloqueadas con:

```c
sigset_t signal_set;

sigfillset(&signal_set);

if (sigprocmask(SIG_UNBLOCK, &signal_set, NULL) == -1)
{
    // Error al desbloquear señales...
    return /* ... */;
}
```

> **!** Importante
>
> Bloquear señales no las ignora ni las descarta. Las señales bloqueadas quedan pendientes y se entregarán cuando se desbloqueen. Sin embargo, debemos tener en cuenta que las señales estándares pendientes no se acumulan. Si la misma señal llega múltiples veces mientras está bloqueada, solo se entregará una vez cuando sea desbloqueada.

H.4.3. Consultar las señales pendientes

La función `sigpending()` permite averiguar qué señales bloqueadas hay pendientes de entregar:

```c
int sigpending(sigset_t *set);
```

`set` es un puntero al conjunto de señales donde se almacenarán las que estén pendientes.

Retorna `0` si tiene éxito, o `-1` estableciendo `errno`.

A diferencia de las funciones de la familia `sigwait()` (ver la Sección H.5), no bloquea la ejecución del proceso esperando a que llegue una señal ni consume las señales pendientes. Por ejemplo, para comprobar si hay alguna señal de terminación pendiente:

```c
sigset_t pending_set;

if (sigpending(&pending_set) == -1)
{
    // Error al consultar las señales pendientes...
    return /* ... */;
}

if (sigismember(&pending_set, SIGTERM) ||                    (1)
    sigismember(&pending_set, SIGINT))
{
    // Se ha pedido terminar...
```

```
}
```

① `sigismember()` comprueba si una señal concreta está en el conjunto devuelto por `sigpending()`, como se vio en la Sección H.4.1.

Esto permite intercalar comprobaciones dentro de una tarea larga para, por ejemplo, abandonarla en cuanto se reciban determinadas señales, en lugar de enterarse al acabarla.

H.5. Esperar señales de forma síncrona

Como hemos comentado, las señales se entregan de forma asíncrona por defecto. Es decir, pueden interrumpir lo que esté ejecutando el proceso en cualquier momento. Sin embargo, es posible esperar señales de forma síncrona usando funciones como `sigwait()`, `sigwaitinfo()` o `sigtimedwait()`:

```
int sigwait(const sigset_t *set, int *sig);
int sigwaitinfo(const sigset_t *set, siginfo_t *info);
int sigtimedwait(const sigset_t *set, siginfo_t *info,
  const struct timespec *timeout);
```

Esto quiere decir, que en lugar de dejar que las señales interrumpan el flujo normal del programa, el proceso puede decidir cuándo quiere esperar y procesar las señales. Para ello, se utilizan estas funciones para consultar las señales pendientes y atenderlas de forma controlada, dentro del flujo normal de ejecución del programa.

Si al llamar a alguna de estas funciones no hay ninguna señal del conjunto `set` pendiente, la función bloquea la ejecución del proceso hasta que llegue una señal incluida en dicho conjunto.

De las tres funciones mencionadas, vamos a centrarnos en `sigwaitinfo()`. Sus argumentos son:

`set` Conjunto de señales por las que se quieren esperar. Debe contener las señales que queremos recibir.

`info` Puntero a una estructura `siginfo_t` donde se devolverá información detallada sobre la señal recibida. Esto permite al proceso saber detalles como qué señal ha llegado, para poder actuar en consecuencia, o qué proceso la envió:

```
typedef struct siginfo {
    int      si_signo;    // Número de señal
    int      si_code;     // Código de la señal
    pid_t    si_pid;      // PID del proceso que envió la señal
    uid_t    si_uid;      // UID del proceso que envió la señal
    // Otros campos específicos de la señal...
} siginfo_t;
```

Retorna el número de la señal recibida, o `-1` estableciendo `errno`.

> **!** Importante
>
> Para usar `sigwait()`, `sigwaitinfo()` o `sigtimedwait()` correctamente, las señales por las que se quiere esperar **deben estar bloqueadas** antes de llamar a la función. Si no están bloqueadas, las señales se entregarían de forma asíncrona en lugar de ser capturadas por estas funciones.

El siguiente ejemplo muestra cómo usar `sigwaitinfo()` para esperar las señales `SIGTERM` y `SIGINT`:

```
sigset_t signal_set;
siginfo_t signal_info;

sigemptyset(&signal_set);
sigaddset(&signal_set, SIGTERM);
sigaddset(&signal_set, SIGINT);

if (sigprocmask(SIG_BLOCK, &signal_set, NULL) == -1)                    ①
{
    // Error al bloquear señales...
    return /* ... */;
}

// Aquí el proceso puede realizar otras tareas...

int result = sigwaitinfo(&signal_set, &signal_info);
if (result == -1)
{
    // Error esperando señales...
    return /* ... */;
}

if (signal_info.si_signo == SIGINT)                                     ②
{
    std::println("Recibida señal SIGINT ({}) por el proceso {}",
      signal_info.si_signo, signal_info.si_pid);
}
```

① Bloquear las señales del conjunto para que `sigwaitinfo()` pueda capturarlas.
② Comprobar qué señal se ha recibido y actuar en consecuencia. En este caso, se comprueba si es `SIGINT` y se muestra un mensaje con su número y el PID del proceso que la envió.

> **!** Importante
>
> `sigwaitinfo()` también retorna `-1`, con `errno` igual a `EINTR`, si mientras espera llega una señal que no está en el conjunto y tiene un manejador instalado –por ejemplo, `SIGTERM` en un programa que use además el patrón de la Sección H.3.3–. No es un error real. El manejador ya se ha ejecutado, así que lo habitual es comprobar las variables que este haya podido establecer –como `quit_requested`– y volver a llamar a `sigwaitinfo()` si el programa debe continuar.
> Además, `sigwaitinfo()` es una de las llamadas que nunca se reinician automáticamente, aunque el manejador se haya instalado con `SA_RESTART`.

Usar las funciones de la familia `sigwait()` simplifica el código al evitar la complejidad de los manejadores de señales –donde habría que preocuparse por funciones reentrantes y condiciones de carrera– al permitir manejar las señales dentro del flujo normal del programa.

H.6. Enviar señales

Para enviar una señal a un proceso se utiliza la función `kill()`:

```
int kill(pid_t pid, int sig);
```

`pid` Identificador del proceso (PID) al que enviar la señal. Valores especiales:

- `> 0`: Envía la señal al proceso con ese PID.
- `0`: Envía la señal a todos los procesos del mismo grupo que el proceso actual.
- `-1`: Envía la señal a todos los procesos (requiere privilegios).

`sig` Número de la señal a enviar. El valor especial `0` no envía ninguna señal, pero verifica si el proceso existe y si tenemos permisos para enviarle señales.

Retorna `0` si tiene éxito, o `-1` estableciendo `errno`: normalmente `ESRCH` si no existe ningún proceso con ese PID, o `EPERM` si no tenemos permiso para enviarle señales.

Por ejemplo, para enviar la señal `SIGHUP` a un proceso con PID conocido:

```
pid_t other_pid = 12345;

// Enviar SIGHUP al servidor
if (kill(other_pid, SIGHUP) == -1)
{
    if (errno == ESRCH)
    {
        // Error porque el proceso no existe...
    }
    else if (errno == EPERM)
```

```
    {
        // Error de permisos para enviar la señal...
    }
    else
    {
        // Otro error...
    }
    return /* ... */;
}
```

H.7. Verificar si un proceso existe

Para verificar si un proceso con un PID determinado existe, se puede usar `kill()` con la señal `0`. El proceso no recibirá ninguna señal, pero, si la llamada a `kill()` retorna `-1` con `errno` igual a `ESRCH`, sabremos que el proceso no existe.

```
pid_t other_pid = 12345;

if (kill(other_pid, 0) == -1)
{
    if (errno == ESRCH)
    {
        // El proceso con PID 12345 no existe
    }
}
else
{
    // El proceso con PID 12345 existe
}
```

I. Operaciones con sockets

Un *socket* es una abstracción que representa un extremo de un canal de comunicación entre dos procesos. Cada proceso usa su *socket* para enviar y recibir mensajes a través de ese canal, sin tener que entrar en los detalles de cómo funciona el protocolo de red escogido ni de cómo se gestionan los dispositivos por los que viajan los datos. De eso se encargan el sistema operativo y el hardware, no el programador de la aplicación.

Los *sockets* se usan tanto para comunicar procesos que se ejecutan en máquinas distintas, a través de la red, como para comunicar procesos de la misma máquina, en cuyo caso son una alternativa a las tuberías con nombre.

I.1. Crear un socket

Un *socket* se crea con la función `socket()`, que retorna un descriptor de archivo con el que identificarlo en el resto de las operaciones:

```
int socket(int domain, int type, int protocol);
```

`domain` Es el dominio o familia de la dirección, es decir, la tecnología de comunicación que usará el *socket*: * `AF_INET`, para comunicarse por TCP/IP. * `AF_INET6`, para TCP/IP versión 6. * `AF_UNIX`, para comunicar procesos de la misma máquina a través de un archivo especial en el sistema de archivos.

`type` Es el tipo de *socket*, que determina cómo se entregan los mensajes: * `SOCK_DGRAM`, para comunicación por **datagramas**: mensajes independientes, cada uno con su destinatario, que conservan sus fronteras. Con `AF_INET` significa usar UDP. * `SOCK_STREAM`, para comunicación **orientada a conexión**: primero se establece una conexión con el otro extremo y luego los datos viajan como un flujo continuo de bytes, sin fronteras entre mensajes. Con `AF_INET` significa usar TCP.

`protocol` Es el protocolo concreto que se quiere usar dentro de esa familia. Tanto `AF_INET` como `AF_UNIX` solo admiten un protocolo para cada tipo de *socket*, así que este argumento siempre debe ser `0`.

Retorna el descriptor del nuevo *socket* o `-1` estableciendo `errno`:

```
int sock_fd = socket(AF_INET, SOCK_STREAM, 0);
if (sock_fd < 0)
{
    // Error al crear el socket...
```

```
    return /* ... */;
}
```

> **Nota**
>
> La diferencia entre los dos tipos de *sockets* importa al escribir el programa. Con `SOCK_DGRAM`, cada mensaje se recibe entero o no se recibe: un envío es una recepción o el mensaje se perdió. Con `SOCK_STREAM`, lo que llega es un flujo de bytes, así que un mensaje puede llegar partido en varias entregas, o pegado al siguiente, y es el programa el que tiene que saber dónde empieza y dónde acaba cada uno.

I.2. Direcciones de socket

Para que un proceso pueda mandarle un mensaje al *socket* de otro, el destinatario debe tener una dirección conocida. Su formato depende de la familia del *socket*.

La interfaz de *sockets* se diseñó para dar cabida a cualquier tecnología de comunicación, presente o futura, así que las funciones no reciben las direcciones en un formato concreto, sino un puntero a una estructura `sockaddr` genérica y el tamaño de la estructura real que hay detrás. Por eso siempre hay que convertir el tipo con `reinterpret_cast<sockaddr*>()`.

I.2.1. Direcciones `AF_INET`

En TCP/IP una dirección la forman la dirección IP de la máquina y un número de puerto entre 1 y 65535. Se representan con la estructura `sockaddr_in`, declarada en `netinet/in.h`:

```
struct sockaddr_in {
    sa_family_t     sin_family;     // = AF_INET
    in_port_t       sin_port;       // Puerto
    struct in_addr  sin_addr;       // Dirección IP
};

struct in_addr {
    uint32_t        s_addr;         // Dirección IP como entero de 32 bits
};
```

`sin_port` Es el número de puerto, que debe convertirse al orden de bytes de la red con `htons()`. Si vale `0`, el sistema operativo asigna un puerto libre cualquiera.

`sin_addr.s_addr` Es la dirección IP como entero de 32 bits, también en el orden de bytes de la red: * `inet_aton()` convierte una cadena como `"192.168.200.5"` en ese entero, y retorna `0` si la cadena no contiene una dirección válida. `inet_ntoa()` hace la conversión inversa, que es muy útil para los mensajes de error y depuración de los programas. *

`INADDR_LOOPBACK` es la dirección `127.0.0.1` de la interfaz local, a la que solo se puede acceder desde la propia máquina. * `INADDR_ANY` –la dirección `0.0.0.0`– indica que el *socket* debe escuchar en todas las direcciones IP de la máquina. Ambas constantes hay que convertirlas con `htonl()`.

> **!** Importante
>
> Que un servidor escuche en `INADDR_ANY` significa que cualquiera que pueda llegar por la red a esa máquina puede conectarse a él. Por seguridad, es mejor que el valor por defecto, mientras el usuario no indique otro valor, sea `INADDR_LOOPBACK`.

I.2.2. Direcciones `AF_UNIX`

En el dominio UNIX la dirección es la ruta de un archivo especial en el sistema de archivos. Se representan con la estructura `sockaddr_un`, declarada en `sys/un.h`:

```
struct sockaddr_un {
    sa_family_t     sun_family;     // = AF_UNIX
    char            sun_path[108];  // Ruta del socket
};
```

La ruta se copia en `sun_path`, que tiene un tamaño fijo de 108 bytes contando el terminador, así que **hay que asegurarse de que la ruta cabe** antes de copiarla.

El archivo lo crea el sistema al asignar la dirección al *socket* y no lo borra nadie cuando el proceso termina, así que:

- Antes de asignar la dirección hay que eliminar con `unlink()` el archivo que haya quedado de una ejecución anterior, o la operación fallará con `EADDRINUSE`.
- Al terminar, el programa debe eliminarlo también, para no dejar basura en el sistema de archivos.

Los permisos del archivo determinan quién puede conectarse o mandar mensajes al *socket*, igual que con cualquier otro archivo, y se ven afectados por la máscara `umask` del proceso.

I.3. Asignar una dirección al socket

Un *socket* recién creado no tiene dirección. Para que otros procesos puedan enviarle mensajes hay que asignarle una con `bind()`:

```
int bind(int sock_fd, const sockaddr* addr, socklen_t addrlen);
```

Retorna `0` si tiene éxito, o `-1` estableciendo `errno`, por ejemplo con `EADDRINUSE` si la dirección ya está ocupada por otro *socket*.

```
sockaddr_in local_address{};                                                    ①
local_address.sin_family = AF_INET;
local_address.sin_addr.s_addr = htonl(INADDR_LOOPBACK);
local_address.sin_port = htons(8080);

int result = bind(sock_fd,
    reinterpret_cast<const sockaddr*>(&local_address),                          ②
    sizeof(local_address));
if (result < 0)
{
    // Error al asignar la dirección...
    return /* ... */;
}
```

① La inicialización con `{}` pone a cero toda la estructura, que es lo recomendado antes de rellenar sus campos.
② `bind()` espera un puntero al formato genérico `sockaddr`, pero la dirección es un `sockaddr_in`.

> **Nota**
>
> Un *socket* que solo envía mensajes no necesita `bind()`, pues el sistema le asigna una dirección cualquiera en el primer envío. Pero entonces tampoco puede recibirlos, porque nadie conoce esa dirección.

I.4. Enviar y recibir datagramas

Con un *socket* `SOCK_DGRAM` cada mensaje se envía con `sendto()`, indicando en cada envío a quién va dirigido:

```
ssize_t sendto(int sock_fd, const void* buffer, size_t length, int flags,
    const sockaddr* dest_addr, socklen_t dest_len);
```

Los tres primeros argumentos son como los de `write()` –descriptor, datos y número de bytes–, `flags` será siempre `0`, y los dos últimos son la dirección del destinatario y su tamaño. Retorna el número de bytes enviados o `-1` estableciendo `errno`.

```
sockaddr_in dest_address{};
dest_address.sin_family = AF_INET;
dest_address.sin_addr.s_addr = htonl(INADDR_LOOPBACK);
```

```
dest_address.sin_port = htons(8080);

std::string message("¡Hola, mundo!");

ssize_t bytes_sent = sendto(sock_fd, message.data(), message.size(), 0,
    reinterpret_cast<const sockaddr*>(&dest_address),
    sizeof(dest_address));
if (bytes_sent < 0)
{
    // Error al enviar el mensaje...
    return /* ... */;
}
```

Y se recibe con `recvfrom()`, que además indica quién lo ha enviado:

```
ssize_t recvfrom(int sock_fd, void* buffer, size_t length, int flags,
    sockaddr* src_addr, socklen_t* src_len);
```

```
sockaddr_in src_address{};
socklen_t src_len = sizeof(src_address);                                    ①

std::string message;
message.resize(MESSAGE_MAX_SIZE);                                           ②

ssize_t bytes_read = recvfrom(sock_fd, message.data(), message.size(), 0,
    reinterpret_cast<sockaddr*>(&src_address), &src_len);
if (bytes_read < 0)
{
    // Error al recibir el mensaje...
    return /* ... */;
}

message.resize(bytes_read);                                                 ③
```

① A la entrada indica cuánto espacio hay en `src_address` y a la salida cuánto ha ocupado la dirección del emisor. Si no interesa saber quién envió el mensaje, se le pueden pasar dos punteros nulos.
② El tamaño del búfer lo decide cada programa, según el mensaje más grande que espere recibir.
③ Ajustar el tamaño de la cadena al número real de bytes recibidos.

`recvfrom()` se bloquea hasta que llega un mensaje, y retorna exactamente uno en cada llamada. Si el mensaje recibido es más grande que el búfer, la parte que no cabe **se descarta**, así que el búfer debe ser suficiente para el mensaje más grande que el programa pueda recibir.

I.5. Sockets orientados a conexión

Con `SOCK_STREAM` no se envían mensajes sueltos. Primero hay que establecer una conexión entre los dos *sockets* y después los datos viajan por ella como un flujo de bytes. El proceso que espera conexiones es el **servidor** y el que las inicia es el **cliente**.

I.5.1. Escuchar conexiones

El servidor asigna una dirección conocida a su *socket* con `bind()` y después lo pone a escuchar con `listen()`:

```
int listen(int sock_fd, int backlog);
```

`backlog` es el número máximo de conexiones que pueden quedar en la cola esperando a ser aceptadas. Si llega una conexión con la cola llena, se rechaza.

> **Nota**
>
> Cuando un servidor termina, su dirección queda reservada durante unos minutos, por si quedan datos en tránsito de la conexión anterior. Por eso, al reiniciarlo enseguida, `bind()` suele fallar con `EADDRINUSE`.
> Para evitarlo se activa la opción `SO_REUSEADDR` **antes** de llamar a `bind()`:
>
> ```
> int enabled = 1;
> setsockopt(sock_fd, SOL_SOCKET, SO_REUSEADDR,
>     &enabled, sizeof(enabled));
> ```

I.5.2. Aceptar conexiones

Con `accept()` el servidor recoge la siguiente conexión de la cola:

```
int accept(int sock_fd, sockaddr* addr, socklen_t* addrlen);
```

Se bloquea hasta que hay una conexión que aceptar y retorna **un descriptor nuevo**, que representa esa conexión concreta y es el que se usa para hablar con ese cliente. En `addr` deja la dirección del cliente, que es útil para los mensajes del servidor.

El *socket* que escucha no se cierra ni se usa para transferir datos, sino que se queda aceptando las siguientes conexiones. Por eso el servidor suele utilizar un bucle:

```
while (true)
{
    sockaddr_in client_address{};
    socklen_t client_len = sizeof(client_address);

    int client_fd = accept(sock_fd,
        reinterpret_cast<sockaddr*>(&client_address), &client_len);
    if (client_fd < 0)
    {
        // Error al aceptar la conexión...
        continue;
    }

    // Atender al cliente y cerrar client_fd al terminar...
}
```

I.5.3. Conectar con un servidor

El cliente no necesita `bind()`. Simplemente crea su *socket* y llama a `connect()` con la dirección del servidor:

```
int connect(int sock_fd, const sockaddr* addr, socklen_t addrlen);
```

Retorna `0` si la conexión se establece, o `-1` estableciendo `errno`. Los errores más frecuentes son:

- `ECONNREFUSED`: no hay nadie escuchando en esa dirección, normalmente porque el servidor no está en ejecución.
- `ENOENT`, en el dominio UNIX, si el archivo del *socket* no existe.
- `ETIMEDOUT` o `EHOSTUNREACH`, en TCP/IP, si no se puede llegar hasta la máquina indicada.

A partir de ahí, el descriptor del cliente representa la conexión y se usa igual que el que `accept()` le dio al servidor.

I.5.4. Enviar y recibir datos

Sobre una conexión se usan `send()` y `recv()`, que no necesitan direcciones porque el *socket* ya sabe con quién está hablando:

```
ssize_t send(int sock_fd, const void* buffer, size_t length, int flags);
ssize_t recv(int sock_fd, void* buffer, size_t length, int flags);
```

Se comportan como `write()` y `read()` sobre un archivo, y tienen exactamente las mismas dos consecuencias que se explicaron en la Sección A.5.1, que aquí son la norma y no la excepción:

- `send()` puede enviar **menos bytes** de los que se le pidieron, así que hay que llamarla en un bucle hasta colocar todo el mensaje.
- `recv()` puede retornar **menos bytes** de los que caben en el búfer, así que hay que llamarla en un bucle hasta reunir lo que se espera, con la misma forma que el de `read_data()`: acumular lo que se lleva recibido, avanzar esa misma cantidad en el búfer y descontarla de lo que queda por pedir.

Además, `recv()` retorna `0` cuando el otro extremo ha cerrado la conexión y ya no van a llegar más datos. Esto no es un error. Es el equivalente al fin de archivo de `read()`.

> **!** Importante
>
> Si el otro extremo cierra la conexión mientras enviamos, `send()` falla con `EPIPE` y el sistema envía la señal `SIGPIPE`, que termina el proceso por omisión. Es lo mismo que puede ocurrir al escribir en una tubería usando `write()`. Por eso es recomendable bloquear `SIGPIPE` al principio del programa o utilizar el *flag* `MSG_NOSIGNAL` en cada envío, para que falle con `EPIPE` sin enviar la señal.

I.5.5. Cerrar media conexión

Cuando un extremo termina de enviar pero quiere seguir recibiendo, puede cerrar solo su mitad de la conexión con `shutdown()`:

```
int shutdown(int sock_fd, int how);
```

Retorna `0` si tiene éxito, o `-1` estableciendo `errno`. Con `SHUT_WR` el otro extremo recibirá `0` en su `recv()`, sabrá que ya no llegan más datos y podrá responder por la conexión, que sigue abierta en ese sentido.

I.6. Esperar con un plazo

`accept()`, `recv()` y `recvfrom()` se bloquean indefinidamente mientras no llegue nada. Eso es un problema para un programa que tiene que poder terminar de forma ordenada. Por ejemplo, un programa bloqueado en `accept()` no comprueba su `std::stop_token`, ni ninguna otra condición, hasta que llegue una conexión.

La solución es esperar con un plazo usando `poll()`, y llamar a la función bloqueante solo cuando ya se sabe que hay algo que recoger:

```
int poll(struct pollfd* fds, nfds_t nfds, int timeout);
```

`fds` es un array de `nfds` estructuras `pollfd`, una por cada descriptor que se quiere vigilar:

```
struct pollfd {
    int     fd;         // Descriptor que se vigila
    short   events;     // Lo que se espera de él
    short   revents;    // Lo que ha ocurrido de verdad
};
```

En `events` se indica qué interesa de ese descriptor –`POLLIN`, que haya algo que leer– y `poll()` deja en `revents` lo que realmente ha pasado. Por eso, al esperar por varios descriptores a la vez, hay que recorrerlos al volver para ver **cuál** de ellos tiene su bit en `revents`.

Retorna el número de descriptores con algún suceso, `0` si expiró el plazo sin que ocurriera nada, o `-1` estableciendo `errno`.

```
pollfd descriptors[] = {{ .fd = sock_fd, .events = POLLIN }};

int result = poll(descriptors, 1, 100);                                    ①
if (result < 0)
{
    // Error al esperar...
    return /* ... */;
}
else if (result > 0)
{
    // Hay algo que recibir: accept(), recv() o recvfrom()
    // ya no se van a bloquear...
}
```

① El plazo se indica en milisegundos.

El mismo patrón mostrado para `sigtimedwait()` en la Sección G.7.1 sirve aquí: el hilo espera en `poll()` –en uno o en varios *sockets* a la vez– y, cada vez que expira el plazo, comprueba su *token* de parada antes de volver a esperar.

I.7. Cerrar un socket

Los descriptores de los *sockets* se cierran con `close()`, igual que los de los archivos, y hay que cerrarlos todos antes de terminar: el que escucha, el de cada conexión aceptada y el del cliente.

En el dominio UNIX, además, cerrar el *socket* no elimina su archivo del sistema de archivos: es necesario borrarlo con `unlink()`, tal y como se explicó en la Sección I.2.2.